Expert Systems With Applications 199 (2022) 116830

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Review

Reinforcement learning in urban network traffic signal control: A systematic literature review

Mohammad Noaeen [a,b,*], Atharva Naik [c,1], Liana Goodman [c,1], Jared Crebo [d,e,1], Taimoor Abrar [c,1], Zahra Shakeri Hossein Abad [f,2], Ana L.C. Bazzan [g,2], Behrouz Far [c,2]

[a] Harvard Business School, Harvard University, Boston, USA
[b] Department of Civil and Mineral Engineering, University of Toronto, Toronto, Canada
[c] Department of Electrical and Software Engineering, University of Calgary, Calgary, Canada
[d] Department of Mechanical and Manufacturing Engineering, University of Calgary, Calgary, Canada
[e] Department of Physics and Astronomy, University of Calgary, Calgary, Canada
[f] Department of Biomedical Informatics, Harvard University, Boston, USA
[g] Institute of Informatics, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brazil

ARTICLE INFO

Keywords:
Reinforcement learning
Traffic light control
Urban network
Multi-agent system
Intelligent transportation system
Artificial intelligence

ABSTRACT

Improvement of traffic signal control (TSC) efficiency has been found to lead to improved urban transportation and enhanced quality of life. Recently, the use of reinforcement learning (RL) in various areas of TSC has gained significant traction; thus, we conducted a systematic literature review as a systematic, comprehensive, and reproducible review to dissect all the existing research that applied RL in the network-level TSC domain, called as RL in NTSC or RL-NTSC for brevity. The review only targeted the network-level articles that tested the proposed methods in networks with two or more intersections. This review covers 160 peer-reviewed articles from 30 countries published from 1994 to March 2020. The goal of this study is to provide the research community with statistical and conceptual knowledge, summarize existence evidence, characterize RL applications in NTSC domains, explore all applied methods and major first events in the defined scope, and identify areas for further research based on the explored research problems in current research. We analyzed the extracted data from the included articles in the following seven categories: (i) publication and authors' data, (ii) method identification and analysis, (iii) environment attributes and traffic simulation, (iv) application domains of RL-NTSC, (v) major first events of RL-NTSC and authors' key statements, (vi) code availability, and (vii) evaluation. This paper provides a comprehensive view of the past 26 years of research on applying RL to NTSC. It also reveals the role of advancing deep learning methods in the revival of the research area, the rise of using non-commercial microscopic traffic simulators, a lack of interaction between traffic and transportation engineering practitioners and researchers, and a lack of proposal and creation of testbeds which can likely bring different communities together around common goals.

1. Introduction

With an explosion in urban and rural population rates, city transportation systems become less efficient at handling the ever-growing numbers of commuters. A lack of space and resources with which to improve infrastructure poses problems in accommodating the increasing urban population. The resulting congestion further leads to increased pollution caused by sitting idle in traffic jams, traffic delays and bottlenecks, and a rise in accidents. The secondary issues that arise are just as severe, including economic loss and an overall decrease in quality of life. This presents the problem of improving traffic flow and traffic signal control (TSC) within the already existing infrastructure.

Traffic signals are most often regularized through fixed time, actuated, or adaptive control methods, whether the state-of-the-art methods or the methods deployed in the real-world, such as SCATS (Sims et al., 1981), SCOOT (Hunt, Robertson, Bretherton, & Winton, 1981), and

* Corresponding author at: Harvard Business School, Harvard University, Boston, USA.
E-mail addresses: m.noaeen@utoronto.ca (M. Noaeen), atharva.naik@ucalgary.ca (A. Naik), liana.goodman@ucalgary.ca (L. Goodman), jared.crebo@ucalgary.ca (J. Crebo), taimoor.abrar@ucalgary.ca (T. Abrar), zahra_shakeri@hms.harvard.edu (Z.S.H. Abad), bazzan@inf.ufrgs.br (A.L.C. Bazzan), far@ucalgary.ca (B. Far).
[1] These authors contributed equally to this article.
[2] These authors share co-senior authorship.

https://doi.org/10.1016/j.eswa.2022.116830
Received 25 March 2021; Received in revised form 23 February 2022; Accepted 3 March 2022
Available online 17 March 2022
0957-4174/© 2022 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

TUC (Diakaki, Papageorgiou, & Aboudolas, 2002). Fixed time signal control involves a repeating pattern that does not change with the live traffic situation and which continues through its cycles regardless of dynamic traffic changes in that area. The actuated control method operates traffic signals based on real-time data of loop detectors. Despite being traffic responsive, the actuated control method is not designed to fully address fluctuating traffic demands, thereby rendering it less than optimal, specifically in highly saturated volumes. Conversely, an adaptive signal is a more efficient solution as it has the built-in capacity to adapt to traffic changes without the restrictions that plague the actuated method. Reinforcement Learning (RL) (Sutton, Barto, et al., 1998) is generally employed to advance the category of the adaptive methods in TSC. Traffic phenomena are highly dynamic and TSC involves high complexity in adjusting to different traffic conditions. Therefore, different methods are proposed to solve research problems in this area (Noaeen, 2021). These methods include (i) traffic theory based (Daganzo, 1995; Mohajerpoor, Saberi, & Ramezani, 2019; Noaeen, Mohajerpoor, Far, & Ramezani, 2021; Noaeen, Rassafi, & Far, 2016; Varaiya, 2013), (ii) simulation based (Chong & Osorio, 2017; Osorio & Selvam, 2017), and (iii) data-driven methods (Balaji, German, & Srinivasan, 2010; Noaeen & Far, 2019, 2020). RL fits in the third category.

An advantage of RL over conventional methods, e.g. traffic theory based and heuristic methods, is that RL can learn from the interaction with the environment via trial and error to take appropriate actions based on the feedback it receives from the environment, rather than relying on pre-defined rules which are often used in conventional methods. TSC is a complex problem of sequential decision making, where conventional methods may not sufficiently be suitable to compute optimal solutions. As such, TSC can be a good fit to the framework of Markov Decision Processes (MDPs) and RL. Furthermore, RL has shown superior performance over conventional methods, e.g. in the articles we reviewed in this paper. Nevertheless, RL has its own issues and challenges in solving TSC and the network-level TSC (NTSC) problems and these obstacles, which require to be solved before deployment in a real-world setup, have kept this research line active. These obstacles include computational complexity, generalization, heuristic setting of various parameters in RL, etc.

Due to the rising popularity of RL in TSC recently, specifically in NTSC, we aim to thoroughly characterize the existing research in the area of urban traffic networks where RL is applied and to provide a complete account of what has already been explored. To this end, we exclude the research that has only proposed or tested for single isolated intersection control. Thus, we only concentrate on the application of RL in the network-level TSC, called as RL in NTSC or RL-NTSC for brevity, that tested the proposed methods in networks with two or more intersections.

It is worth noting, however, that there are several surveys and review papers that do cover this area. For instance, Abdulhai and Kattan (2003), Bazzan (2009), Bazzan and Klügl (2014), Mannion, Duggan, and Howley (2016), Wei, Zheng, Gayah, and Li (2019, 2021), Yau, Qadir, Khoo, Ling, and Komisarczuk (2017) all present general reviews or surveys on TSC methods, compiling a list of the most recent methods and algorithms related to RL in TSC. Additionally, two very recent papers, Gregurić, Vujić, Alexopoulos, and Miletić (2020), Haydari and Yilmaz (2020), discuss the applications and opportunities regarding Deep Reinforcement Learning (Deep RL) in TSC, while a number of relevant studies that do not exclusively focus on RL in TSC, e.g. Eom and Kim (2020), Jácome et al. (2018), Liu (2007), Nguyen, Kieu, Wen, and Cai (2018), Tahilyani, Darbari, and Shukla (2013), Wang, Yang, Liang, and Liu (2018), Wang, Zhang, Liu, Dai, and Lee (2019), Zhao, Dai, and Zhang (2011) were found to exist. Nonetheless, to the best of our knowledge, there is no such *systematic literature review* aimed at examining the existing research of RL-NTSC. This research aims to: (i) collect all the existing relevant papers in the defined area and present a systematic, explicit, comprehensive, and reproducible review for identifying, evaluating, and synthesizing the existing body of the literature (based on the definition of a systematic literature review presented in Fink (2019) and Okoli and Schabram (2010)), (ii) provide statistical and conceptual knowledge based on the qualitative and descriptive data analysis on data extracted from the included articles to investigate what has been done in the area, which methods and techniques were used (alone or as a core or combined method in integration with other methods), and which patterns, trends, and information can be extracted using data analysis techniques from the data reviewed, (iii) show which methods and which NTSC application domains have still room to be more elaborated in further research, (iv) identify the major first events in RL-NTSC to get familiar with how the research novelties and contributions are temporally located in the course of research in the area, which is specifically well to find the very recent research problems, (v) explore the recent research problems and domains with their frequencies that helps identify potential future research, (vi) provide common future directions based on what the included papers recommended, (vii) summarize existence evidence, and (viii) identify and summarize areas for further research.

Our paper scrutinizes several aspects that may have not been provided by the other existing articles in the literature. For instance, none of them provide overview of the "community" but only "science". Countries, departments, and affiliations of authors are examined which finds the collaborative aspects between disciplines and with industry. Furthermore, the older literature review articles provide basis but the field has moved on significantly, while some of the newer ones do not review older research works. In contrast, our paper collects and studies all articles in the literature within the defined scope. From the included papers, data were extracted and grouped into the following seven categories, which covers a wide range of dimensions and attributes:

(i) *Publication and authors' data:* publication venue, type of publication venue, year of publication, number of publications, authors' country, number of authors, authors' university department, and authors' affiliation,
(ii) *Method identification and analysis:* identifying core and combined RL/non-RL methods, control method scheme, NTSC method (centralized, hierarchical, and decentralized/distributed methods), RL methods' components and types including elements in state definition, elements in reward definition, types of actions, action selection methods and parameters, state- and action-space discretization, tabular and approximation based methods, and deep reinforcement learning based methods),
(iii) *Environment attributes and traffic simulation:* number of intersections, road network type, type of map, location of real-world map, lanes and turns configuration, data collection method, traffic simulator, saturation and spillback considerations,
(iv) *RL-NTSC application domains:* the NTSC application domains (or scenarios) and the frequency of publications in each domain based on the defined control method scheme and environment attributes,
(v) *Major first events in RL-NTSC and authors' key statements:* an infographic timeline to identify past and current trends in the research areas,
(vi) *Code availability:* availability of code or implementation,
(vii) *Evaluation:* presence of evaluation, compared methods, and performance measures.

For convenience, Table 1 summarize the main acronyms in RL and NTSC domains. The rest of this paper is organized as follows: Section 2 provides an overview of RL and its basics; Section 3 will present the methodology used to conduct this review, including the search strategy, selection criteria, and data extraction; Section 4 delves into the results and findings presented in section one; Section 5 discusses common future works and key findings; Section 6 covers threats to the validity of our work; and Section seven wraps up with a look at future implications of the current paper.

Table 1
Summary of abbreviations in RL and NTSC.

Acronym	Description
AC	Actor-Critic
AV	Autonomous Vehicle
ADP	Approximate Dynamic Programming
BPNN	Back Propagation Neural Networks
CAV	Connected and Autonomous Vehicles
CBR	Case-Based Reasoning
CMAC	Cerebellar Model Articulated Controller
CMOP	Correlated Multi-Objective Problem
CNN	Convolutional Neural Networks
CPT	Cumulative Prospect Theory
CQF	Connectionist Q-learning Framework
CRFs	Conditional Random Fields
CTM	Cell Transmission Model
DAE	Denoising Auto Encoder
DDQ	Double Deep Q-learning
Deep CNN	Deep Convolutional Neural Networks
Deep NN	deep Neural Networks
Deep RL	Deep Reinforcement Learning
DNN	Dueling Neural Networks
DP	Dynamic Programming
DQF	Decomposable Q-function
FFNN	Feed Forward Neural Networks
FNN	Fuzzy Neural Networks
GA	Genetic Algorithms
GAN	Graph Attentional Network
GCN	Graph Convolutional Network
GT	Game Theory
I2V	Infrastructure-to-Vehicle
IGMN	Incremental Gaussian Mixture Network
KNN	K-Nearest Neighbors
LA	Learning Automata
LC	Linear Combination
LCS	Learning Classifier System
LIME	Local Interpretable Model-Agnostic Explanations
Linear FA	Linear Function Approximation
LMT	Linear Model Tree
LR	Linear Regression
LSTM	Long Short-Term Memory
LTL	Linear Temporal Logic
MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process
ML	Machine Learning
MOPSO	Multi-Objective Particle Swarm Optimization
MP	Max Pressure
NFQI	Neural Fitted Q-iteration
NLP	Natural Language Processing
NN	Neural Network
Pro-CEP	Proactive Complex Event Processing
RBF	Radial Basis Functions
RL-CD	Reinforcement Learning with Context Detection
RM	Ramp Metering
RNN	Recurrent Neural Networks
SF	Smoothed Functional
SHAP	Shapley Additive exPlanation
SI	Signalized Intersection
SPSA	Simultaneous Perturbation Stochastic Approximation
SR	Sigmoid Regression
TD	Temporal Difference
TF	Triangular-shaped Functions
V2I	Vehicle-to-Infrastructure
V2V	Vehicle-to-Vehicle
VMS	Variable Message Signs
WNN	Wavelet Neural Networks

2. Background

Derived from the natural learning processes observed in animals, RL allows a TSC system to learn and adapt to their environment. In a traffic environment, several components, such as pedestrians, drivers, vehicles and traffic signals may interact with each other. In TSC, traffic signals are the most common agents. Some TSC systems have a single agent in the RL environment, however it is common to have multiple agents work either cooperatively or competitively, in what is called Multi-Agent Reinforcement Learning (MARL). The benefit here is that agents work across a large environment while still having the precision of a single agent or close to it. The agents interact in a simulated traffic environment in different situations to learn the optimal way of interacting with the environment in a real-world setup. RL works based on a reward system that promotes long-term goals in an environment. The learning process is a feedback cycle of state, action and reward, where RL learns how an agent should map the states to actions to maximize a reward (this is discussed in detail in Section 2.1.). See Fig. 1. The action often involves setting the phase duration. However, there are other action types, including setting the phase order, cycle time, offset, etc. We define these TSC-related terms in the following.

A phase is defined as a period of time during which a set of non-conflicting traffic movements receive a green signal. A cycle is composed of several phases and cycle time is the time required to complete a full sequence of the phases. The proportion of the cycle that is green is called split. Moreover, in a coordinated system, offset is defined as the time that the green phase at an intersection begins after the beginning of green time of the reference signal. The main goal of TSC is to improve the environment or network performance (e.g. delay time, travel time, queue length, and speed) by controlling the actions of the agents. The main focus of this paper is on controlling the timing of traffic signal agents, although this control can be integrated with the control of actions of other types of agents, like vehicles in a connected vehicle environment.

2.1. Reinforcement learning fundamentals in traffic signal control

A Markov Decision Process is a mathematical framework well suited to optimize decision-making processes under uncertainty. An MDP is a four-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T} \rangle$, including, respectively, state space $\mathcal{S}$, action space $\mathcal{A}$, reward function $\mathcal{R}$, and transition function $\mathcal{T}$. An MDP satisfies the Markov Property if the transition function, whether known or unknown, depends only upon the current state and the action taken, not on the sequence of events that preceded it.

If the reward and transition functions are known, the optimal policy can be found using Dynamic Programming (DP) methods via the recursive definitions of the value function. However, when the environmental dynamics are not known, i.e. reward and transition functions are unknown, the agent has to estimate the value of taking action in a state without using knowledge about the reward function and transition probabilities. In this situation, RL is suitable. RL can be model-based where the agent samples from the environment to estimate the reward and transition functions and find an optimal policy. Unlike model-based RL, in model-free RL algorithms the agent directly estimates the Q-function (Q-function is discussed later in this section) from experience while the reward and transition functions are unknown beforehand.

In model-free RL, an agent tries to learn the optimal way of interacting with an environment. RL learns how an agent should map the states to actions to maximize a numerical reward. At each time step (or decision point) k, based on a policy π that is intended to be optimized during the learning process towards reaching an optimal policy π^*, the agent takes action a_k from a set of possible actions $\mathcal{A}$ in response to the current state s_k from a set of possible states $\mathcal{S}$; i.e. $a_k = \pi(s_k)$. Simply put, a policy is a rule that the agent follows in selecting actions based on its current state. At the end of step k, the agent receives a reward r_k from the environment based on a reward function $\mathcal{R}$ where the elements of the reward can be collected through sensors. A sequence of state, action, and reward is a history of an agent saved in memory. At each time step, the RL agent tries to learn an optimal policy from its history of interactions with the environment that maximizes the discounted cumulative reward:

$$\mathcal{R}_k = \sum_{\phi=k}^{\infty} \gamma^{(\phi-k)} r_\phi = r_k + \gamma r_{k+1} + \gamma^2 r_{k+2} + \cdots = r_k + \gamma(r_{k+1} + \gamma r_{k+2} + \cdots)$$

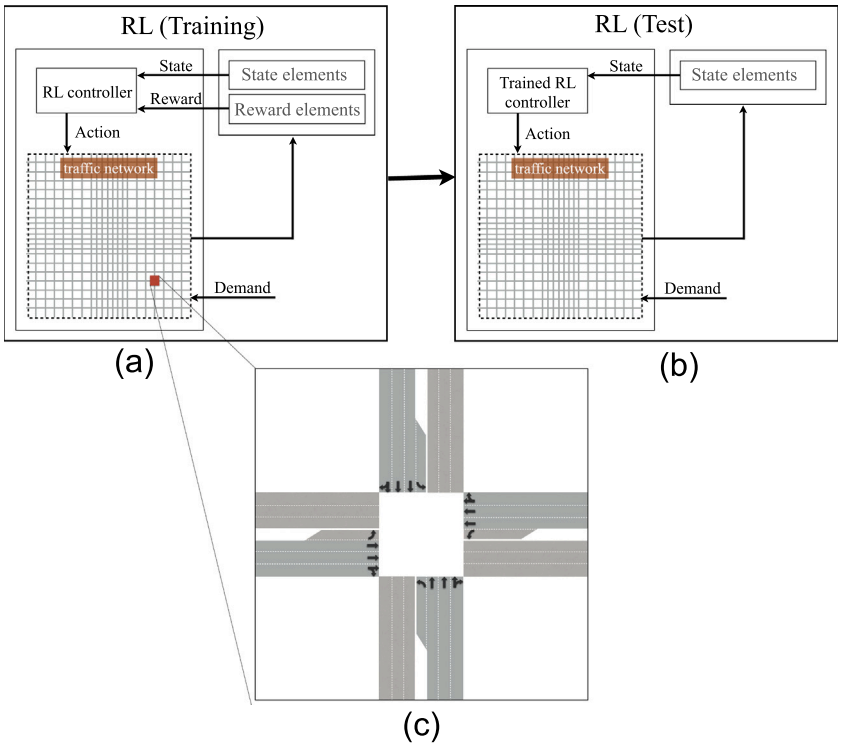


Fig. 1. A general RL process: (a) RL training, (b) RL testing, and (c) a sample intersection.

$$= r_k + \gamma \mathcal{R}_{k+1}, \tag{1}$$

where $\gamma \in [0,1]$ is a discount factor. The discount factor is associated with time horizons and is used to balance immediate and future rewards. It determines how much the RL agent cares about rewards in the distant future compared to those in the immediate future. If γ is 0, the agent only cares about the most immediate reward ($\mathcal{R}_k = r_k$). If γ is 1, the reward is not discounted and the distant future reward is considered ($\mathcal{R}_k = r_k + r_{k+1} + r_{k+2}$+...). As we set γ closer to 1, future rewards are given greater emphasis relative to the immediate reward. For more details, see Sutton and Barto (2018).

One of the most frequently used and successful RL methods in traffic signal control is Q-learning (Sutton & Barto, 2018), which was first investigated in 1989. Q-learning is a model-free RL. It is also an off-policy RL algorithm that uses a different policy for estimating Q-values than for action-selection. It updates the Q-values of the current state–action pair using the greedy policy to estimate the Q-value of the optimal policy of the next state–action pair. In other words, the optimal policy π^* is learned by estimating a second function, called Q-function, that specifies the value of an action (following a given policy π) given the current state. Q-function calculates the quality of a state–action combination. Assuming the agent continues to follow the optimal policy, the Q-value is defined as the expected discounted future reward of taking action a_k in state s_k:

$$Q^{\pi}(s_k, a_k) = \mathbb{E}_{\pi}[\mathcal{R}_k | s_k, a_k] = \mathbb{E}_{\pi}[\sum_{\phi=k}^{\infty} \gamma^{(\phi-k)} r_{\phi} | s_k, a_k]. \tag{2}$$

The Q-value is estimated by iterative Bellman updates:

$$Q^{\pi}(s_{k+1}, a_{k+1}) = Q^{\pi}(s_k, a_k) + \alpha(\Psi_k - Q^{\pi}(s_k, a_k)), \tag{3}$$

where $\alpha \in [0,1]$ is the learning rate that is set through experimentation, and $\Psi_k = r_k + \gamma \max_{a_{k+1}} Q^{\pi}(s_{k+1}, a_{k+1})$ is the target. Hence:

$$Q^{\pi}(s_{k+1}, a_{k+1}) = Q^{\pi}(s_k, a_k) + \alpha[r_k + \gamma \max_{a_{k+1}}[Q^{\pi}(s_{k+1}, a_{k+1})] - Q^{\pi}(s_k, a_k)], \tag{4}$$

Q-learning uses Q-table to decide which action to take. A simply put summary of Q-learning (Sutton & Barto, 1998) shows the algorithm to follow these steps:

- s_k is the current state in which the agent resides.
- The agent chooses action a_k from the available or acceptable actions.
- In response, the agent receives a reward r_k for action a_k and the next state s_{k+1}. As already discussed, this state s_{k+1} is merely a projection or estimation of the next state, rather than a known value.
- $Q(s_{k+1}, a_{k+1})$ is then updated using Eq. (4).
- The entire process is repeated.

Here, we should note that $\mathcal{R}_{k+1}$ in Eq. (1) is the reward that is obtained at the next time step (after taking action), but it is not necessarily known upfront. That is why Q-learning is a model-free RL that does not know about the model of the environment, i.e. the reward and transition functions.

In the learning process, there is the exploration–exploitation dilemma. The agent tries to exploit based on what it already learned to achieve the reward, and at the same time, it also must explore possible actions for each state to find the one that has received the highest reward for exploiting it.

3. Review method

In this study, search strategy, inclusion and exclusion criteria, and data extraction are intertwined, and the selection criteria and data extraction are performed within the search and search evaluation steps; hence, we explore this as a whole rather than separately. It should be noted that to identify the relevant literature we conducted both manual and automated searches, and included all articles published up to the end of March 2020. We designed and implemented our review based on the guidelines provided by Tricco et al. (2018).

3.1. Search strategy, selection criteria, and data extraction

To identify the most appropriate search terms and strings for the automated search, we used literature review, manual content analysis, and Natural Language Processing (NLP) in the steps that follow.

Table 2
Details of the search process.

Journal/Conference	#QGS$_{(2017–2019)}$
IET Intelligent Transport Systems	2
Journal of Intelligent Transportation Systems	3
Engineering Applications of Artificial Intelligence	2
IEEE Transactions on Intelligent Transportation Systems	2
IEEE Transactions on Vehicular Technology	1
International Conference on Intelligent Transportation Systems (ITSC)	2
ACM International Conference on Information Knowledge Management (CIKM)	3
Total	15

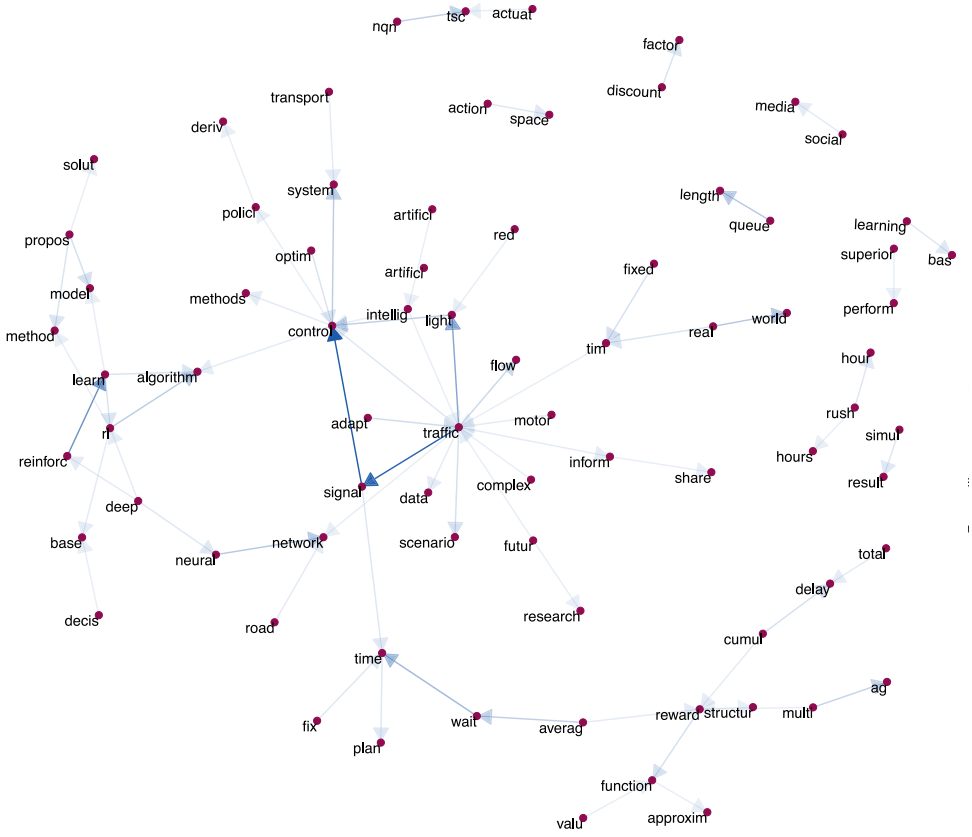


Fig. 2. Directed graph of common bi-grams resulted from the NLP that is used to define the search strings.

1: Based on knowledge in the area of the study, we identified seven venues, including five journals and two conferences as listed in Table 2, and manually searched for and reviewed the relevant published studies from 2017 to 2019 based on their title, abstract, and keywords. In very limited cases, we also examined the conclusion and searched for specific, relevant key terms to help with the inclusion decision. Fifteen pertinent articles were found during this first stage search. These papers were also used in the fourth step to form the Quasi-Gold Standard (QGS) (Abad, Noaeen, & Ruhe, 2016) to evaluate the quality of the search strings.

2: We performed NLP, including language modeling and lexical association analysis (Abad, Gervasi, Zowghi, & Far, 2019), to extract the most frequently used terms in the 15 retrieved articles for the purpose of identifying search strings. Fig. 2 shows the directed graph of common bi-grams formed from the QGS set, based on the frequency analysis on the pre-processed texts collected from the title, abstract, introduction, section/subsection headings, and conclusion of the 15 articles. Based on the results from the NLP, several inspections and investigations, and using various combinations of the most related search terms, the following single search string was chosen: ⟨*reward AND action AND (traffic light OR traffic signal) AND reinforcement learning*⟩.

3: Using the identified search string we queried Google Scholar, which served as our main search base, yielding 2887 articles. To increase the reliability of our findings, the search was complemented by searching within Web of Science, IEEE Xplore, ACM Digital Library, Springer Link, and Science Direct databases after the data extraction. This accelerated our search process by providing a view of those papers that were included and excluded, based on the defined criteria.

4: We formed the Quasi-Gold Standard. Our automated search retrieved all 15 articles, indicating the quasi-sensitivity of 100%.

5: We began the process of inclusion and exclusion by carefully defining the selection criteria. This iterative process continued even during data extraction, (i.e. the last step) to ensure the correctness of the included and excluded papers. Fig. 3 depicts the flowchart of the article selection process in this paper. Publications that meet the following criteria were excluded: (i) those articles not related to RL-NTSC, (ii) duplicate papers, (iii) review and survey papers, (iv) presentations, abstracts, extended abstracts, viewpoints, letters, reports, technical reports, projects, table of contents, any papers that have not been peer reviewed, theses and dissertations, books, and book chapters, (v) publications in any language other than English, (vi) those papers whose better or updated version was found in another journal

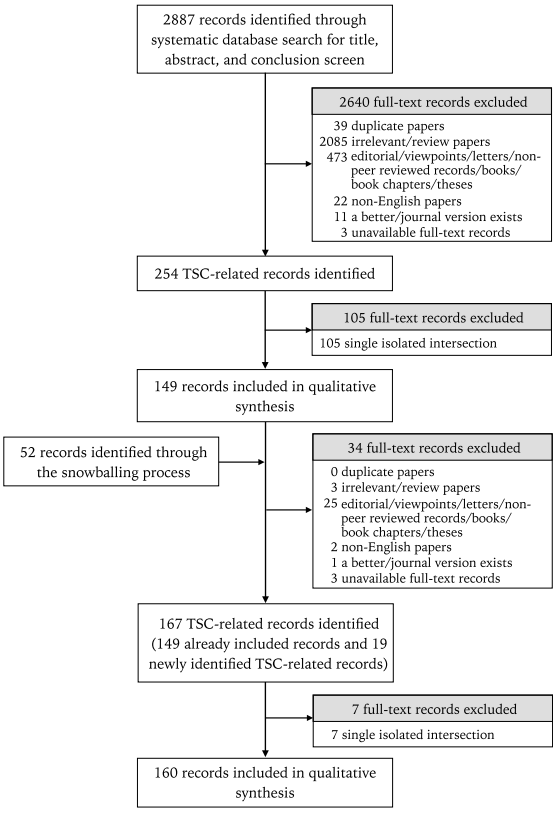


Fig. 3. The flowchart of systematic literature review process.

or publication and used, and (vii) the articles whose full-text are not available on the internet.

Since the scope of our research is network-scale, i.e. any scales involving two or more intersections, another important exclusion criterion used was those papers that propose or test a proposed method in a single isolated intersection only.

We included those papers containing an RL method, whether it was the core or combined method applied in the context of NTSC. These were included whether or not they provided evaluation and simulation, and if they provided only a framework.

This investigation is designed to cover the papers that apply RL in controlling traffic signals, and as such, if RL is used to control only vehicles, it is out of scope for our paper. In essence, as long as RL is being applied in NTSC, whether vehicle control is involved or not, the paper was included in the current study. Hence, the connected and autonomous vehicle (CAV) environment is covered in our paper as long as NTSC is involved. The CAV in the current study is inclusive of all environments of cases of CAV, including: vehicle-to-vehicle (V2V), vehicle-to-infrastructure (V2I), and infrastructure-to-vehicle (I2V).

Despite the fact that below-mentioned research areas found in articles during our search process have traffic components (including: public transit, bikes, and pedestrians) in common with our research, they are out of scope since TSC is not their main focus as it is in the current study. The research areas include: ramp metering, freeway, traffic control (not TSC), public transit where the focus is on bus control and TSC is excluded, route choice, routing systems, pedestrian routing, reactions of cyclists to speed advice, ride-sharing, best path selection, lane changing, traffic congestion detection, driver behavior, traffic signal control simulation environment, online calibration, traffic assignment problems, couriers management in express systems, fleet management, toll plaza, traffic control architecture based on fog computing paradigm where the focus is on fog paradigm (not TSC), image-based learning, and optimizing the sensor installation locations in a traffic network. Nevertheless, we found the articles with the topics, including ramp metering, public transit, emergency vehicles, and fog computing in NTSC, in which NTSC is included; thus we included these studies.

6: After applying the selection criteria and identifying the included papers, we used both, backward snowballing by screening the reference list of the included papers, and forward snowballing by scanning the citation of the included papers (Greenhalgh & Peacock, 2005) to reduce the probability of missing some papers in our inclusion process; see Fig. 3.

7: In addition to Google Scholar, five other electronic databases were searched, yielding no new articles. All articles found here had already been covered in the search using Google Scholar and the snowballing method.

8: From the included papers, data were extracted and analyzed.

In the following section, we provide an analysis of the data we extracted. In Table 3, we present an overall comparison along with details of some of the features extracted for all 160 papers included in our study. It should be noted that RL methods in Table 3 in the second column are classified into three main categories: model-free, model-based and RL. In this table, for model-free RL methods we present the specific model-free RL method that is used, such as Q-Learning, SARSA, etc. However, the model-based RL methods are shown as "Model-based RL". Also, in this table, by "RL" we mean to represent RL in general, where no specific RL method is mentioned in the paper. Moreover, there are articles that do not provide any specific RL method rather they mention the temporal difference (TD) update strategy. In this case, we report the method as TD as presented in those papers.

We also developed a visual dashboard (accessible at https://rl-ntsc.herokuapp.com) to complement the results and findings of this study.

4. Results

4.1. Publication and authors' data

4.1.1. Year, number, and venue of publications, and number of authors

As shown in Fig. 4, the number of studies regarding RL-NTSC is increasing. This topic continues to gain momentum, and hence, importance as the world and specifically, urban populations increase and this is demonstrated in the large number of papers published recently. During the first 15 years, from 1994 to 2008, a total of 22 papers on the subject were published, compared to 27 papers in the year 2019 alone. These statistics further attest to the fact that an in-depth review like ours is important at this time as it provides future works a comprehensive view of the past 26 years.

The 160 included articles have been published in 104 different publication venues, including conferences (57%), journals (39%), and workshops and symposia (4%), see Fig. 5. The first (Mikami & Kakazu, 1994) and second (Cao et al., 1999) articles in RL-NTSC were published in 1994 and 1999, respectively. These papers proposed model-free RL methods, while the first model-based RL method in RL-NTSC (Wiering, 2000) was proposed in 2000. Among the 160 included papers, 6 papers (Houli et al., 2010; Khamis & Gomaa, 2012, 2014; Khamis et al., 2012; Kuyer et al., 2008; Wiering et al., 2004) used or extended this model-based method, called as Wiering method. In addition to these Wiering method based research works, Da Silva et al. (2006) proposed a model-based method that is based on a mechanism for creating, updating and selecting one among several partial models of the environment.

4.1.2. Countries, departments and affiliations

Fig. 6 exhibits our data of the distribution of research papers among each country. 160 papers from a total of 30 different countries were included in this review, with most of the research papers coming from 7 countries: China, USA, India, Iran, Ireland, Canada, and Brazil. These papers represent 67% of our pool of papers, strongly suggesting that

Table 3
Overall comparison of the included articles. For the options in columns 6 (Action type) and 8 (Spillback or Sat.), see Table 6 and Section 4.3.2, respectively. In column 7, ✓ (i.e. yes) and ✗ (i.e. no), respectively, indicate whether discretization is considered or not.

Citation	RL method	Combined method	Approx. method	Action selection	Action type	Discretization	Spillback or Sat.
(Mikami & Kakazu, 1994)	LA	Genetic Alg.	–	–	7	✗	Sat
(Cao, Ireson, Bull, & Miles, 1999)	LCS	Fuzzy	–	Credit Assignment	7	✗	Neither
(Cao, Ireson, Bull, & Miles, 2000)	LCS	–	–	Credit Assignment	7	✗	Sat
(Wiering, 2000)	Model-based RL	–	–	ϵ-greedy	7	✗	Sat
(Camponogara & Kraus, 2003)	Q-learning	–	–	ϵ-greedy	7	✗	Sat
(Choy, Srinivasan, & Cheu, 2003)	RL	Fuzzy, Evol. Alg., NN	FNN	–	5	✓	Sat
(Wiering, Vreeken, Van Veenen, & Koopman, 2004)	Model-based RL	–	–	–	5	✗	Neither
(Dowling, Cunningham, Harrington, Curran, & Cahill, 2004)	RL	–	–	Greedy	7	–	No evaluation
(Ling & Shalaby, 2005)	Q-learning	–	IA	ϵ-greedy	6	✓	Neither
(Da Silva, Basso, Bazzan, & Engel, 2006)	Model-based RL	–	–	–	3	✓	Sat
(Srinivasan, Choy, & Cheu, 2006)	RL	Fuzzy, Evol. Alg., NN	SPSA, FNN	–	7	✓	Sat
(de Oliveira et al., 2006)	RL	–	–	–	3	✓	Sat
(Richter, Aberdeen, & Yu, 2007)	Actor-Critic	–	Linear and Non-linear FAs	Softmax	7	✓	Spillback
(Su & Tham, 2007)	Q-learning	SensorGrid	CMAC	Softmax	6	✗	Neither
(Zhang, Aberdeen, & Vishwanathan, 2007)	Policy-Gradient	CRFs	–	–	7	✗	Sat
(Bazzan, Klügl, & Nagel, 2007)	LA	–	–	–	Multiple	✗	Sat
(Ritcher, 2007)	Policy-Gradient	–	Linear FA	–	7	✓	Neither
(Kuyer, Whiteson, Bakker, & Vlassis, 2008)	Model-based RL	Max-plus	–	ϵ-greedy	7	✓	Sat
(Salkham, Cunningham, Garg, & Cahill, 2008)	Q-learning	–	–	Softmax	5	✓	Neither
(Li, Zhao, & Yi, 2008b)	ADP	–	FFNN	–	5	✗	Neither
(Lu, Liu, & Dai, 2008)	Q-learning	Multiband	–	ϵ-greedy	6	✗	Sat
(Li, Zhao, & Yi, 2008a)	ADP	Neuro-fuzzy	–	–	3	✗	Sat
(Xinhai & Lunhui, 2009)	Q-learning	–	–	–	7	–	No evaluation
(Cai, Wong, & Heydecker, 2009)	ADP	–	Linear FA	–	7	✗	Sat
(Dusparic & Cahill, 2009a)	W-learning	–	–	DWL	7	✗	Sat
(Zhao, Li, Yu, & Shang, 2009)	Q-learning	GT	BPNN	–	7	✗	Sat
(Dusparic & Cahill, 2009c)	W-learning	–	–	DWL	7	✓	Sat
(Wu, Shufeng, & Hongxiu, 2009)	Q-learning	–	–	–	Not clear	✗	Neither
(Dusparic & Cahill, 2009b)	W-learning	–	–	–	7	–	No evaluation
(Prothmann et al., 2009)	LCS	Evolutionary Alg.	–	–	5	✗	Sat
(Medina, Hajbabaie, & Benekohal, 2010)	Q-learning	–	–	Multiple	7	✓	Spillback
(Prashanth & Bhatnagar, 2010)	Q-learning	–	Proposed Method	–	7	✓	Sat
(Bazzan, De Oliveira, & da Silva, 2010)	Q-learning	–	–	Softmax	3	✓	Sat
(Arel, Liu, Urbanik, & Kohls, 2010)	Q-learning	–	FFNN		6	✗	Sat
(Davarynejad, Davarynejad, Vrancken, & van den Berg, 2010)	Q-learning	Fuzzy Granulation	–	–	7	–	No evaluation
(Dai, Zhao, & Yi, 2010)	ADP	–	NN	ϵ-greedy	7	✓	Sat
(Balaji et al., 2010)	Q-learning	–	–	Greedy	5	✓	Spillback
(Natarajan et al., 2010)	Inverse RL	–	–	Softmax	7	✓	Neither
(Waskow & Bazzan, 2010)	Q-learning	–	Tile Coding	ϵ-greedy	3	✓	Sat
(El-Tantawy & Abdulhai, 2010)	Q-learning	GT (Coalition)	–	–	7	–	No evaluation
(Salkham & Cahill, 2010)	Q-learning	–	–	ϵ-greedy	5	✓	Sat
(Houli, Zhiheng, & Yi, 2010)	Model-based RL	–	–	–	7	✗	Spillback
(Prashanth & Bhatnagar, 2011)	Multiple RL	–	Average Cost Alg.	–	6	✓	Sat
(Song, Jin, & Zhu, 2011)	Q-learning	–	–	–	7	✗	Neither
(Li, Yan, Lin, & Zhang, 2011)	Q-learning	–	–	–	Not clear	✗	Sat
(Lu, Zhang, & Xie, 2011)	Actor-Critic	Swarm Optimization	–	–	7	✗	Neither
(Natarajan, Joshi, Tadepalli, Kersting, & Shavlik, 2011)	Inverse RL	–	–	–	7	✓	Neither
(Heinen, Bazzan, & Engel, 2011)	Q-learning	–	IGMN	ϵ-greedy	3	✗	Sat
(Abdoos, Mozayani, & Bazzan, 2011)	Q-learning	–	–	ϵ-greedy	4	✓	Sat
(Dai, Hu, Zhao, & Zhu, 2011)	RL	–	FFNN	–	7	✗	Sat
(Dusparic & Cahill, 2012)	W-learning	–	–	Softmax	7	✗	Sat

(*continued on next page*)

Table 3 (*continued*).

Citation	RL method	Combined method	Approx. method	Action selection	Action type	Discretization	Spillback or Sat.
(Medina & Benekohal, 2012)	Q-learning	Max-plus	–	–	7	✗	Spillback
(Khamis, Gomaa, & El-Shishiny, 2012)	Model-based RL	–	–	–	7	✗	Sat
(Khamis & Gomaa, 2012)	Model-based RL	–	–	Softmax	7	✓	Sat
(Jadhao & Kulkarni, 2012)	Q-learning	–	–	–	7	✓	Spillback
(Xu, Xia, & Luo, 2013)	Q-learning	–	–	Softmax	3	✗	Sat
(Moghadam & Mozayani, 2013)	Q-learning	–	–	Random	3	✗	Neither
(Chin, Tham, Rao, Bolong, & Teo, 2013)	Q-learning	–	–	ϵ-greedy	5	✓	Sat
(Nuli & Mathew, 2013)	Actor-Critic	–	–	ϵ-greedy	5	✗	Sat
(El-Tantawy, Abdulhai, & Abdelgawad, 2013)	Q-learning	–	–	–	7	✗	Sat
(Abdoos, Mozayani, & Bazzan, 2013)	Q-learning	–	–	ϵ-greedy	4	✓	Sat
(Box & Waterson, 2013)	Q-learning	–	NN	ϵ-greedy	7	✗	Sat
(Pham et al., 2013)	Q-learning	–	Tile Coding	ϵ-greedy	7	✗	Sat
(Brys, Nowé, Kudenko, & Taylor, 2014)	Q-learning	–	Tile Coding	ϵ-greedy	7	✓	Neither
(Prabuchandran, An, & Bhatnagar, 2014)	Q-learning	–	–	ϵ-greedy	4	✓	Sat
(Chanloha, Chinrungrueng, Usaha, & Aswakul, 2014)	Q-learning	CTM	–	ϵ-greedy	7	✓	Sat
(Fagan & Meier, 2014)	Actor-Critic	–	–	ϵ-greedy	7	✗	Neither
(Marsetič, Šemrov, & Žura, 2014)	Q-learning	–	–	ϵ-greedy	7	✓	Sat
(Khamis & Gomaa, 2014)	Model-based RL	–	–	Multiple	7	✗	Sat
(El-Tantawy, Abdulhai, & Abdelgawad, 2014)	Multiple RL	–	–	Comparison	Multiple	✓	Sat
(Sadigh, Kim, Coogan, Sastry, & Seshia, 2014)	TD	–	–	–	7	✓	Neither
(Abdoos, Mozayani, & Bazzan, 2014)	Q-learning	–	Tile Coding	–	3	✓	Sat
(Liu, Liu, Peng, & Zhu, 2014)	Q-learning	–	Gradient-descent	–	Not clear	✗	Sat
(Jadhao & Jadhao, 2014)	Q-learning	Edge Detection Alg.	–	Random	7	✗	Sat
(Teo, Yeo, Chin, Chuo, & Tan, 2014)	Q-learning	–	–	–	5	✓	Sat
(Xiang & Chen, 2015)	Q-learning	Grey Theory	BPNN	Greedy	7	✗	Spillback
(Xu, Zhong, Wu, & Ni, 2015)	RL	–	–	–	Not clear	–	No evaluation
(Yin, Dridi, & E. Moudni, 2015)	ADP	–	Gradient-descent	–	7	✗	Sat
(Araghi, Khosravi, & Creighton, 2015)	Q-learning	–	–	ϵ-greedy	7	✓	Sat
(Ajorlou, Awasthi, & Aghdam, 2015)	Q-learning	CTM	–	ϵ-greedy	7	✗	Neither
(Ozan, Baskan, Haldenbilen, & Ceylan, 2015)	Q-learning	–	–	Random	6	✗	Sat
(Zhu, Aziz, Qian, & Ukkusuri, 2015)	RL	Junction Tree Alg.	–	ϵ-greedy	7	✓	Spillback
(El-Tantawy, Rezaee, & Abdulhai, 2015)	Q-learning	–	Linear Model Tree	–	7	✗	Sat
(Prabuchandran, An, & Bhatnagar, 2015)	Q-learning	–	–	ϵ-greedy	2	✓	Sat
(Abdelgawad, Abdulhai, El-Tantawy, Hadayeghi, & Zvaniga, 2015)	Q-learning	–	–	ϵ-softmax	7	✗	Spillback
(Mashayekhi & List, 2015)	Q-learning	Auction Theory	–	ϵ-greedy	7	✗	Sat
(Abdoos, Mozayani, & Bazzan, 2015)	Q-learning	–	–	–	5	✗	Neither
(Tahifa, Boumhidi, & Yahyaouy, 2015)	Q-learning	Swarm Optimization	–	ϵ-greedy	3	✗	Neither
(Van der Pol & Oliehoek, 2016)	Q-learning	–	Deep NN	Greedy	7	✗	Neither
(Dusparic, Monteil, & Cahill, 2016)	W-learning	–	–	Softmax	7	✗	Sat
(Gaikwad, Kadarkar, & Kasbekar, 2016)	Q-learning	–	–	ϵ-greedy	3	✓	Sat
(Prashanth & Ghavamzadeh, 2016)	Actor-Critic	–	SPSA, SF	Softmax	7	✓	Neither
(Iyer, Jadhav, Mavchi, & Abraham, 2016)	Q-learning	Fuzzy	–	Greedy	7	✓	Neither
(Wang, Geng, & Li, 2016)	Q-learning	Pro-CEP Model	DQF	–	Not clear	✓	Sat
(Prashanth, Jie, Fu, Marcus, & Szepesvári, 2016)	RS RL	CPT Paradigm	SPSA	–	7	✗	Neither
(Chu, Qu, & Wang, 2016a)	Q-learning	–	CNN	UCB	7	✗	Neither
(Shen, 2016)	Q-learning	Cellular Automata	WNN	–	7	–	Spillback
(Chu, Qu, & Wang, 2016b)	Q-learning	–	Linear Q-function	ϵ-greedy	7	✓	Neither
(Rosyadi, Wirayuda, & Al-Faraby, 2016)	Q-learning	–	–	–	7	✓	Sat
(Yin, Dridi, & E. Moudni, 2016)	Q-learning	–	Linear FA	–	7	✗	Sat
(Chen, Yao, He, Chen, & Jin, 2017)	Q-learning	–	–	–	7	✗	Sat
(Kristensen & Ezeora, 2017)	Q-learning	JADE	–	Greedy	7	✓	Neither
(Darmoul, Elkosantini, Louati, & Said, 2017)	Q-learning	Immune Network	–	–	6	✓	Spillback
(Jin & Ma, 2017)	SARSA	–	–	–	7	✗	Neither
(Liu, Qin et al., 2017)	Q-learning	Dynamic Clustering	Gradient-descent, Tile Coding	–	7	✓	Neither

(*continued on next page*)

Table 3 (*continued*).

Citation	RL method	Combined method	Approx. method	Action selection	Action type	Discretization	Spillback or Sat.
(El Hatri & Boumhidi, 2017)	Q-learning	–	–	–	6	✗	Incident
(Lu, Wen, & Gen, 2017)	Q-learning	–	–	Softmax	7	✓	Sat
(Chu & Wang, 2017)	Q-learning	Max-sum	LR, SR	ϵ-greedy	7	✗	Sat
(Liu, Liu, & Chen, 2017)	Q-learning	–	–	ϵ-greedy	7	✗	Sat
(Vidhate & Kulkarni, 2017)	Q-learning	–	–	–	7	✓	Neither
(Aslani, Mesgari, & Wiering, 2017)	Actor-Critic	–	Tile Coding, RBF	ϵ-greedy	5	✓/✗	Sat
(Al Islam, Aziz, Wang, & Young, 2018)	Q-learning	–	–	ϵ-greedy	7	✗	Sat
(Daeichian & Haghani, 2018)	Q-learning	Fuzzy, GT	–	–	3	✗	Sat
(Wei, Zheng, Yao, & Li, 2018)	Q-learning	–	Phase Gate	ϵ-greedy	7	✗	Sat
(Aslani, Seipel, Mesgari, & Wiering, 2018)	Multiple RL	–	Tile Coding	ϵ-greedy	5	✓/✗	Spillback
(Aziz, Zhu, & Ukkusuri, 2018)	TD	–	–	ϵ-greedy	7	✓	Spillback
(Aslani, Seipel, & Wiering, 2018)	CR RL	–	Tile Coding, RBF, TF	–	5	✓/✗	Spillback
(Li, Yan, Zhou, Wu, & Wang, 2018)	Q-learning	–	DAE	ϵ-greedy	7	✗	Sat
(Kao & Wu, 2018)	Q-learning	Self-Organizing Map	–	–	5	✗	Neither
(Liu, Ding, Borst, & Walid, 2018)	Q-learning	–	NN	ϵ-greedy	5	✗	Neither
(Torabi, Wenkstern, & Saylor, 2018)	RL	Self-adaptive NTSC	–	–	Act 1	✓	Sat
(Wang, Yang, Liu, & Liang, 2018)	Q-learning	Bayesian Optimization	–	–	6	✗	Sat
(Shi & Chen, 2018)	Q-learning	Transfer Learning	RNN	ϵ-greedy	7	✗	Sat
(Jin & Ma, 2018)	SARSA	–	KNN	ϵ-greedy	7	✗	Sat
(Vinitsky et al., 2018)	RL	–	NN	–	Multiple	✗	Sat
(Lemos, Bazzan, & Pasin, 2018)	Q-learning	–	–	ϵ-greedy	7	✓	Sat
(Nishi, Otaki, Hayakawa, & Yoshimura, 2018)	Q-learning	–	GCN	–	7	✓	Sat
(Jin & Ma, 2019)	SARSA	–	LC,KNN,CMAC,FFNN,CQF	ϵ-greedy	3	✗	Sat
(Wei, Xu et al., 2019)	Q-learning	–	NN, GAN	Softmax	7	✗	Sat
(Kitagawa, Moustafa, & Ito, 2019)	Q-learning	–	Deep NN	–	6	✗	Sat
(Reda, Mountassir, & Mohamed, 2019)	Q-learning	–	–	–	7	✗	No evaluation
(Shabestray & Abdulhai, 2019)	Q-learning	–	CNN	ϵ-greedy	7	✓	Spillback
(Wu et al., 2019)	RL	Fog Computing	–	–	4	✗	Neither
(Chen, Zhu, & Lu, 2019)	Q-learning	–	CNN	Greedy	7	✗	Sat
(Rizzo, Vantini, & Chawla, 2019a)	Policy Gradient	–	Deep NN	Softmax	7	✗	Sat
(Aslani, Mesgari, Seipel, & Wiering, 2019)	Actor-Critic	–	–	Comparison	7	✓	Neither
(Huang, Hu, Huo, & Pei, 2019)	Q-learning	–	Deep NN	–	5	✓	Sat
(Gao, Liu, Li, & Yuan, 2019)	Q-learning	Fuzzy, Edge Computing	–	ϵ-greedy	6	✓	Sat
(Tan et al., 2019)	Q-learning	–	–	ϵ-greedy	7	✗	Sat
(Horsuwan & Aswakul, 2019)	Q-learning	–	Deep NN	–	7	✗	Spillback
(Chu, Wang, Codecà, & Li, 2019)	Actor-Critic	–	Deep NN	ϵ-greedy	7	✗	Sat
(Wei, Chen et al., 2019)	Q-learning	–	Deep NN	–	7	✗	Spillback
(Xu, Zheng, Xu, Zhu, & Li, 2019)	Q-learning	Transfer Learning	LSTM	–	7	✗	Sat
(Rizzo, Vantini, & Chawla, 2019b)	Policy-Gradient	–	Deep NN	Softmax	7	✗	Sat
(Zheng, Xiong et al., 2019)	Q-learning	FRAP Model Design	Deep NN	–	7	✗	Sat
(Gong, Abdel-Aty, Cai, & Rahman, 2019)	Q-learning	–	Deep CNN	–	7	✗	Sat
(Zhou, Braud, Alhilal, Hui, & Kangasharju, 2019)	Q-learning	Edge Computing	Deep NN	ϵ-greedy	Act 1	✗	Sat
(Bouderba & Moussa, 2019)	Q-learning	Cellular Automata	–	–	Not clear	✓	Sat
(Higuera, Lozano, Camacho, & Higuera, 2019)	Q-learning	–	–	ϵ-greedy	7	✗	Neither
(Ni & Cassidy, 2019)	Actor-Critic	–	GCN	ϵ-greedy	Act 2	✗	Sat
(Ge, Song, Wu, Ren, & Tan, 2019)	Q-learning	–	Deep CNN	ϵ-greedy	7	✓	Sat
(Shu, Wu, & Li, 2019)	RL	–	–	–	7	✗	Sat
(Gan, Guo, & Li, 2019)	Q-learning	–	–	Softmax	7	✗	Sat
(Yang, Yang, Wong, & Kang, 2019)	Actor-Critic	–	Deep NN,CNN	Softmax	7	✗	Sat
(Lee, Chung, & Sohn, 2020)	Q-learning	–	Deep CNN	–	2	✗	Sat
(Genders & Razavi, 2020)	Multiple RL	–	FFNN	Random	7	✗	Sat
(Xu et al., 2020)	Q-learning	–	RNN	ϵ-greedy	7	✗	Neither
(Qu, Pan, Chen, Wang, & Li, 2020)	RL	RMS-NE, CTM	–	Softmax	5	✓	Sat
(Zhao, Ma, Shen, & Qian, 2020)	RL	Junction Tree Alg.	–	–	7	✓	Sat
(Kim & Jeong, 2020)	Q-learning	–	DNN	ϵ-greedy	7	✗	Sat

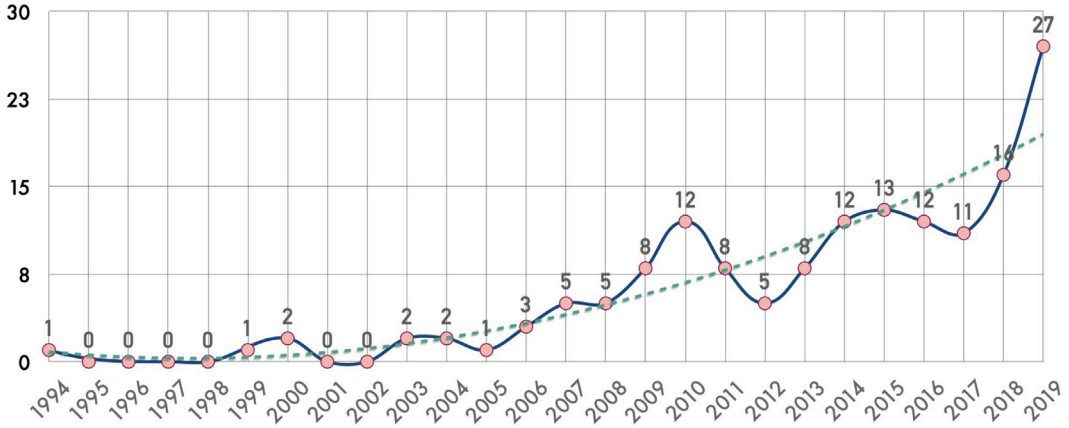


Fig. 4. Distribution (solid line) and trend (dashed line) of number of studies over time. To make the comparison consistent, the 6 included papers published in 2020, until the end of March, are not included in this chart.

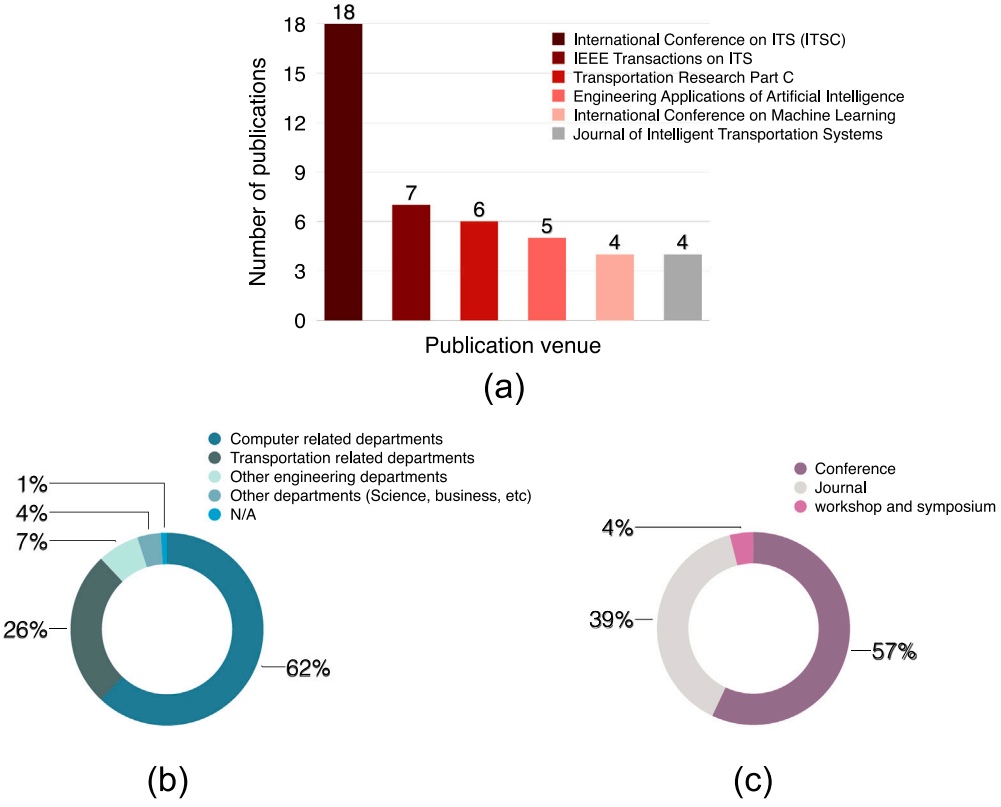


Fig. 5. (a) Publication venues with highest frequency of publications in RL-NTSC, (b) distribution of university departments of authors in RL-NTSC, and (c) distribution of types of publication venues in RL-NTSC.

as of 2020, these countries are the global leaders in the research of RL-NTSC.

An interesting fact to note is that according to the Central Intelligence Agency (CIA) World Factbook (Central Intelligence Agency, C. C. R., 2020), the USA, China, India, and Brazil rank in the top four countries with the longest road network, respectively, with Canada at eighth. The motor data company, INRIX (Inrix, 2020), finds Ireland's capital city of Dublin to be the 7th worst city in the world in terms of hours lost due to traffic congestion based on 2019 data (154 h). And, all seven of the countries mentioned have a traffic index (which indicates inefficiencies in the traffic system) above 140.45 according to (Index, 2014), with Iran having the 6th worst traffic index of the group, at 216.09.

This study also took into consideration the department and affiliations of authors, and identified four groups of departments, as follows: (1) computer related departments, such as computer engineering/science, information technology, and electrical engineering, (2) transportation related departments, including civil and transportation engineering departments, (3) other engineering departments, such as industrial, mechanical, material, and geomatics, and (4) other departments, like science, business, management, astronautics, and English. It was found that 62% of the authors were from computer related departments (group 1), and 26% were from transportation related departments (group 2), while researchers from other departments (groups 3 and 4) accounted for 12% of the total.



Fig. 6. (Left) the distribution of studies based on country of the authors, and (right) the number of authors. The number of authors of the papers published on RL in NTSC ranges from 1 to 10.

Notably, authors from these four groups of departments had very low research collaboration with each other, and had conducted research in only 20 out of 160 articles, generally. The computer related and transportation related departments collaborated in only 11 articles, while they contributed independently in 98 and 36 papers, respectively.

This investigation also uncovered that academia and research institutes have low research collaborations with industry and government, with 11 instances of collaboration between academia and industry and only one appearance of government in research papers that collaborated with academia in a paper in 2018 (Torabi, Wenkstern et al., 2018). Suffice it to say that research exploring the potential benefits of the collaborations between these three parts, i.e. academia, industry, and government is needed, as the findings in the literature (Anderson & Odei, 2018) suggest that increased collaboration in these domains may boost the efficiency of the proposed methods in real-time, real-world applications.

4.2. Method identification and analysis

In this section, we elaborate on the proposed methods and categorize them from different angles.

4.2.1. Methods' contribution and combination

The proposed methods in the included articles consist of (M1) the applied RL methods in NTSC (154 papers) or (M2) theoretical developments in RL with feasibility or applicability assessment of the proposed methods in different contexts, one of which is the NTSC environment (6 papers); see the category of "method contribution" in Fig. 7.

To get familiar with the variations of application of RL in solving different research problems in NTSC and the way that RL is applied in the first group, i.e. (M1), we identified two main groups: (M1.1) RL is used alone or in combination with methods from the same field (i.e. machine learning (ML), game theory (GT), and DP methods) in NTSC (134 papers), and (M1.2) RL is used in combination with other methods from other fields in NTSC (20 papers). In the following, we investigate these two groups as shown in the category of "RL method application" in Fig. 7:

⟨M1.1⟩: The vast majority of the studies (134) are classified in this category where RL is the only applied method, or is used in combination with other ML, GT, and DP methods. Different innovative designs were proposed to tackle the problems of NTSC. The NTSC problem has a continuous state space, infinite horizon, and is only partially observable and difficult to model. In this context, most of the papers try to improve the performance (Moghadam & Mozayani, 2013), dimensionality (e.g. by means of function approximation (Prashanth & Bhatnagar, 2010; Waskow & Bazzan, 2010)), complexity (e.g. by organizing agents in groups of limited size (Bazzan et al., 2010), and using holonic RL methods (Abdoos et al., 2013)), scalability (Nuli & Mathew, 2013), stability (Aslani, Seipel, & Wiering, 2018), speed of optimization (e.g. in Transfer Learning models (Xu et al., 2019)), state and/or action space manageability or generalizability (Araghi et al., 2015; Gaikwad et al., 2016), and centrality problem in applying RL in NTSC. Some studies invest in solving convergence and oscillation problems that commonly appear in the multi-agent context. For instance, Reda et al. (2019) proposed a model based on Double Deep Q-learning (DDQ), with Experience Replay and cooperation between agents. They used NN to reduce the correlation between agents and improve performance. Improving the accuracy, optimum functioning, and efficiency of the results is also of high interest to the researchers, through hierarchical methods (Tan et al., 2019; Wang, Yang, Liu et al., 2018), for example. There are several papers where the main focus of research is on studying the coordination between agents (Higuera et al., 2019), and the integrated network, specifically signalized intersections and ramp metering (El-Tantawy & Abdulhai, 2010; El-Tantawy et al., 2013, 2015). Famous methods proposed in traffic theory context, such as CTM (Ajorlou et al., 2015; Chanloha et al., 2014; Qu et al., 2020), Max-plus (Kuyer et al., 2008; Medina & Benekohal, 2012), and Max Pressure (MP) or back pressure (brought in NTSC from the communications networks theory) (Wei, Chen et al., 2019), are also applied in some research, and in other studies multiple traffic optimization goals are simultaneously optimized (multi-policy RL), e.g. Dusparic et al. (2016).

Considering both vehicular and pedestrian traffic in the network-scale is a recent application of RL, where a distributed multi-agent RL method is proposed by Liu, Liu et al. (2017) for the first time in 2017. By the increasing use of deep RL methods, the number of papers that focused on improving deep RL models in NTSC has increased since 2018 (e.g. Li et al. (2018), Liu et al. (2018), Wei et al. (2018). The co-learning problem of both classes of learning agents, traffic signals and drivers, with different goals (minimizing individual travel times vs minimizing the queues locally), different nature (driver agents learn in episodes that are asynchronous, while traffic light agents learn continuously (non-episodic)), and the nontrivial task of microscopic modeling and simulation (whose actions are highly coupled) is another area of research that was addressed (Lemos et al., 2018). In addition to these lines of research in this category, analyzing what specifically RL does differently (i.e. analysis of the learned policies) than other TSC methods is another research effort that is motivated and conducted by Genders and Razavi (2020).

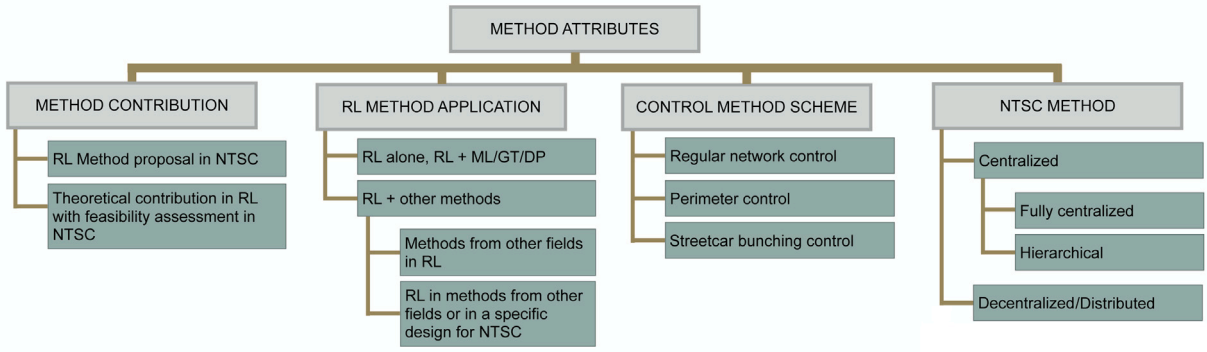


Fig. 7. Method attributes in RL-NTSC.

⟨M1.2⟩: In category (M1.2), the articles applied methods from other fields (rather than ML, GT, and DP) in RL or RL framework (M1.2.1), or used RL in the methods from other fields or in a specific design/model/framework for NTSC context (M1.2.2). Out of 20 articles in category (M1.2), there are 8 articles that either applied RL in optimization problems (e.g. Swarm optimization) or an optimization algorithm is used in RL methods/frameworks in the context of NTSC.

⟨M1.2.1⟩: To cope with non-stationary environments, Da Silva et al. (2006) proposed, formalized and showed the efficiency of a method called the RL-CD, or Reinforcement Learning with Context Detection, which performs well in non-stationary environments, and better than classic RL algorithms (Q-learning and Prioritized Sweeping). In a similar work, de Oliveira et al. (2006) assessed the feasibility of applying RL-CD approach in a more realistic scenario, implemented by means of a microscopic traffic simulator. Zhang et al. (2007) showed how to use Conditional Random Fields (CRFs) to model control processes, where CRFs model joint actions in a decentralized Markov decision process and define how agents can communicate with each other to choose the best joint action. The CRFs model clearly outperformed the independent agents approach. Khamis et al. (2012) enhanced the single-objective controller by developing a multi-agent NTSC system based on a multi-objective sequential decision-making framework using Bayesian interpretation and some innovative reward design. Zhu et al. (2015) proposed a Junction Tree Algorithm (JTA) based approach to obtain an exact inference of the best joint actions in traffic signal coordination that outperformed independent learning (Q-learning), real-time adaptive learning, and fixed timing plans. To predict future system states and avoid unwanted states, Wang, Geng et al. (2016) applied Proactive Complex Event Processing (Pro-CEP) method (in processing proactive traffic congestion control) that uses RL to find the optimal joint policy. This method works well when used to control congestion. To develop learning and adaptation mechanisms to deal with disturbances, Darmoul et al. (2017) proposed a distributed TSC system based on hybridization between Case-Based Reasoning (CBR) and an adaptation of the reinforcement principle within artificial immune networks (a mechanism inspired by biological immunity). The method controls interrupted flow at signalized intersections. Wei, Xu et al. (2019) studied how the attention mechanism helps cooperation (to minimize the average queue length) via using graph attentional networks to facilitate communication, incorporating the temporal and spatial influences of neighboring intersections to the target intersection, and building up index-free modeling of neighboring intersections.

⟨M1.2.2⟩: Su and Tham (2007) integrated sensor networks and grid computing and the usage of web services to implement this integration. They used Q-learning algorithms in distributed Stargates to NTSC. Stargates is a computer with sensor signal processing capabilities. Abdoos et al. (2015) modeled a holonic multi-agent system and proposed a holonic RL multi-agent system method that improves the performance of individual Q-learning in NTSC. Mashayekhi and List (2015) showed the applicability and efficacy of using auction theory combined with an RL in a multi-agent system. In low traffic volume the proposed method outperforms actuated and pre-timed control strategies, but in heavy volume the pre-timed control strategy outperforms the proposed method. Iyer et al. (2016) designed a distributed multi-agent method with coordination between agents through the communication of decision data. In this paper, the effectiveness of integrating fuzzy logic controller (to deal with continuous states and actions) and Q-learning (for learning during the process) is studied.

With regards to the optimization-related articles in category (M1.2), all of the proposed methods in this category are designed to handle the growing complexity and curse of dimensionality and to increase the speed of TSC by using RL in the optimization problems or applying the optimization algorithms in an RL method or framework. To achieve cooperation in the long term, as stated earlier, Mikami and Kakazu (1994) combined RL of a local agent with global optimization through Genetic Algorithms (GA) by which the RL parameters are modified. Cao et al. (1999) used a classifier system with a fuzzy rule representation, with both evolutionary and RL methods to provide a faster method than hierarchical control. Cao et al. (2000) incorporated Learning Classifier System (LCS) and TCP/IP (Transmission Control Protocol/Internet Protocol) based communication server into a distributed learning control strategy to increase the speed of control. Prothmann et al. (2009) managed the complexity by using an organic approach to NTSC and proved the feasibility of the proposed approach. Lu et al. (2011) proposed a multi-agent NTSC by using Swarm Intelligence and Neuro-Fuzzy RL to combine the better attributes of both with improving the learning speed and performance. Liu et al. (2014) optimized the TSC for V2I networks by proposing a cooperative distributed Q-learning algorithm with a fast gradient-descent function approximation. Ozan et al. (2015) applied an RL method in the optimization problem to reach good solutions for NTSC. The results were better than the genetic algorithm and hill-climbing methods in low demand but could not outperform them in medium and high demands. Tahifa et al. (2015) showed that Swarm Q-learning performs better than standard Q-learning in increasing the speed of TSC. To alleviate traffic congestion and limit the effects of incidents on traffic flow, El Hatri and Boumhidi (2017) proposed a Q-learning based traffic management model, which simultaneously optimizes vehicle re-routing and TSC based on the Multi-Objective Particle Swarm Optimization (MOPSO) method.

⟨M2⟩: To provide a simple and efficient method to implement, Natarajan et al. (2011) put into operation a functional gradient boosting approach to imitation learning in relational domains. The proposed approach outperforms both learning a single relational regression tree and performing a propositional functional gradient to represent the policy in all domains. To provide a solution to multi-objective problems with correlated objectives rather than typical multi-objective problems, Brys et al. (2014) proposed an RL-based method combining multiple correlated rewards and shaping signals by measuring confidence (i.e. combining the feedback from all objectives, instead of only looking at a single one), called adaptive objective selection. They formally

Table 4
RL, core non-RL, and combined non-RL methods.

RL method	Core non-RL method	Combined non-RL method
Q-learning (96)	Auction theory	JTA
No specific RL (13)	Fuzzy	CPT paradigm
Actor-critic (10)	Fog Computing	Self-Organizing Map
Model-based RL (8)	Pro-CEP	Model-based and Bayesian optimization algorithms
ADP (6)	Self-adaptive NTSC system	Transfer Learning
W-learning (5)	Cellular Automaton	FRAP model design
Multiple RL (4)	Immune Network algorithm	CTM
Policy-Gradient (4)	NN	RMS-NE
LCS (3)	Multiband	Max-plus
SARSA (3)	SensorGrid	CRFs
TD (2)		GA
LA (2)		Grey theory model
Inverse RL (2)		Qualitative method
Continuous Residual RL (1)		Evolutionary Alg.
Risk-sensitive RL (1)		Dynamic Clustering Alg.
		Neuro-Fuzzy
		JADE
		Coalition
		Fuzzy granulation
		Edge computing framework
		Max-sum message passing Alg.
		Swarm optimization

defined a new class of multi-objective problems called correlated multi-objective problems (CMOP), whose set of solutions being optimal for at least one objective is so restricted that the decision-maker is least concerned about which of these is found, and more so about how fast one is found, or how well one is approximated. Sadigh et al. (2014) proposed a method for synthesizing a control policy for an MDP such that traces of the MDP satisfy a control objective expressed as a linear temporal logic (LTL) formula through using an RL algorithm that finds the policy optimizing the expected utility of every state in the Rabin-weighted product MDP. They prove that the method is guaranteed to find a controller that satisfies the LTL property with probability one if such a policy exists, and they suggest empirically with a case study in traffic control that their method produces reasonable control strategies even when the LTL property cannot be satisfied with probability one. Prashanth and Ghavamzadeh (2016) optimized variance-related risk measures in rewards and demonstrated its usefulness in an NTSC application. The risk-sensitive algorithms result in lower variance but higher long-term cost compared to their risk-neutral counterparts. Prashanth et al. (2016) illustrated the usefulness of modeling human decisions by Cumulative Prospect Theory (CPT) paradigm in RL and suggested that CPT-based criteria is useful in a NTSC application. Gan et al. (2019) proposed a dynamic correlation matrix based MARL approach where the meta-parameters are evolved using an evolutionary algorithm in a distributed manner. This was done to provide meaningful theoretical verification by using both agent-level implementation and system-level convergence verification. Agents using the proposed learning algorithm reach optimal behaviors faster than other canonical learning techniques.

In the following, we introduce the types of RL methods used in the articles we included in this paper. Furthermore, we extracted data about other methods that an RL method has been integrated with to provide a solution, whether as a core or combined method.

4.2.2. RL methods

Q-learning (Watkins & Dayan, 1992) ranks first on the list of used RL methods in 96 (60%) studies. 13 papers do not provide any specific RL method, especially when the goal of the paper is the proposal of a framework or when an RL concept is used. In this case, we used "RL" in Table 3. The remaining papers include methods that are listed (along with their frequency) in the first column of Table 4.

Furthermore, RL is the core method in 149 (93%) of the studies. In 11 studies, other methods were employed as the core method in which RL is used as a combined method, see the second column of Table 4. There are also other non-RL methods that were used in combination with the RL methods or frameworks, see the third column of Table 4. Below is a synopsis of the RL methods.

- *Q-learning and SARSA*: Both Q-learning and SARSA (State, Action, Reward, State, Action) methods are critic-only where they use Q-tables to decide which action to take. The biggest difference between the two is that Q-learning is an off-policy RL method, while SARSA is an on-policy method. Q-learning and SARSA Q-table update equations are respectively presented below, where the policy is TD.
- *Critic-only, Actor-only, and Actor-Critic*: The critic uses the calculated Q-values (or function values) to choose its action while the actor uses the policy to decide. Actor-only methods work to improve the policy. Actor-critic being a combination of both, allows both calculations of the Q-values and the policy to choose appropriate action. The papers that used an actor-critic method are identified in Table 5.
- *W-learning:* W-learning is a multi-policy self-organizing action-selection technique proposed in Humphrys (1996) that builds on Q-learning. In W-learning, there is a competition among selfish Q-learners where agents learn Q-values for state–action pairs for each policy and W-values for each of the states of each of their policies to explore what happens if the nominated action is not followed.
- *Approximate Dynamic Programming (ADP):* The computational complexity of DP algorithms due to excessive system state and their need for an exact algorithm and true value function makes the algorithm impractical in solving large-scale TSC problems. ADP aims not to fall into the predicament of computational complexity by replacing the true value function of the DP with an approximation function. In other words, it is similar to the model-based RL with function approximation. The research papers that used ADP method are identified in Table 5.
- *Policy-Gradient:* The policy gradient method does not need to estimate the state or action value functions. It learns parameterized policy functions directly by searching policy space to maximize a measure based on the accumulated reward. In this way, it averts the convergence problems of estimating value functions.
- *Inverse RL*: In Inverse RL (Ng, Russell, et al., 2000), the reward function of an agent (that the agent tries to optimize) is learned and determined by observing the agent's behavior over time, the environment model, and the environment measurements. This approach is akin to learning from an expert and is helpful in

Table 5
Identification of the included papers in terms of method identification, code, simulation, and evaluation.

Category	Citation
Method Identification	
Actor-critic	Aslani et al. (2019, 2017), Aslani, Seipel, Mesgari et al. (2018), Chu et al. (2019), Dai et al. (2010), Fagan and Meier (2014), Genders and Razavi (2020), Lu et al. (2011), Ni and Cassidy (2019), Nuli and Mathew (2013), Prashanth and Ghavamzadeh (2016), Richter et al. (2007), Yang et al. (2019), Zhang et al. (2007)
Deep Learning	Chen et al. (2019), Chu et al. (2016a, 2019), Ge et al. (2019), Gong et al. (2019), Horsuwan and Aswakul (2019), Huang et al. (2019), Kim and Jeong (2020), Kitagawa et al. (2019), Lee et al. (2020), Li et al. (2018), Liu et al. (2018), Ni and Cassidy (2019), Reda et al. (2019), Rizzo et al. (2019a, 2019b), Shabestray and Abdulhai (2019), Shi and Chen (2018), Tan et al. (2019), Van der Pol and Oliehoek (2016), Vinitsky et al. (2018), Wei, Chen et al. (2019), Wei et al. (2018), Xu et al. (2020, 2019), Yang et al. (2019), Zheng, Xiong et al. (2019), Zhou et al. (2019)
Model-based methods	Da Silva et al. (2006), Houli et al. (2010), Khamis and Gomaa (2012, 2014), Khamis et al. (2012), Kuyer et al. (2008), Wiering (2000), Wiering et al. (2004)
RL and GT	Daeichian and Haghani (2018), El-Tantawy and Abdulhai (2010), El-Tantawy et al. (2013), Qu et al. (2020), Xinhai and Lunhui (2009), Xu et al. (2013), Zhao et al. (2009)
ADP	Dai et al. (2010), Fagan and Meier (2014), Li et al. (2008b), Reda et al. (2019), Yin et al. (2015)
Code, Simulation, and Evaluation	
Code is available	Brys et al. (2014), Chu et al. (2019), Genders and Razavi (2020), Prashanth et al. (2016), Vinitsky et al. (2018), Wei, Chen et al. (2019), Wei, Xu et al. (2019), Wei et al. (2018), Zheng, Xiong et al. (2019), Zhou et al. (2019)
No simulation provided	Davarynejad et al. (2010), Dowling et al. (2004), El-Tantawy and Abdulhai (2010), Reda et al. (2019), Xinhai and Lunhui (2009), Xu et al. (2015)
No evaluation/self-comparison provided	Davarynejad et al. (2010), Dowling et al. (2004), Dusparic and Cahill (2009b), El-Tantawy and Abdulhai (2010), Liu et al. (2018), Prashanth et al. (2016), Prothmann et al. (2009), Reda et al. (2019), Xinhai and Lunhui (2009), Xu et al. (2015)

the domains where the reward may not be easily accessible, like TSC (Natarajan et al., 2010). This method has its origins in Imitation learning (also called as apprenticeship learning, learning by observation, or learning from demonstrations). It is comparable to supervised learning, with the key difference being that the examples are not i.i.d, but instead, follow a meaningful trajectory (Bagnell, Ratliff, & Zinkevich, 2006).

- *Learning Classifier Systems:* A learning classifier system (LCS) (Butz, Goldberg, & Lanzi, 2005) is a rule-based RL system in which each rule (or classifier) is composed of a condition, an action, and a reward (or evaluation). LCS combines an evolutionary process (e.g. a genetic algorithm), with a learning process (e.g. RL), wherein a rule is constructed as {IF 'condition' THEN 'action'}. A genetic algorithm tries to improve condition-action rule space by generating new classifiers from current strong classifiers and removing the weak ones. RL is responsible for selecting the action with the best-rewarded response or evaluation to be executed.
- *Learning Automata:* The action selection in learning automata is performed based on the last selected action and the received reward. A learning automata method forms from a vector of probabilities over the set of actions, which are updated (i.e. increase or remain the same) based on the reward.
- *Model-based and Model-Free*: Model-based methods provide the agent with part or all of a model in which the agent must work. In model-free methods the agent develops its own model in which to work and has fewer restrictions. Essentially, in model-based methods the transition and reward functions are assumed to be available to construct a model, unlike in model-free methods where the agents do not need to have access to information regarding how the environment works. The papers that used model-based methods are identified in Table 5.

4.2.3. Control method scheme

By analyzing the proposed methods in the included articles, we identified three levels of control, including (i) regular network control, which covers the general idea of controlling all traffic signals in a regular network, (ii) perimeter control, which improves the performance of the entire network by controlling a part of traffic signals that located on the boundary of a region in the network, and (iii) streetcar[3] bunching control, which mitigates the effect of streetcar bunching along transit routes by controlling successive signalized intersections; see the category of "control method scheme" in Fig. 7.

The regular network control has been the focus of 158 out of 160 included articles. On the other hand, although there are numerous research works in the domain of perimeter or cordon control, the application of RL for perimeter control is studied, for the first time, in 2019. Ni and Cassidy (2019) explored how RL can be used to re-time traffic signals on the perimeter by developing an RL based controller with NN architectures that controls perimeter with spatially-varying metering rates. With regard to the third level of control, Ling and Shalaby (2005) conducted research to mitigate the effects of streetcar bunching along transit routes through automating streetcar bunching control by means of multiple RL agents that act on a series of successive signalized intersections. The proposed method was able to effectively split up a streetcar bunch and prevent it from forming again.

4.2.4. NTSC method: Centralization (centralized, hierarchical, and decentralized/distributed methods)

When tackling multi-agent problems, there is a spectrum from centralized to decentralized decision making; see the category of "NTSC method" in Fig. 7. At large scale implementation, multiple agents tackle the task given while communicating with each other. This usually results in quicker optimization as each agent learns from its neighbors as well as from itself (Tahifa et al., 2015). With multiple agents, the collected data and actions can be stored centrally in a location that all agents can access to function as one agent. In this setup, the central agent often makes all decisions for the system, and this can slow down the learning process while coordinating the unit (Oroojlooy Jadid & Hajinezhad, 2019). Although DNNs helps enhance the scalability of RL, training a centralized RL agent is still infeasible for large-scale NTSC. Conversely, a distributed approach can be used to store the local information around multiple agents, allowing each one to make its own decision while still communicating with its neighbors (Hüttenrauch, Adrian, Neumann, et al., 2019). In this approach, all agents would be considered "equal" (Baldazo, Parras, & Zazo, 2019). If the local agent does not communicate with other neighbor agents the system is called a decentralized system. Another setup that uses a combination of both centralized and decentralized/distributed methods is a hierarchical system, which can be categorized as a centralized method since it involves centralization. It forms a hierarchy of sorts, where the lower agents may have limited to no ability to enact upon the environment without permission from the "leader". A hierarchical control allows

[3] street railway or tram.

agents to perform micro-actions between tasks to improve the finesse of the agents.

Most (65%) of the proposed methods in NTSC are designed in a decentralized way. There are 7 papers proposing holonic or hierarchical methods while the rest are centralized methods that might be rarely applicable in real-world scenarios in a real-time process.

4.2.5. RL methods' components and types

State, action, and reward. There are normally three main components in RL: state, action, and reward. There are several elements that can define state and action in various papers. The elements of the state can be similar or different from those of the rewards. Based on our collected data of 160 studies, we identified 35 distinct elements of state and 30 of reward. The top 5 frequently used elements in state are queue size with 73 occurrences (38%), phase state (11%), number of vehicles (10%), the position of the vehicles (6%), and speed (6%). In reward, the top 5 are queue size with 71 occurrences (30%), delay (13%), waiting time (9%), the number of vehicles (6%), and number of vehicles passed the intersection (or generally throughput) (4%). The elements of the state in 16 (5%) research papers and those of reward in 18 (7%) papers are not available. The list of all the elements of the state (38 unique elements) and reward (39 unique elements) found in the papers are available in Figs. 8 and 9. This might help the new researchers get some idea in defining these components. These two figures also depict the most frequently used elements in states and reward, respectively.

In addition to state and reward, action needs to be defined. In the majority of the research papers action is defined as a traffic signal or phase switch. However, there are some that will use a different definition than the signal switch. These actions include (Act 1) set the value of a threshold metric for each traffic signal, (Act 2) a link specific metering rate in the perimeter (or cordon) control, (Act 3) select a route as a driver's action, (Act 4) set the acceleration of the vehicles, and (Act 5) set the maximum speed of the vehicles. The last three options are used in a mixed environment where the vehicles are also considered as well as traffic signals. When two or three types of actions were used, it was reported as "Multiple" in Table 3.

The actions directly related to control traffic signals are categorized into two main groups (phase-based and cycle-based), and seven classes where each class is defined based on cycle length, phase duration, and phase order. Each of these three elements can be fixed or variable. See Table 6. The decision point in cycle-based methods is the end of the cycle where cycle length, phase duration, or phase order are determined. In phase-based methods, the decision is made at the end of a phase, which includes phase duration determination and phase selection. In this case, phase duration can be set as fixed for the entire phase or can be allowed to be extended at the end of the phase. Note that in phase-based methods, the cycle and phase order are not applicable. In Table 6, we removed class 1 in which all the three elements (cycle length, phase duration, and phase order) are fixed. This is not applicable in RL design and is counter-intuitive to optimize.

62% of the papers proposed phase-based methods (i.e. class 7), while 32% constructed their methods on cycle-based methods. Only 2 papers focused on using RL, assuming that the phase duration is fixed. Moreover, only 20 (12%) papers worked on applying RL in a fixed cycle-length setup. There are also 3 papers with multiple actions or sets of actions, and 6 papers in which the action is not clearly or completely defined. The details related to each paper are given in Table 3.

Action selection methods and parameters. To select the actions, various action selection methods can be used. Based on the data we retrieved, 71 (44%) papers did not state which action selection they used, which is a significant number. ϵ-greedy has the highest usage and was observed in 51 (32%) papers. Softmax (or Boltzmann) and greedy methods are used in 17 (11%) and 7 (5%) papers, respectively. Other action selection methods with a frequency of 4 or less are Random, Distributed W-learning (DWL), credit assignment algorithms, ϵ-softmax, and Upper Confidence Bound (UCB). Two papers employed multiple or combined action selection methods, while two others compared various action selection methods.

- *ϵ-greedy strategy and SoftMax*: ϵ-greedy uses the epsilon term to balance exploration and exploitation of the environment, encouraging the former early on and switching to the later as the algorithm learns. It randomly selects actions for the next round based on the values of the exploration rate (ϵ), discount factor (γ) and learning rate (α). SoftMax behaves similarly but with weight parameters either assigned or learned to each action. They can be quite sensitive to changes and encourage an outcome where important parameters have more value. The sensitive nature of the weights makes them tricky to learn/find and can affect the performance of the algorithm significantly.

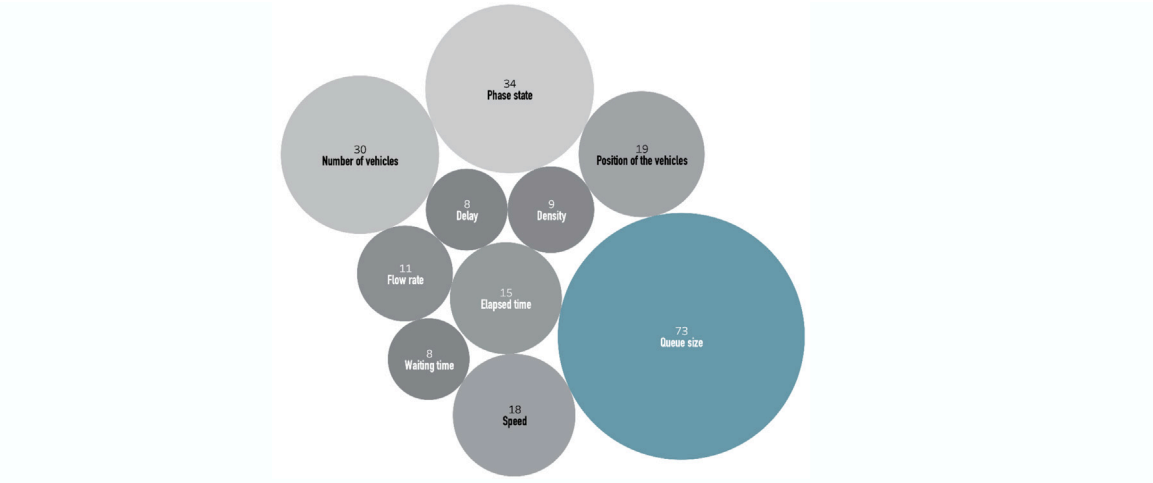
We collected data about the RL parameters, including: exploration rate, discount factor and learning rate. We found that the information was not displayed in a number of cases. 121, 60, and 86 out of 160 papers did not reveal the information about these parameters, respectively. In papers where the data are presented, the most common option was for the authors to acknowledge that these parameters exist within the range of (0,1). While it is a piece of information, it is not especially useful seeing as this information is well known among common practitioners of RL and ML.

State space and action space discretization. Reducing state space (for the states such as queue size, flow rate, and density), action space, and even reward space is one of the ways that can be used to reduce computational cost to make the methods more applicable in real-time process in the real world. To this end, the continuous states are grouped, for example, in 3 levels: low, medium, and high. Another way is using the comparison between the states with the previous step. This way the space is divided into two groups: better or worse. Reducing the space using grouping the data may come at the expense of the lower accuracy of the results, thus lowering the efficiency or optimality. Nonetheless, 59 (37%) of the papers used discretization while 91 (57%) did not discretize the state or action spaces. 3 (2%) papers used discretization only for the discrete methods, but not for the continuous methods that were evaluated.

Tabular vs approximation-based methods. Another point that impacts the efficiency of the performance of the methods in real-world is using tabular RL methods where a look-up table is used to map the spaces. Approximation methods are used to provide a good approximation of the states that were not experienced in training. Interestingly, the number of the papers that used either of these two methods are eerily close: 74 (47%) papers used a tabular method while 72 (45%) used approximation methods.

The approximation methods that are used in our pool of research studies are shown in Table 7. The applied neural networks, as one of the approximation methods, in the papers are also presented in the table. 38 papers used different neural network approximation methods, and the second-most used group is the Tile Coding, observed in 7 papers. One of these applied its own proposed method (Prashanth & Bhatnagar, 2010).

- *Neural Networks*: NNs are loosely designed after the brain and are constructed for tasks such as pattern recognition, labeling, and processing of data. They are commonly used for clustering, classification, and predictions. In the case of traffic control, prediction is the prominent use, though other use does occur. NNs consist of 3 main sections: input section, hidden section, and output section. Usually, there is only one of each input and output layers, but the hidden section may contain more. Each layer contains nodes. In the output layer, the number of nodes often corresponds with the number of possible outputs. In the input layer, the number of



ELEMENTS IN STATE DEFINITION	#	ELEMENTS IN STATE DEFINITION	#
Queue size or average queue size (in phase, lane, downstream lane, upstream lane, link, intersection, network), Congestion level or queue level (in phases, link, intersection, neighboring intersection), Pedestrian queue size (in direction), Number of waiting vehicles (in lane, link, departing link, intersection, network), Number of stopped vehicles (in cycle, link), Maximum queue size (phase, associated with each phase, associated with each direction), Saturation level in the current cycle, Congestion based on comparison between consecutive phases - no threshold is set as in congestion level (in links)	73	Distance of position of a particle with previous best position and position of its leader in Swarm	1
Current phase, Current signal state, Next phase, Phase state (in intersection, neighboring intersections), Signal control state, Index of the current green phase, Phase scenario (whether the signal group agent has to wait for other signal group agents), Index of the phase with maximum queue length for the intersection, Index of the adjacent intersection with maximum queue length, Index of the phase that has to be set green, Phase information	34	Cycle duration	1
Number of vehicles (in phase, lane, lane in neighboring links, downstream lane, link, intersection, network), Number of waiting and approaching vehicles (in link), Vehicle accumulation, Average number of vehicles per lane associated with the lanes that are/are not governed by the phase agent	30	Number of vehicles enter the network	1
Position of the vehicles (in intersection, adjacent intersections), Position of the vehicle within certain distance from the stop line, Distance to the intersection of the n nearest vehicles, Position of the vehicles in the queue, Position (in terms of segments) for the first vehicle approaching the intersection	19	The code of combination phase	1
Speed or average speed (in green signal phase lane, lane in neighboring links), Average vehicle speed (in link), Speed of the vehicles (in intersection, adjacent intersections), Velocities of the n nearest vehicles	18	Traffic light (id)	1
N/A	16	Current time of the day	1
Elapsed time (in phase, lane), Elapsed green time, The time that current phase has been green, The time since the most recent light change, The time since the second-to-last light change, The time duration since the last detection sent by the short/long detector, The time that the current phase will last	15	Presence of public transport vehicles (in link)	1
Flow rate or average flow rate (in phase, lane, link, upstream neighboring intersections), Relative flow, Flow within the previous five minutes (in lane)	11	Number of priority vehicles (in network)	1
Density (in lane, link, based on camera image of the road), Density level	9	Travel time (in link)	1
Delay (in phase, lane, link, intersection, network), Path-wise delay, Cumulative delay (of the corresponding phase, of the first vehicle), Delay of stopped vehicles	8	Vehicles' current headway	1
Waiting time or average waiting time (in phase, lane, link, intersection, network), Waiting time level (in lane), Accumulated waiting time, Waiting time of the lead vehicle	8	Number of neighbor intersections	1
Green times associated with each phase, Green time after minimum green time is passed (for current signal group, for candidate signal group), Maximum value of green times among the other signal groups in the current phase, Elapsed times since the signal turned red, Phase duration, Current phase duration, Green time duration	8	Inter-arrival times	1
Occupancy or average occupancy (in lane, link, downstream link, upstream link, intersection, network, current signal group, candidate signal group), Cell occupancies (in CTM), Relative time occupancy	7	Total stop time	1
Image-like representation (of vehicles' position, of queues), Video images	5	Acceleration of the vehicles	1
Destination of vehicles	5	Time	1
Static attributes of links, Capacity, Link indicator, Jam density	4	Vehicle information	1
Number of arriving/approaching vehicles (in link, intersection), Maximum arrivals in the green phase	4	Direction of the vehicle	1
Node where each vehicle stands at	3	The cloud control from traffic could center	1
Value of an objective function	1	Detector activation state	1
Number of incoming lanes (in intersection)	1		

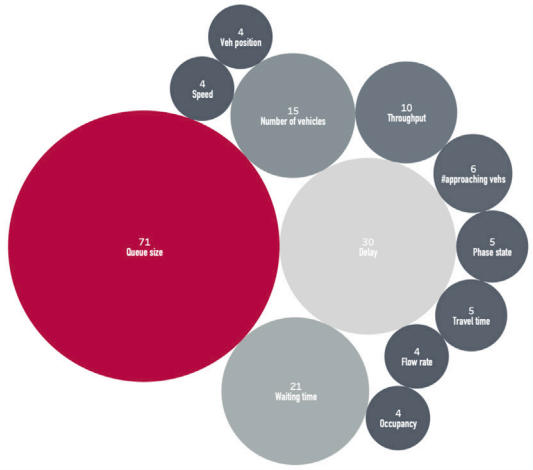
Fig. 8. Elements in state definitions: (Top) The most frequently used elements in state definitions. The numbers indicate the frequency of occurrences in the included papers, and (Bottom) All elements used in state definitions (with their frequency).

Table 6
Categorization of types of traffic signal based actions in the RL methods. The numbers in parentheses indicate the number of publications in which an action type is used.

Class	Cycle length	Phase duration	Phases order	Frequency	Cycle-/phase-based (#)
2	fixed	fixed	variable	(2)	cycle-based (48)
3	fixed	variable	fixed	(13)	
4	fixed	variable	variable	(4)	
5	variable	variable	fixed	(18)	
6	variable	variable	variable	(11)	
7	–	variable	–	(100)	phase-based (100)

Other action types		
Class	Description	Frequency
Act1	set the value of a threshold metric for each traffic signal	(2)
Act2	set a link specific metering rate in the perimeter control	(1)
Multiple Actions	including: (**Act3**) set the value of a threshold metric for each traffic signal, (**Act4**) set a link specific metering rate in the perimeter control, and (**Act5**) select a route as a driver's action	(3)

nodes often corresponds with the different types/sources of input data. The number of nodes within the hidden layers depends on what each layer is designed to do, as well as changes with the purpose of the network. There are many types of NNs in deep RL.



ELEMENTS IN REWARD DEFINITION	#	ELEMENTS IN REWARD DEFINITION	#
Queue size or average queue size (in phase, lane, link, intersection, neighboring intersection), Number of waiting vehicles (in lane, link, intersection), Congestion level or average congestion level (in intersection, neighboring intersections), Pedestrian queue size (in intersection), Maximum queue size (in phase, lane, link), Number of stopped vehicles that are passing the intersection, Congestion cost, Number of stopping vehicles (in network), Number of vehicles being stopped when signal is switched from green to red, Number of stopped vehicles (in link), Congestion based on comparison between consecutive phases - no threshold is set as in congestion level (in links)	71	Total number of intersections, Number of neighboring intersections, Number of intersections that are involved in the decision about the new plan	1
delay or average delay (in link, intersection), total cumulative delay (in intersection, network), red light delay, delay of each vehicle, average delay per vehicle, average travel delay of vehicles, delay of leaving vehicles (in intersection), delay of stopped vehicles, delay incurred during the yellow-red transition	30	Presence of public transport vehicles (in link)	1
waiting time or average waiting time (in link, intersection), average waiting time level (in intersection), cumulative waiting time (in intersection), waiting time for the vehicles in cells on the links	21	The time required to travel between the upstream and downstream detectors	1
N/A	18	Number of emergency vehicles (in network)	1
Number of vehicles (in cycle, neighboring cycle, link, intersection, network), Number of vehicles in approaches receiving green, Number of vehicles in approaches receiving red	15	Capacity (link)	1
Number of vehicles passed the intersection, Throughput (in lane, link, intersection, network), Total discharge, Number of vehicles exit the network at each time step, Total volume of passing vehicles	10	Total stop time	1
Number of approaching/arriving/moving vehicles	6	The distance traveled by the vehicle in the current time step	1
Current phase, Indicator of light switches (0 or 1), Signal state, Phase state	5	Pressure (based on density)	1
Travel time, Average travel time, Total travel time of vehicles, Total travel time of vehicles passed the intersection	5	Number of of official vehicles, e.g. bus, taxi (in network)	1
Position of the vehicles	4	Vehicle information	1
Speed or average speed (in cycle, neighboring cycle, link, intersection), Speed of the vehicles	4	Number of stops	1
Flow rate (in lane, on congested link, passes through the cordon)	4	A penalty for giving green to approaches with congested receiving (downstream) links	1
Number of vehicles enter (the intersection, the network)	3	The cloud control from traffic could center	1
Occupancy or average occupancy (in lane, link, intersection, network), Occupancy ratio of waiting vehicles (in network),	4	An incentive to give green to approaches expected to receive several vehicles from upstream links	1
Number of departed vehicles (from the roundabout)	2	The importance of the vehicle information and cloud control flow	1
Whether crossing the intersection or not (vehicle level)	2	Number of vehicles within a certain distance from the intersection	1
Density or average density (of the phase in intersection), Jam density (in link)	2	Distance travelled	1
Cycle length	2	Whether the lane is on the main road or not	1
Fuel consumption, Energy consumption (in intersection)	2	Value of an objective function	1

Fig. 9. Elements in reward definitions: (Top) The most frequently used elements in reward definitions. The numbers indicate the frequency of occurrences in the included papers, and (Bottom) All elements used in reward definitions (with their frequency).

Table 7
First column) approximation methods used in RL-NTSC, and second column) the types of NNs used in RL-NTSC.

Approximation method	Neural Networks types
NN	denoising auto encoder (DAE)
tile coding	Deep NN
linear model tree	feed forward NN (FFNN)
simultaneous perturbation stochastic approximation (SPSA)	recurrent NN (RNN)
smoothed functional (SF)	convolutional NN (CNN)
phase gate	deep convolutional network (Deep CNN)
decomposable Q-function (DQF)	perceptron
radial basis functions (RBF)	graph attentional network (GAN)
triangular-shaped functions (TF)	DNN
linear function approximation (Linear FA)	graph convolutional network (GCN)
average cost algorithm	incremental Gaussian mixture network (IGMN)
iterative approximation	back propagation NN (BPNN)
KNN	fuzzy NN (FNN)
linear combination (LC)	neural fitted Q-iteration (NFQI)
cerebellar model articulated controller (CMAC)	wavelet NN (WNN)
connectionist Q-learning framework (CQF)	
gradient-descent linear function approximation	
double deep Q-learning (DDQ)	
fuzzy granulation	
linear regression (LR)	
sigmoid regression (SR)	
long short-term memory (LSTM)	
linear Q-function	

- Artificial Neural Networks (ANN): Known as a feed forward network, all incoming information is only processed and pushed in the forward direction.
- Recurrent Neural Networks (RNN): Unlike ANN, this neural network pushed processed data back to previous layers and nodes. It shares parameters across different time steps and results in fewer overall parameters. The fewer parameters allows for a smaller network.
- Convolutional Neural Networks (CNN): These networks use an extra step called convolution (for which it was named), which involves applying different "filters" to reduce the complexity of the input data. As the information filters through the network, these filters can be applied to highlight specific features of the data. This is one of the most common neural networks and is used in many disciplines.

With neural networks, deep RL becomes highly suited for complex environments presented by intersections and the dynamic changes that occur within a day of traffic.

- *Tile Coding*: Tile coding is another well-known function approximator. Unlike the continuous methods such as radial basis functions (RBF), tile coding is a discretization method which is used in RL. It is a piece-wise constant approximation method that approximates the action-value functions by partitioning the state space into small regions with a constant reward value. Tilings design considers three main components: width of tiles, the resolution, and the number of required tilings based on the hyper-volume of the whole state space. For more information about Tile coding method, the reader can refer to Abdoos et al. (2014).

Deep reinforcement learning based methods. Deep RL takes a different approach when dealing with the complex influx of data associated with traffic. It incorporates neural networks into RL algorithms and combines the advances in training layered neural networks into abstract high-level representations of the raw input data, giving non-linear methods (El-Tantawy et al., 2015; Li et al., 2018). These "layers" allow the agent to look at smaller, more reduced versions of data to extract information without an overload. The neural networks are what allow an algorithm to go "deep" and work with large or complex data input more efficiently.

Here, we compare Q-learning and Deep Q-learning. Q-learning, is the process of iteratively updating Q-Values for each state–action pair using the Bellman Equation until the Q-function eventually converges to Q*. Instead of estimating the Q-value of each state–action pair separately in Q-learning, deep reinforcement learning algorithms (Mnih et al., 2013) use deep neural networks as function approximators to map from states to Q-values. This makes possible the use of a larger and/or continuous state space through parameterization (Lillicrap et al., 2015). The integration of artificial Neural Nets (NNs) into the Q-learning process is referred to as Deep Q-learning, and a network that uses NNs to approximate Q-functions is called a Deep Q-Network (or DQN). In other words, DQN is a Q-learning, which is parameterized with deep NN with parameters θ, i.e. $Q(s, a; \theta)$. The neural network input is the state, the number of output neurons is the number of the possible actions, and the targets are the Q-values of each of the actions.

Unlike Q-learning, whose convergence in the limit (infinity) is guaranteed, we do not have such guarantees for DQN. This is because (i) the data set is not i.i.d. and (ii) as the agent learns, the targets move (Van der Pol & Oliehoek, 2016). To ameliorate this issue, different methods can be used, such as dueling architecture (Wang et al., 2016) to improve stability and target network (Mnih et al., 2015) to solve the overoptimistic problem. Dropout (Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014) can also be used to make the controller more robust and prevent the neural network from overfitting.

Learning rate needs to be optimized in training deep neural networks. To this purpose, different optimization methods can be used, such as stochastic gradient descent, Adagrad, and Rmsprop. In this line, Adam optimizer (Kingma & Ba, 2014) is an adaptive learning rate optimization algorithm, which is computationally efficient, has little memory requirements, and is generally fairly robust to the choice of hyper parameters (Goodfellow, Bengio, Courville, & Bengio, 2016). In addition, experience replay is also used to help with stability and convergence behavior of the algorithm when using a non-linear function approximator.

Deep learning was used in 27 (17%) papers. This is certainly a reasonable portion of the study, and the point to be noted is that using deep learning in RL in the network-scale began in 2016 with 2 papers. After a year with no cases, deep learning was used in 5 studies in 2018, followed by a sharp increase in 2019 with 18 papers. It is expected that this trend will continue into the future, as in the first season of 2020, 3 out of 6 research papers used deep learning. The papers that used deep RL methods are identified in Table 5. SUMO has the frequency of 16 (out of 28) in conducting simulations for the deep learning related methods. The traffic simulation software tools used for deep learning are depicted in Fig. 10.

4.3. Environment attributes and traffic simulation

In this section, we discuss the environment attributes, including network type, vehicle class, data source, and data communication processing. This classification presented in Fig. 11.

4.3.1. Simulation and simulated networks

16 traffic simulator software or platforms are identified in these studies to simulate the traffic. These include SUMO,[4] VISSIM,[5] PARAMICS,[6] GLD,[7] AIMSUN,[8] ITSUMO (da Silva, Bazzan, Andriotti, Lopes, & de Oliveira, 2004), CityFlow,[9] TRANSYT[10]/TRANSYT-7F,[11] MATLAB,[12] AIM (Dresner & Stone, 2008), TSIS,[13] MATISSE (Torabi, Al-Zinati, & Wenkstern, 2018), USTCMTS (used in Shi and Chen (2018)), SMPL (used in Su and Tham (2007)), and SeSAm (used in Bazzan et al. (2007)). See the distribution of frequency of the traffic simulators in included papers in Fig. 10. Since the source of the last three simulation software tools is not found, we referred the reader to the papers in which they are used, for further information.

Of interest is that SUMO was used for the first time in 2015 and has since become the most frequently used software in this area, with 17 out of 27 occasions in 2019 alone. The second most frequently used software, (i.e. VISSIM) started being used in 2010, and PARAMICS, the first well-known traffic simulator has been in use since 2003. Prior to this the tools were either custom-built or not clearly outlined in the research. In 18 studies, the authors designed or used a custom-built environment for simulations. What is surprising is that 27 studies (16.7%) did not state the simulation tools that they used for the test, not including the those that did not use any simulations.

With regard to the type of maps used to test the proposed RL methods by the included studies, of those that used maps, 100 (62%) papers used a synthetic map, 45 (28%) papers used a real-world map, and 9 (6%) papers used both types, see Fig. 12. Moreover, 49 real maps are used from 14 countries for simulation purposes, with China topping the list at 10 maps, followed by the USA, Ireland and Canada, see Fig. 12. These maps are mostly large-scale networks of intersections.

[4] https://www.eclipse.org/sumo/.
[5] https://www.ptvgroup.com/en/solutions/products/ptv-vissim/.
[6] https://www.paramics.co.uk/en/
[7] http://www.sf.net/projects/stoplicht.
[8] https://www.aimsun.com/.
[9] https://cityflow-project.github.io/.
[10] https://trlsoftware.com/products/junction-signal-design/transyt/.
[11] https://mctrans.ce.ufl.edu/mct/index.php/hcs/transyt-7f/.
[12] https://www.mathworks.com/products/matlab.html.
[13] https://mctrans.ce.ufl.edu/featured/tsis/Version6/index.htm.

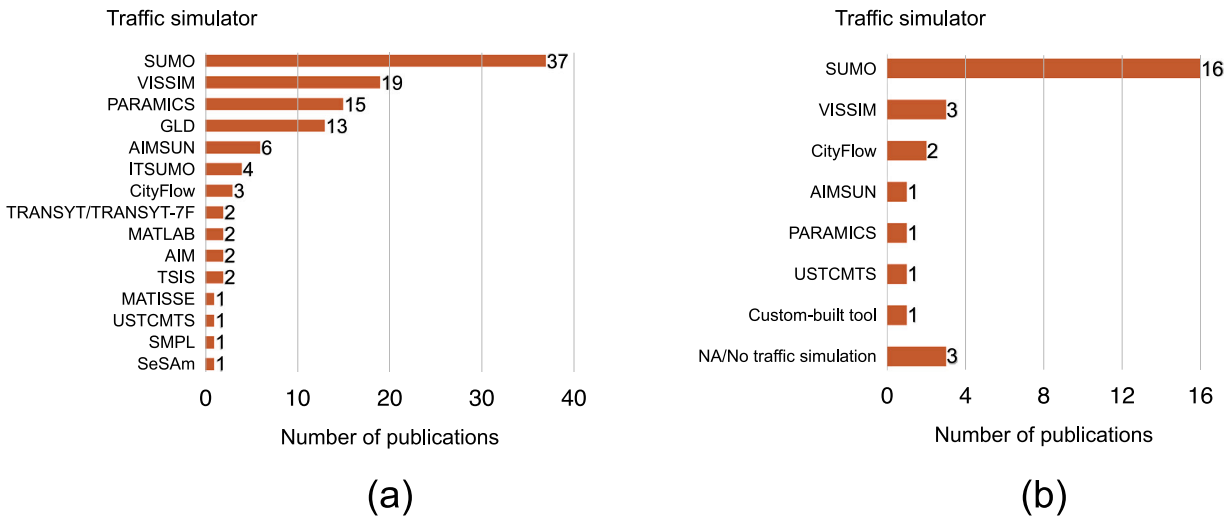


Fig. 10. Traffic simulators: (a) distribution of frequency of the traffic simulators in RL-NTSC, and (b) distribution of frequency of the traffic simulators in deep RL-based methods in RL-NTSC.

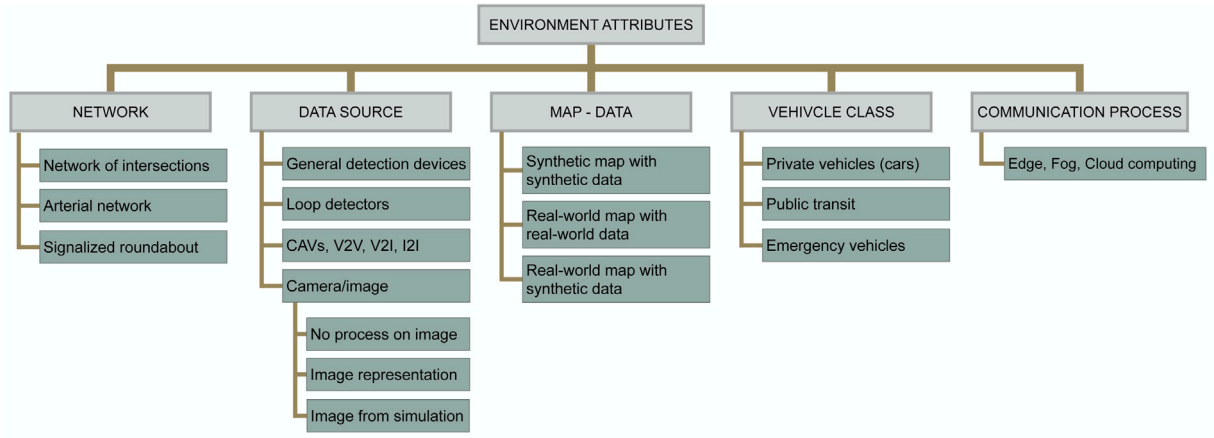


Fig. 11. Environment attributes in RL-NTSC.

Fig. 11 shows the distribution of the number of intersections studied. Though many studies are still currently being held with fewer than a dozen intersections, there is also work on 25+ intersections. And while 44% of the papers run simulations in small-scale networks with 8 intersections or lower, we observed that the recent trend is to test the proposed methods in medium and large-scale networks. The size of the network is also important as it may impact the exponential growth of state-space and action-space and generally the complexity and computational effort required to reach a solution through an RL method, whether for training or testing stages.

We categorized the type of the testbed network into three main groups, including: (1) the network of intersections, (2) arterial network, and (3) the signalized roundabout networks. As already mentioned, the isolated intersection case is excluded from our study. The arterial network is an open network, as compared to a closed network. Arterial network control is applied on a sequence of intersections to provide preference to progressive traffic flow along the arterial. Unlike the isolated intersections, the intersections in the arterial network operate as a system and the system coordinates timing of adjacent intersections. On the other hand, the network of intersections is considered as a closed loop. Being a closed loop, the network demands at least four intersections. The networks of two and three intersections may have the characteristics and behavior of both groups (the network of intersections and arterial network); therefore, we considered them separately as two sub-groups under the first group, i.e. the network of intersections. Finally, the signalized roundabout networks deals with both approaching and circulatory lanes, which are fundamentally different from the first two groups. The signalized roundabout networks involve more than one set of traffic signals in a node. Rizzo et al. (2019a, 2019b), both published in 2019 are only two papers in the literature that studied the signalized roundabouts, which indicates a potentially open research problem on the RL area.

The dynamics of signalized roundabouts is complex because it deals with both approaching and circulatory lanes. The conflicts between approaching and circulatory flows cannot be solved by metering only the approaching lanes and reacting consequently because the circulatory lanes may be occupied. Rizzo et al. (2019b) proposed a deep RL method for signalized roundabouts in congested networks to maximize traffic flow while being able to avoid traffic jams in connected junctions. In another research conducted in this domain, Rizzo et al. (2019a) studied the possibility of deriving explanations from a neural network agent (trained using Policy Gradient) for TSC in a signalized roundabout. How the agent learns to react differently based on each specific lane's traffic by implicitly predicting the route of the traffic and thus its future circulatory occupancy is explored. This is done by analyzing the relation between the agent phase preferences and the actual traffic, assessing the agent capability of reacting to the current detectors state, and estimating the effect of the road detectors state on the agent selected

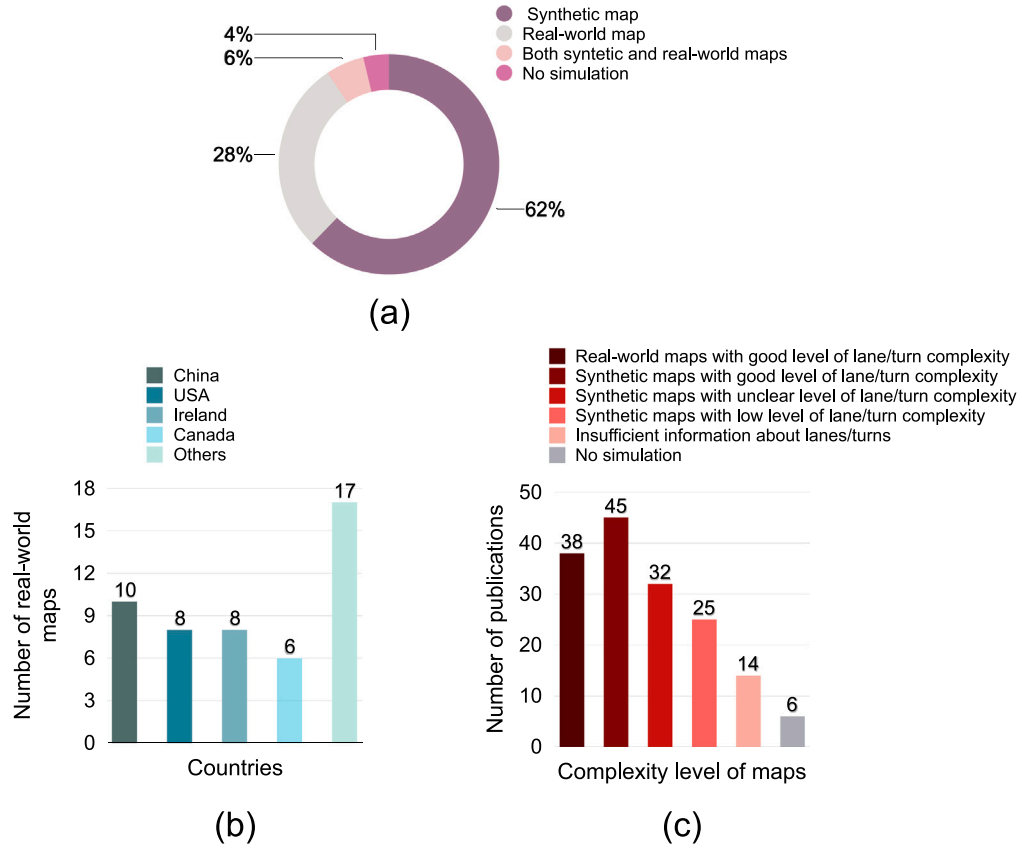


Fig. 12. Maps: (a) distribution of type of maps used in RL-NTSC, (b) frequency of real-world maps in terms of countries in RL-NTSC, and (c) distribution of type of maps in terms of level of lane/turn complexities used in RL-NTSC.

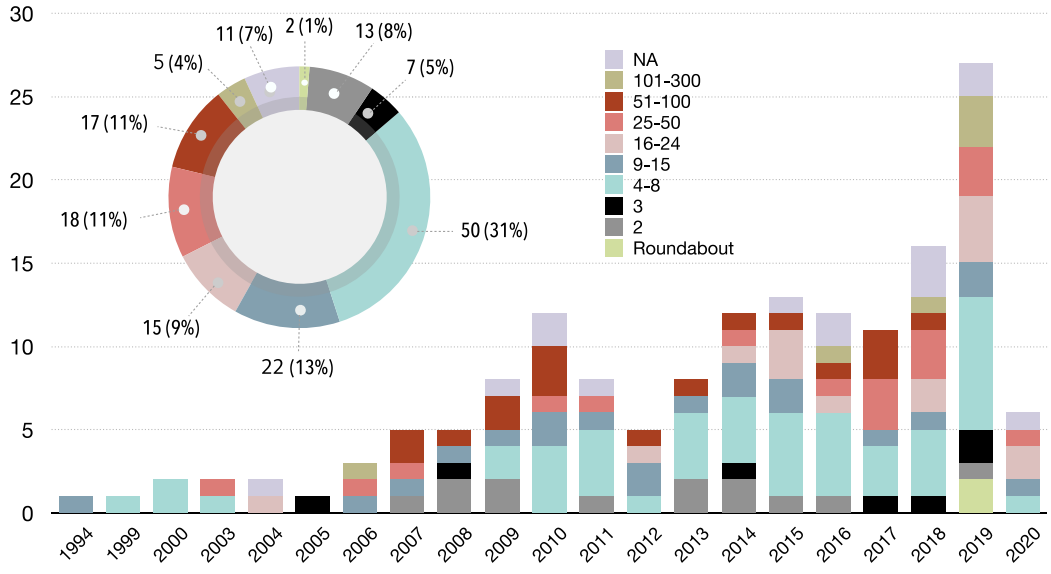


Fig. 13. Distribution of number of intersections in simulations: in all studies (pie chart), and over time (bar chart).
Notes: (1) The data of arterial and network have been merged and the highest network size in each paper is reported, (2) NA represents the papers in which the number of intersections in the arterial or network is not explicitly revealed, (3) The years with no publication, i.e. 1995–1998 and 2001–2002, are not shown.

phases through the SHAP model-agnostic technique. The results of this research reveal that it is possible to extract meaningful explanations on the decisions taken by the policy. Further research involves the study of the trade-off of accuracy in comparison with a complex deep learning controller.

And, only 17 papers worked on arterial networks (out of 189 network configurations in 160 papers), all between 4–8 intersections

except one with 16 intersections, which is the maximum number in an arterial network. The highest numbers of intersections in a network are 225 (Ni & Cassidy, 2019; Wang, Geng et al., 2016), 196 (Wei, Xu et al., 2019), and 127 (Zhou et al., 2019), respectively. The paper (Wang, Geng et al., 2016) is the first paper in the literature that tested the proposed method in a network with the highest number of intersections: 225 intersections in 2016. In 5 of the papers that we came across, the number of intersections of the network or arterial network is not released, and in 6 papers there is no simulation used. See Fig. 13.

We identified three vehicle classes, including private vehicles (cars), public transit (buses), and emergency vehicles (ambulances). Among 160 articles, only 3 of them addressed the public transit (2 papers) and emergency systems (1 paper) in RL-NTSC.

In the public transit domain, there are two types of vehicles, including cars and buses. We address streetcars in one of the three control schemes, which is streetcars bunching control. Excluding streetcars, there are 2 research works regarding buses. One considers transit priority, while another one does not. In 2014, Chanloha et al. (2014) developed a distributed CTM-Based MARL for network-scale signal control with transit priority that outperforms pre-emptive and differential priority control methods because of the improved awareness of the signal switching cost. To eliminate the need for feature extraction in the state space and to directly use available information received from the high-detailed traffic sensors (Shabestray & Abdulhai, 2019) proposed a multimodal Deep RL based traffic signal controller that combines both regular traffic and public transit and minimizes the overall travelers' delay through the intersection.

With regard to emergency vehicles, in 2017, to detect and give priority to emergency vehicles, Kristensen and Ezeora (2017) proposed a reinforced traffic control policy that reduces the waiting time of emergency vehicles at intersections as well travel time of other vehicles using a multi-agent system development framework (JADE). Also, Liu, Liu et al. (2017) is among very few articles that addressed pedestrian element along with private vehicle class in the network.

It is important that the considered map as a testbed would be consistent with a real-world set-up. For example, one-lane or one-way crossing links cannot replicate the common cases in real-world scenarios. This may dramatically impact the usability and performance evaluation of the methods, specifically in terms of computation efficiency. Based on the collected data, most of the papers used a good level of complexity in the number of lanes and turns. 24% of the papers used real-world maps with multiple lanes, 28% used two or more lanes (including 13% using three or more lanes) with a good level of complexity, and 20% used the synthetic maps with multiple lanes but without significant information on the number and types of turns. See the number of publications in each category in Fig. 12. By a good level of complexity, we mean that in a regular driving style the through and left lanes are involved regardless of the right turn. The through and left lanes in the opposite approaches have conflicting right of ways and add to the state and action spaces, which increases the complexity of the problem. In the regular driving style, right turns can usually be accommodated simultaneously with either through or left lanes and do not impact the state and action dimensions. Also, in a right-side driving style, the through and right lanes are considered regardless of a left turn (similar logic to the regular driving applies here). In 25 (15%) of the papers, a low level of lane/turn complexity, including one-lane links, one-way crossing links, or multiple-lane links with only through lanes is represented. 9% of the papers did not reveal any or enough information about the lanes and turns.

4.3.2. Traffic data collection and traffic demand

We identified four categories of data source for the proposed methods, including general detection devices (67%), loop detectors (17%), vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) and infrastructure-to-infrastructure (I2I) (11%), and camera/image (5%). By general detection devices, we mean that the authors did not specify any specific data-source. The papers that use loop detectors provide either no specific design or specific designs like (1) one detector at the stop-line, (2) one detector at a distance from the stop-line, (3) one detector at the upstream (end of the lane), (4) two detectors at the stop-line and at a distance from the stop-line, or (5) two detectors at the stop-line and at the end of the lane.

The third category covers the methods in V2V, V2I, and I2I environment. Connected vehicles is a specific class in this category where vehicles can exchange data with other vehicles and the infrastructure. The fourth category presents the methods that just use traffic cameras as a data source without any process on images or are inherently image-based. In image-based methods, there are two directions: (i) an image-like representation, i.e. image representation of vehicles' positions, where an image is a matrix of pixel values of a view of intersections, and (ii) images retrieved from traffic simulation software are used as the data source.

In line with the data and the data sources, computing paradigms in the data communication process in RL-NTSC is a new research area and the research on application of computing paradigms, specifically edge and fog computing, in RL-NTSC is topical and has started since 2019 with three papers. Gao et al. (2019) designed a framework of edge computing under the NTSC scene and proposed a cooperative NTSC algorithm based on MARL to avoid the curse of dimensionality, provide minimal response time, and reduce network load. Zhou et al. (2019) proposed a large-scale Edge-based RL (ERL) solution to better alleviate congestion in complex traffic scenarios based on Edge Computing nodes for traffic data collection. They concluded that ERL in distributed edge servers has much better scalability and faster Deep NN training than the cloud service. Wu et al. (2019) proposed a traffic control architecture based on fog computing paradigm and a distributed RL algorithm (that connects traffic signals, vehicles, fog nodes and traffic cloud) to overcome communication bandwidth limitation and reduce communication delay, make real-time traffic condition information available to vehicles, and lower the probability of traffic congestion in the city through generating traffic signal control flow and communication flow for each intersection. This is not only suitable for current vehicles but also more useful for driverless vehicles anticipated in the future, as it will be able to plan its route much more intelligently with information from the fog node.

Although there are numerous methods listed in the literature, for traffic signal control, they are mostly effective in low traffic loads. The main challenge in traffic control is managing the high load of traffic representing rush-hour in the morning or afternoon in a network that may lead to a spillback condition in different links. Our research found that 74% of the studies simulated the traffic condition in high demands, close to saturation, saturated, or oversaturated conditions to test the efficiency of the proposed methods in traffic congestion (labeled as *Sat* in Table 3). Nevertheless, among these, only 5% explicitly applied and addressed a spillback prevention strategy, and 6% analyzed or mentioned that the proposed method is able to prevent spillback (labeled as *Spillback* in Table 3). 6% of the papers explained spillback but never considered or applied it, whereas in 56% of the studies with high demand, spillback is not even mentioned. 22% neither addressed high demand and spillback, nor explicitly mentioned them (labeled as *Neither* in Table 3), and 4% provided no evaluation (labeled as *No Evaluation* in Table 3).

4.4. RL-NTSC application domains

Fig. 14 depicts the NTSC application domains (or scenarios) and the frequency of publications in each domain based on the control method scheme and environment attributes defined in Figs. 7 and 11. The references to the publications, except for the first row to save space, are also provided. This figure helps to locate which areas have already researched, what are the publications in the areas, and in which areas there is still room for improvement and development. This can be found out based on the frequency of the publications in the areas.

Control method scheme	Vehicle class	Network	Communication process	Data source	Image data appliacation	CAVs application	Number of publications [reference]
Regular network control	Private vehicles	Network/ Arterial	-	General/Loop detectors	-	-	127
			-	V2V/V2I/I2I	-	CAVs	6 (Houli et al. 2010, W. Liu et al. 2014, W. Liu et al. 2017, Al Islam et al. 2018, Aziz et al. 2018, Vinitsky et al. 2018)
					-	-	9 (Wiering 2000, Kuyer et al. 2008, Khamis et al. 2012, L. Xu et al. 2013, Box and Waterson 2013, Khamis and Gomaa 2014, Gaikwad et al. 2016, Kao and Wu 2018, Lemos et al. 2018)
			-	Camera/ image	No process on image	-	2 Y. Liu et al. 2017, Gong et al. 2019)
					Image representatio	-	4 (Chu et al. 2016a, Wei et al. 2018, Reda et al. 2019, Yang et al. 2019)
					Image from simulation	-	2 (N. S. Jadhao and A. S. Jadhao 2014, J. Lee et al. 2020)
			Edge, Fog, Cloud computing	General/Loop detectors	-	-	1 (Gao et al. 2019)
				V2V/V2I/I2I	-	CAVs	1 (Q. Wu et al. 2019)
				Camera/ image	No process on image	-	1 (Zhou et al. 2019)
		Signalized roundabout	-	General/Loop detectors	-	-	2 (Rizzo et al. 2019a,b)
	Public transit	Network/ Arterial	-	General/Loop detectors	-	-	2 (Chanloha et al. 2014, Shabestray and Abdulhai 2019)
	Emergency vehicles	Network/ Arterial	-	V2V/V2I/I2I	-	CAVs	1 (Kristensen and Ezeora 2017)
Perimeter control	Private vehicles	Network/ Arterial	-	General/Loop detectors	-	-	1 (Ni and Cassidy 2019)
Streetcar bunching control	Public transit	Network/ Arterial	-	General/Loop detectors	-	-	1 (Ling and Shalaby 2005)

Fig. 14. NTSC application domain and frequency of publications in each domain.

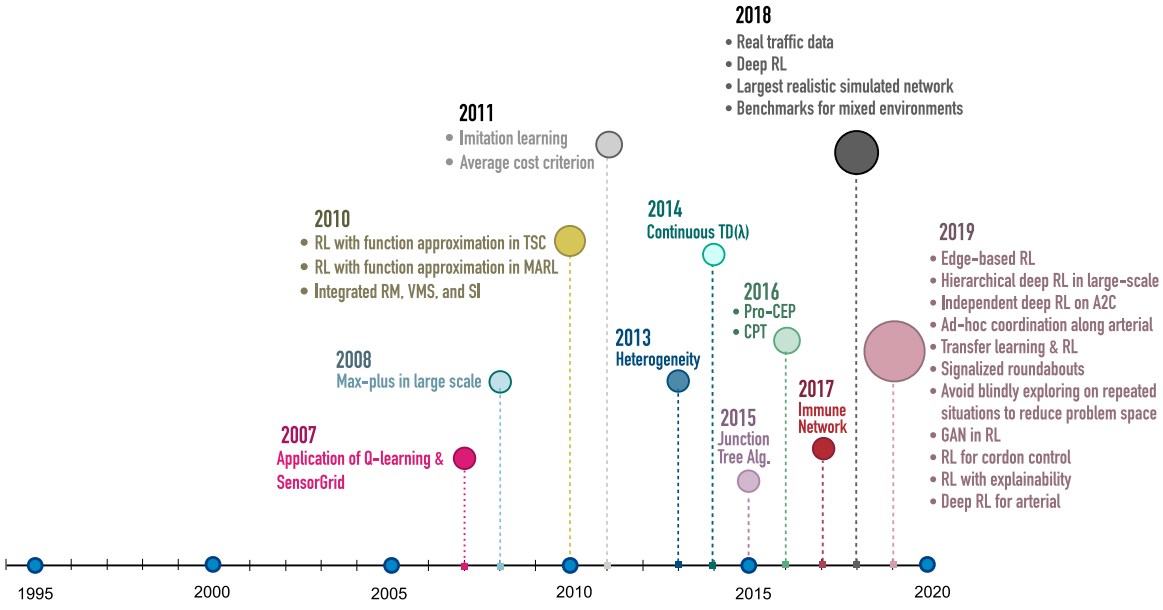


Fig. 15. Major first events in RL-NTSC collected from the included papers.

4.5. Major first events in RL-NTSC and authors' key statements

Fig. 15 is an infographic timeline that aids in identifying past and current trends, specifically highlighting the research areas and challenges that have come to the fore most recently. It essentially indicates major first events in RL-NTSC. This helps to find out since when a research line has started. In addition, Table 8 includes key statements made by the authors of the studies included in this paper.

4.6. Code availability

Still another feature that we investigated is the availability of the code, which we considered a good resource for those new to the field, and useful for reproducing the research. We found that in 10 papers in NTSC the authors made their code available, and these can be found in Table 5. Brys et al. (2014) is the first paper in the area of RL in NTSC that made the code available in 2014. 7 of these 10 papers are deep methods published in 2018 and 2019. 2 papers provide actor-critic methods, while the rest are based on Q-learning. The codes are written for SUMO (5 papers), CityFlow (3 papers), GLD (1 paper), and AIM (1 paper). The approximation methods include NN, SPSA, Phase gate, and Tile coding.

4.7. Evaluation

150 (94%) of the studies provided an evaluation, while 7 (4%) did not. Three (2%) papers provided self-comparison, meaning that they compared the variations of the proposed methods with each other, but not with other NTSC methods. The papers that did not provide an evaluation are identified in Table 5.

Table 8
The details of major first events in RL in NTSC collected from the included papers.

Citation	Statement
(Su & Tham, 2007)	This **application of Q-learning and SensorGrid** can be seen as the first step towards expanding the usage of SensorGrid.
(Kuyer et al., 2008)	The first application of **max-plus** to a **large-scale** problem (not in small applications) and thus verifies its efficacy in realistic settings.
(El-Tantawy & Abdulhai, 2010)	The first study that has tackled the Integrated traffic control problem (Ramp Metering (RM), Variable Message Signs (VMS), and Signalized Intersections (SI)) to find a **closed-loop optimal control solution** using a **coordination** mechanism that minimizes the communication requirements.
(Prashanth & Bhatnagar, 2010)	The first application of RL with function approximation for NTSC.
(Waskow & Bazzan, 2010)	The first attempt to tackle the dimensionality problem in MARL by means of **function approximation**.
(Natarajan et al., 2011)	The first adaptation of a Statistical Relational technique for the problem of learning relational policies from an expert (**imitation learning**).
(Prashanth & Bhatnagar, 2011)	The first to design RL-based NTSC algorithms that minimize a long-run **average cost** criterion.
(Nuli & Mathew, 2013)	No traffic adaptive control model exists to account for **traffic heterogeneity** and limited lane discipline.
(El-Tantawy et al., 2014)	The first study to investigate the effect of TD(λ) methods for NTSC as a **continuing task** (i.e., not a finite episode) with a **discounted reward** in which looking ahead to future steps is less important compared to a finite episodic task with undiscounted reward.
(Zhu et al., 2015)	Junction Tree Algorithm has not been applied to address the coordinated signal control problem.
(Wang, Geng et al., 2016)	Other works with the same functionality (predicting future system states by Proactive Complex Event Processing) have not been found.
(Prashanth et al., 2016)	The first work to combine **cumulative prospect theory (CPT)** with RL, and to investigate (and define) **human-centered RL**.
(Darmoul et al., 2017)	The first to integrate Case-based reasoning and RL for NTSC and integrate **immune features** within MARL to achieve disturbance management.
(Wei et al., 2018)	None of existing studies have used the **real traffic data** to test their methods.
(Li et al., 2018)	The study of the **applicability of deep RL** on the road network has not yet been carried out.
(Torabi, Wenkstern et al., 2018)	Validated on the **largest realistic simulated traffic network** published to date for collaborative multi-agent based NTSC.
(Vinitsky et al., 2018)	The first to propose **a standard set of benchmarks for traffic control in a micro-simulator** and a framework for simultaneously learning control for a **mixture of AVs interacting with human drivers and infrastructure** in which **deep RL** can be applied to the control task.
(Zhou et al., 2019)	Edge based RL is the first RL proposal to optimize traffic signals on neighborhood scale.
(Tan et al., 2019)	The first attempt to use **hierarchical deep RL** models in large-scale NTSC.
(Chu et al., 2019)	The first paper to present a fully scalable and decentralized MARL algorithm for the state-of-the-art deep RL agent: **independent advantage actor critic (IA2C)**, within the context of NTSC by extending the idea of Independent Q-learning on A2C.
(Wei, Chen et al., 2019)	The first time that the individual RL model **automatically achieves coordination along arterial without any prior knowledge**.
(Xu et al., 2019)	The first work to consider the impact of slow learning in RL on real-world applications by the effective **transfer of RL algorithms** trained on simulated traffic to the real-world traffic to reduce the mistakes to be made in the real world.
(Rizzo et al., 2019b)	The first to address **signalized roundabouts** in congested network, as a complex TSC scenario using a **deep RL** method.
(Zheng, Xiong et al., 2019)	The first work to **reduce the problem space** and explore different scenarios more efficiently, so that the RL algorithm can find the optimal solution within a minimal number of trials, **instead of blindly exploring on repeated situations**.
(Wei, Xu et al., 2019)	The first work to use GAN in RL for NTSC and to conduct experiments on the large-scale road network with **hundreds of traffic signals**.
(Ni & Cassidy, 2019)	The first to extend RL to the **cordon-control** problem.
(Rizzo et al., 2019a)	The first to consider **model explanation methods** such as LIME and SHAP for the explanation and interpretation of RL agents decisions that can be verified by domain experts (RL with Explainability).
(Chen et al., 2019)	This study is among the earliest to apply **deep RL** for **arterial** adaptive signal control.

The authors used different TSC methods and performance measures to compare and validate their proposed method. Fixed time methods alone are inefficient in evaluating other methods because they are unable to adapt to the traffic flow changes, however, they are sufficient to use in exploring the feasibility of a proposed method or the proof of concept. We found that 27 (17%) of the studies used only fixed time or random methods/policies for comparisons. In 39 (24%) cases, the TSC methods are used where no RL method such as actuated and adaptive methods are included, as they provide for better evaluation. In 76 (47%) studies, RL has been used either alone or with other types of methods. Involving RL methods for evaluation, however, cannot always guarantee a perfect evaluation. Generally, if an RL method is used along with actuated and adaptive methods, it can provide a great evaluation, specifically when the comparison is made with state-of-the-art RL methods proposed by the other authors in the field. We collected these RL methods as a reference for the readers in Table 9. The table also provides the citation of referenced papers that used evolutionary and meta-heuristics algorithms, real-world, fixed time, and adaptive methods.

It might be of interest to the reader to know the number of methods that are used in these papers for evaluation/comparison purposes: 1 (33%), 2 (31%), 3 (14%), 4 (5%), and 5 to 8 (5%), demonstrating that comparison of a method with only one other is the most common.

Among the performance measures, delay is the most frequent performance measure in the papers with 71 occurrences (20%), followed by travel time and waiting time and queue size (each 12%), number of stops and speed and throughput (e.g. the number of the vehicles passed the intersection) (each 6%), and environmental measures (5%), which accounts for 80% of the papers. We found 33 unique performance measures that are listed in Fig. 16. Each unique measure delegates several similar measures. The list shows a variety of measures that authors may consider or use in their following research works. Fig. 16 also depicts the top ten performance measures.

5. Discussion

5.1. Common future works and research opportunities

During the course of our investigation, a few recurring steps the authors took in order to advance their research into the future, were noticed. One of the most recommended areas for future investigation involves testing the traffic signal controllers in the real world. Given that

Table 9
Methods used for comparison.

Compared Methods	citation
RL methods	
Distributed and multi-agent RL	Araghi et al. (2015), Arel et al. (2010), Camponogara and Kraus (2003), Dusparic and Cahill (2009a), El-Tantawy et al. (2013), Jin and Ma (2018), Kuyer et al. (2008), Prabuchandran et al. (2014), Xu et al. (2013),(Abdulhai, Pringle, & Karakoulas, 2003; Choy, Srinivasan, & Cheu, 2006; Claus & Boutilier, 1998; Riedmiller, 2005; Tan, 1993; Vu, Aknine, & Ramchurn, 2018; Watkins & Dayan, 1992)
Actor-critic RL	Baird (1999), Konda and Borkar (1999), Sutton and Barto (1998)
Deep RL	Chu et al. (2019), Nishi et al. (2018), Van der Pol and Oliehoek (2016), Wei et al. (2018),(Gao, Shen, Liu, Ito, & Shiratori, 2017; Schutera, Goby, Smolarek, & Reischl, 2018; Wu, Kreidieh, Parvate, Vinitsky, & Bayen, 2017; Zheng, Zang et al., 2019)
Distributed Intersection Management Protocol	Liang, Yan, Lee, and Wang (2018)
Junction Tree Algorithm	Zhu et al. (2015)
TILDE	Blockeel and De Raedt (1998)
Propositional Function-Gradient Boosting	Dietterich, Ashenfelter, and Bulatov (2004)
Wiering Controller (TC-1)	Wiering (2000)
Traffic Controller with State Bit for Congestion	Bakker et al. (2005)
SPSA-based multi-agent	Spall and Chin (1997)
Evolutionary and meta-heuristics methods	
Genetic Algorithm	Wiering et al. (2004)
Cooperative Ensemble	Srinivasan and Choy (2007)
Real-world methods	
GLIDE (SCATS-like method in Singapore)	Keong (1993)
Fixed time methods	
Optimized fixed time (TRANSYT)	Robertson (1969)
Fixed Time	Miller (1963)
Fixed time with random offsets	Koonce and Rodegerdts (2008)
Webster method	Webster (1958)
Adaptive methods	
Saturation balancing method	Richter et al. (2006)
Self-Organizing Traffic Light Control (SOTL)	Cools, Gershenson, and D'Hooghe (2013), Gershenson (2004)
Max Pressure	Varaiya (2013)
Longest Queue First - Maximal Weight Matching	Wunderlich, Elhanany, and Urbanik (2007)
Anticipated All Clearing policy	Lämmer and Helbing (2008)
Cluster based adaptive methods	Daeinabi, Rahbar, and Khademzadeh (2011), Wang, Liu, Zhou, and Ansari (2008)

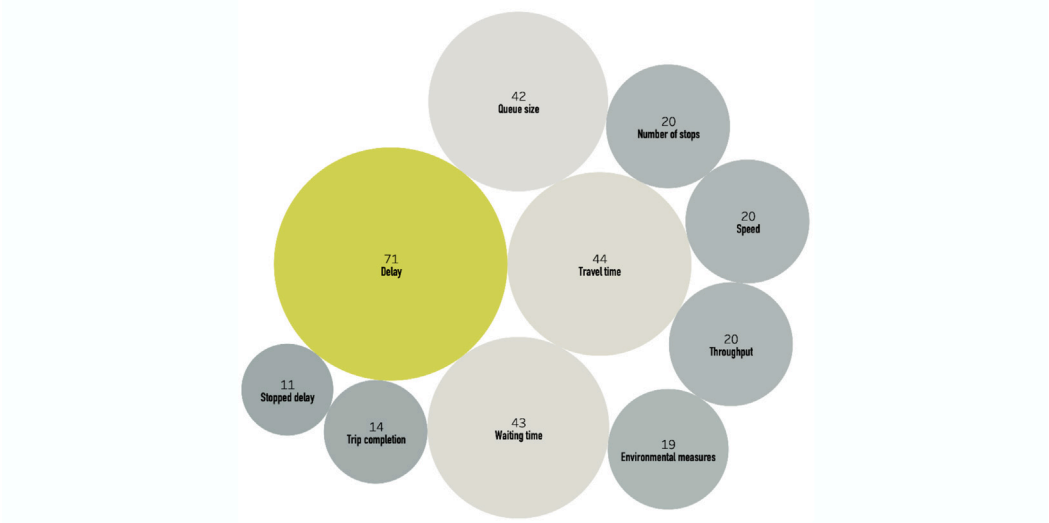
the final hurdle from theory to implementation is to see if the concept can successfully direct traffic at busy and unpredictable intersections and not just in simulations, this comes as no surprise. Authors also look to expand their work to a bigger network and to increase the number of phases that controllers could select, in addition to making traffic signal control a multi agent system and adapting to bigger intersections. In the same vein, the diversification of the proposed traffic signal formulations so that they could potentially be of greater use to more people, is also important. Adapting plans to different modes of transport, including motorized traffic such as public and mass transit, taxis, and freight vehicles, and non-motorized traffic like pedestrians and bikes are challenging at best. Incorporating better communication methods between their controllers, accounting for delays in communication and addressing noise (unwanted data) that their sensors might pick up are other points of focus for the future. Still another popular theme that arose out of our study is improving the performance of controllers. Specifically, a common direction is to change the definition of the reward function and obtain an improved state space, thereby allowing the controller to render a decision. The final future implication from this study is online learning, which is a popular choice since its strength lies in the controller's ability to continuously adapt to traffic signal conditions. Although some efforts are already underway to achieving this, such as focusing on reducing the time required for learning during this continuous process, the area is still in its infancy.

5.2. Key findings

This paper allows us to have a comprehensive view of the past 26 years of research on applying RL to NTSC. This view allows us to see that the community has employed classical approaches (e.g., Q-learning) in the vast majority of the investigations. Thus, we see a large avenue for extensions, especially given that Q-learning is a tabular method and, as such, it is not fully equipped to deal with continuous spaces and/or with centralized approaches, in which the state space tends to be vast. Related to this issue, deep learning is advancing to fill the gap left by methods that do not fully deal with huge state (and possibly, action) spaces. The number of papers employing deep learning is increasing, as demonstrated in our literature review.

The use of non-commercial microscopic traffic simulators is on the rise, with SUMO being used more and more, especially within the computer science community. Associated with this trend, is that there has been an increase in the exchange of code and experiences (e.g., SUMO has an active mailing list), which is certainly a positive trend.

Furthermore, we have noted that there is a lack of interaction between traffic and transportation engineering practitioners and researchers investigating the use of RL in NTSC. One of the consequences of this is that a high number of papers does not include or deal with real data, thus challenging the proper validation of the experimental results. Also, no testbeds are being proposed. In fact, real-world scenarios are lacking, for which one could find at the very least a detailed map



PERFORMANCE MEASURES	#	PERFORMANCE MEASURES	#
Delay, Average delay, System delay, Average delay per vehicle, Depart delay, Bus delay, Total passenger delay	71	Green time	2
Travel time, Average travel time, Pedestrian travel time, Time Through system, Person Travel Time, Time per vehicle	44	Congestion level, Congestion factor	1
Waiting time, Average waiting time, Maximum waiting time in a queue, Pedestrian waiting time, Bus Waiting time	43	Preasure	1
Queue size, Average queue size, Maximum queue size, Number of stopped vehicles, Number of waiting vehicles, Pedestrian queue size, Percent of stopped vehicles, Cross-blocking, Standard deviation of queue lengths at intersection, Stopped vehicle volume rate, Number of refused cars to enter to network in virtual queue, Maximum number of waiting vehicles in a queue	42	Distance to optimal value	1
Speed, Average speed, Harmonic speed, Distribution of speed	20	Precision-recall	1
Throughput, Number of vehicles crossed the intersection during green time, Number of passed vehicles, Passenger throughput	20	Gap between Cars, Gap time	1
Number of stops, Number of stops of buses, Trip Stops, Emercency Stops around intersection	20	Mean squared error	1
Fuel consumption,energy consumption, Emission, Air Pollution, Noise pollution	19	The time for queue dissipation after incident clearance	1
Vehicles leaving the system, Completed passenger trips, Trip completion flow, Complete Trip, Number of arrived vehicles, Total arrived road users, Completion	14	Decision time	1
Stopped delay, Stop time	11	Time loss	1
Reward, Average reward, Distribution of the discounted cumulative cost, Cumulative returns	9	Objective function value	1
N/A	9	Number of moving vehicles	1
Flow rate, Volume, Volume rate, Emergency vehicle volume rate, Maximum discharging flow	7	Arrival flow rate, Number of arriving vehicles	1
Number of vehicles (in network), Vehicles left in the system, Cumulative number of vehicles, Number of Road Users, Total number of vehicles in the coordinated lanes	5	Average lifetime of the cluster	1
Density	4	Packet delivery ratio	1
Occupancy, Occupancy Level	2	Convergence of a parameter	1
Operating cost, Communication costs	2		

Fig. 16. Performance measures and the number of their respective occurrences: (Top) The most frequently used performance measures. The numbers indicate the frequency of occurrences in the included papers, and (Bottom) All utilized performance measures (with their frequency).

(including geometry), actual demand, fixed-time signal timings and target measurements to be used for comparison purposes. Moreover, the creation of testbeds would likely bring different communities together around common goals.

Based on the revealed data by the authors of the included papers and our analysis, the literature of RL in NTSC motivates the following areas to expand as open research problems and research opportunities for future work:

- *Using different RL methods for different research topics in NTSC*: There are a few studies that compared the efficiency of different methods. For instance, in Aslani, Seipel, Mesgari et al. (2018), several methods are evaluated, including discrete state Q-learning(λ), discrete state SARSA(λ), discrete state actor-critic(λ), continuous state Q-learning(λ), continuous state SARSA(λ), continuous state actor-critic(λ), continuous state residual actor-critic(λ), which is a combination of the residual algorithm with actor-critic(λ). In this study, continuous state actor-critic(λ) showed the best robustness and performance. In another research, Chu et al. (2019) used independent advantage actor critic (IA2C) where deep neural networks are employed for both policy and value approximations. Using different RL methods may provide an insight about how to improve the performance, robustness, speed, and efficiency of the proposed methods.
- *Using various state, action, and reward elements in RL methods*: defining and designing effective states and actions and reward functions are very important in the RL process to reach efficient results. Fig. 8, Table 6, and Fig. 9, respectively, help to get familiar with the options of state, action, and reward elements that exist and already used and to guide reaching new definitions of these components.
- *Using and extending the idea of independent RL*: In independent RL, the local agents learn their own policy independently by modeling other agents in the environment. This approach is scalable, however its convergence issue needs to be addressed, like by using Experience Replay.
- *Using deep RL and hierarchical deep RL methods in large-scale networks*: The efficiency of the deep RL methods and hierarchical methods has already been discussed. The first hierarchical deep RL in large-scale network was published in 2019.
- *Using deep RL for arterial networks*: This topic was only researched in 2019.
- *Automatically achieving coordination along arterial without any prior knowledge using RL*: Coordination in arterial can be achieved by using (i) the conventional coordination systems where a fixed offset among all intersections is used (Urbanik et al., 2015), (ii) the optimization-based methods (Little, Kelson, & Gartner, 1981), and (iii) centralized RL-based optimization methods, which consider jointly modeling the action between learning agents (Van

der Pol & Oliehoek, 2016). The methods of the last option are computationally expensive as they need to negotiate between the agents in entire network. An alternative is to use decentralized RL agents to achieve coordination. Research was conducted into this interesting area once in 2019.

- *Using different function approximation (e.g. GAN) in RL methods*: It is worth noting that the application of RL-NTSC with function approximation started since 2010. Each function approximation has its own advantage and strength in different contexts, environment, and scale.
- *Establishing an appropriate tradeoff between optimality and scalability*: This is a long-run research topic that is still very important, specifically in real-world large-scale networks.
- *Defining manageable state-space, action-space, and reward function*: Considering the high volume of computation efforts in RL and the need for faster solutions, specifically in online learning, the necessity of defining efficient state-space, action-space, and reward function is of importance, while keeping the accuracy of the results.
- *Reducing the problem space*: For instance, by not blindly exploring on repeated situations, one can reduce the problem space. This was researched for the first time in 2019.
- *Defining multiple reward functions for different traffic situations*: Generally, a single reward function is defined for all traffic conditions; however, rewards can be defined dynamically as a response to the traffic states and multiple reward structure can be used (Ngai & Yung, 2011). This is done whether as pre-defined rewards for different time of analysis where the congestion level is known in advance (Houli et al., 2010), or the reward function can dynamically be adjusted with varying congestion states at the intersection (Aziz et al., 2018).
- *Using real traffic data, realistic networks, and large-scale networks, and considering traffic heterogeneity*: Real traffic are highly dynamic over time and realistic traffic network environment demands more challenges and concerns in dealing with applying RL rather than simple hypothetical setups using traffic simulations. Although there are a few research papers which recently focused on this challenge (Torabi, Wenkstern et al., 2018; Wei et al., 2018), there are still several challenges in dealing with the application of RL in the real-world setup. Considering traffic heterogeneity in traffic control models can also be of interest to research (Nuli & Mathew, 2013).
- *Using RL and SensorGrid*: The application of SensorGrid, i.e. the integration of sensor networks and grid computing, in RL started in 2007 but its usage in NTSC has never expanded since then.
- *Integrating the concepts, methods, and frameworks from other fields with RL, such as JTA (for coordinated TSC problems), Pro-CEP (for predicting future system states), CPT (as a human-centered RL), Immune Network (for disturbance management)*: There are only 4 papers that applied RL in the methods/models/frameworks from other fields (i.e. group V3). This trend has ceased since 2016. Nevertheless, the application of other methods in RL (i.e. group V2) is still in progress. Except for the general NTSC (AD1), in other NTSC application domains we do not observe any applications of V2 to V5. This essentially motivates to propose methods based on incorporating various RL methods with the methods from other fields and optimization methods, and developing theoretical aspects of RL methods.
- *Using RL in optimization problems and optimization methods in RL*: Optimization algorithms, such as swarm optimization, can be integrated with RL for improvement. For instance, swarm optimization can be applied to find the optimal parameters in the reward function as a sub-problem in RL (Lu et al., 2011). It can also be applied to rapidly find the global optimal solution for functions with wide solution space (Tahifa et al., 2015). On the other hand, RL method can be applied in the optimization problem to reach good solutions for signal timing optimization (Ozan et al., 2015). The application of integration of optimization algorithms and RL is not very recent and not very frequently used (appeared in only 8 articles) but shows the efficiency of the integration compared to using RL or the optimization algorithm alone.
- *Proposing or extending theoretical RL methods/models with feasibility assessment in NTSC*: There are only 6 articles focusing on theoretical aspects of RL which has been tested in NTSC. These aspects include: (i) learning relational policies from an expert (i.e. imitating expert) (Natarajan et al., 2011), (ii) defining a new class of multi-objective problems called CMOP (Brys et al., 2014), (iii) synthesizing a control policy for an MDP such that traces of the MDP satisfy a LTL control objective (Sadigh et al., 2014), (iv) optimizing variance-related risk measures in rewards (Prashanth & Ghavamzadeh, 2016), (v) modeling human decisions (Prashanth et al., 2016) in RL, and (vi) proposing a dynamic correlation matrix based MARL approach to reach optimal behaviors faster than other canonical learning techniques. More theoretical methods can still be proposed and examined to address the requirements of NTSC problem.
- *Using RL in combination with traffic theories (e.g. CTM, MP, shock wave theory, etc.)*: Although the usefulness of this type of combination has been shown in a few research works (Ajorlou et al., 2015; Chanloha et al., 2014; Qu et al., 2020; Wei, Chen et al., 2019), this trend can be developed more, for instance, for shock wave theory and RL.
- *Using RL as a core or combined method in integration with other methods*: A part of research works employed the integration of the methods and frameworks from other fields with RL, which are listed in Table 4.
- *Using RL in the integrated traffic systems/networks, including ramp metering, variable message signs, network of signalized intersections, arterials, and freeways*: The integration of different traffic systems is a topic that started in 2010 and contains a few research works, yet there is room for more exploration and improvement.
- *Using statistical relational techniques, such as imitation learning*: This topic started in 2011, but the number of publications in this area have been very limited since then.
- *Using Transfer learning in RL*: It is the transfer of RL algorithms trained on simulated traffic to the real-world traffic to reduce the mistakes made in the real world. This topic is currently demanding in the area of NTSC to make RL methods well suited and deployable for a real world setup.
- *Using RL in Signalized roundabouts, specifically in congested networks*: The idea of applying RL for signalized roundabouts was first proposed in 2019 in 2 articles.
- *Using RL in perimeter or cordon control*: Although there are several publications in the area of perimeter control, RL was applied to this area only once in 2019, which shows a great opportunity to investigate new solutions in this area of research.
- *Focusing on explainability when using RL*: This is a very recent topic, which is explored in 2019 for the first time, where the RL agents decisions can be explained, interpreted, and verified by domain experts. There is only 1 article published on this topic.
- *Cloud/edge/fog-based RL*: High latency communication between vehicles (or driver-less vehicles) in a connected vehicles network and limited communication bandwidth to apply the real traffic infrastructure are among the problems that NTSC still suffers from. Integrating computing paradigm such as edge and fog and RL algorithms has been shown to be a good solution for these research problems (Wu et al., 2019). This topic was researched in 2019 in 3 articles, which shows a great potential and tendency towards improving the communication efficiency when using RL.

- *Using RL for public transit, whether with or without public transit priority*: Modeling and analyzing bi-modal traffic environment, including cars and buses, are very important in urban traffic management and it is a long-lasting research area. However, there are only 2 publications in the area of RL-NTSC published in 2014 and 2019.
- *Using RL for emergency vehicles*: We found only one study that uses RL in the domain of emergency vehicles. The study models and simulates a TSC for autonomous vehicles at intersections, that also gives priority to emergency vehicles. By advancement of communication between vehicles and infrastructure, controlling traffic signals based on emergency vehicles in both traditional and connected vehicles environment using RL should become a hot topic in the area of RL-NTSC.
- *Using RL in image-based NTSC methods*: Based on our systematic review in the area of RL-NTSC we did not find any articles that uses real images from video detection devices as a data source for the proposed RL methods. Instead, image representation and images from simulation have been used for this purpose. Image-based RL methods in NTSC will be a great avenue for research to adopt to real-world situations.
- *Using RL in automating streetcar bunching control in transit routes*: This interesting idea is not new (back to 2005) but it was never extended in further research works.
- *Considering the use of RL in mixed environments, including regular, connected and autonomous vehicles*: With the development of wireless communication, connected vehicles environments (called as vehicular ad hoc network or VANET), provides the capabilities to collect real-time traffic information for adaptive TSC. The connected vehicles technology facilitates two communication modes, including V2V and V2I, where the vehicles send vehicles identification number (ID), position, current time, speed data, and a timestamp to the intersection agents. And the intersection agents process the information, and can share it with the other neighboring intersection agents. Using RL in this type of network in NTSC is common and research in this area still continues to address several challenges, such as the difficulty of transportation modeling and optimization.
- *Establishing standard benchmarks for traffic control in traffic simulators*: Benchmarks helps researchers to concentrate on algorithmic improvements and control techniques rather than system and environment design. Using the benchmarks, the researchers can evaluate their results against other compared methods effectively. In different contexts, several benchmarks have been prepared to evaluate and compare RL methods, such as the Arcade Learning Environment (ALE) (Bellemare, Naddaf, Veness, & Bowling, 2013) for evaluating algorithms designed for tasks with high-dimensional state inputs and discrete actions, rllab (Duan, Chen, Houthooft, Schulman, & Abbeel, 2016) for tasks with partial observations and hierarchically structured tasks, and NGSIM dataset (US Department of Transportation, 2008) for microscopic data on human driving behavior. Vinitsky et al. (2018) is the first and only article that provides new benchmarks in the use of deep RL in a micro-simulator to create controllers for mixed-autonomy traffic, where CAVs interact with human drivers and infrastructure. They characterize a set of RL algorithms by their effectiveness in training deep NN policies.

  As indicated in this study, the unexplored open questions include fairness, decentralization, generalization, and sample-efficiency. Future works can provide similar standard benchmarks in other traffic simulators and to address the abovementioned open questions in addition to considering other factors and cases (such as considering buses or integration of different methods with RL) and for other contexts in NTSC.
- *Employing online learning*: This line of research in RL-NTSC started with Choy et al. (2003) in 2003 and continued in 5 other articles (Cai et al., 2009; Dai et al., 2011; Dusparic & Cahill, 2012; Srinivasan et al., 2006; Yin et al., 2015), where the TSC agents need to continuously learn in the traffic network and update the weights and connections in the NN in real-time. The regular RL methods like Q-learning and W-learning require exploration periods while learning optimal actions and weights and make online learning almost impossible. Moreover, in the online learning process when dealing with an infinite horizon control problem, the issue of lack of stochastic exploration and the possibility of getting stuck in local minima need to be addressed (Kohonen, 2012; Yen, Yang, & Hickey, 2002). Multistage online learning processes that involve reinforcement learning, weight adjustment, and adjustment of fuzzy relations are proposed in Choy et al. (2003), Srinivasan et al. (2006). Cai et al. (2009) investigated two online learning techniques, including reinforcement learning and monotonicity approximation. A reinforcement training based online learning NTSC is designed in Dai et al. (2011), which employs a feed-forward neural network. Dusparic and Cahill (2012) used DWL and Yin et al. (2015) used ADP for online learning. The online learning process helps continue learning in real-time, where the adaptation of approximation is processed online.

It is also worth noting that the classification provided in Fig. 14 can help with finding the open research problems. For instance, based on this figure, there is only one article that presents the application of RL in the perimeter control. We also see that the focus of the paper is on general detection and network of intersections. Thus, different open research questions can be explored, such as extending the perimeter control for a bi-modal environment, considering bus priority, image-based traffic data detection, or perimeter control in a connected vehicle environment. This logic can also be applied for the other application domains. Furthermore, more open research questions can be found based on a range of excluded papers provided in step 5 of Section 3.1. By considering NTSC within the context of these excluded papers, this becomes possible. For instance, application of RL-NTSC considering or combining with following components: bikes, pedestrians, route choice, ride-sharing, fleet management, and image processing.

6. Threats to validity

There are threats to the validity of the results and findings of our review, which will now be discussed. Although attempts were made to select our search systematically and so that they capture the existing articles in the area under investigation, part of the included articles were retrieved during the forward and backward snowballing process. This suggests that the possibility of losing part of the existing evidence is real. Moreover, research papers that may be relevant based on our criteria might have been excluded. The authors of this paper strove to collect as much relevant data as possible, and to cross reference the information for accuracy, inaccuracy remains a possibility due to the large number of research papers and features we were dealing with. Differences in our understanding, as well as the intersection of concepts with RL, such as GT, ADP, DP, and LA may have also led to omitting some relevant research papers that the key term ”reinforcement learning” in our search string could not capture.

7. Conclusion

This paper presented a comprehensive, systematic literature review on the application of RL in NTSC. The main goal of this research is to identify all eligible articles in the defined area, analyze the data of the included articles, provide statistical and conceptual knowledge based on the qualitative and descriptive data analysis, provide the highlights, variety of the applied methods, patterns, trends, frequency of existing

research works in various application domains, major first events in RL-NTSC, common future directions based on what the included papers recommended, and other useful information for further research in the area of RL-NTSC. In addition to the detailed material throughout the paper, the key findings are summarized and discussed. Considering all the published review papers we uncovered in this area in our literature review, this paper covers the highest number of articles.

The review of the literature for the application of RL in single isolated intersections can be considered as an implication for future practice, which is complementary to this review paper. The integration of the results and findings of both scales can provide useful insights.

CRediT authorship contribution statement

Mohammad Noaeen: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Visualization, Project administration. **Atharva Naik:** Validation, Investigation, Writing – original draft, Writing – review & editing. **Liana Goodman:** Validation, Investigation, Writing – original draft, Writing – review & editing. **Jared Crebo:** Validation, Investigation, Writing – original draft, Writing – review & editing. **Taimoor Abrar:** Validation, Investigation, Writing – original draft, Writing – review & editing. **Zahra Shakeri Hossein Abad:** Conceptualization, Methodology, Formal analysis, Writing – review & editing, Visualization, Supervision. **Ana L.C. Bazzan:** Conceptualization, Writing – review & editing, Supervision. **Behrouz Far:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Abad, Z. S. H., Gervasi, V., Zowghi, D., & Far, B. H. (2019). Supporting analysts by dynamic extraction and classification of requirements-related knowledge. In *2019 IEEE/ACM 41st international conference on software engineering* (pp. 442–453). IEEE.

Abad, Z. S. H., Noaeen, M., & Ruhe, G. (2016). Requirements engineering visualization: a systematic literature review. In *2016 IEEE 24th international requirements engineering conference* (pp. 6–15). IEEE.

Abdelgawad, H., Abdulhai, B., El-Tantawy, S., Hadayeghi, A., & Zvaniga, B. (2015). Assessment of self-learning adaptive traffic signal control on congested urban areas: independent versus coordinated perspectives. *Canadian Journal of Civil Engineering*, *42*, 353–366.

Abdoos, M., Mozayani, N., & Bazzan, A. (2011). Traffic light control in non-stationary environments based on multi agent q-learning. In *2011 14th International IEEE conference on intelligent transportation systems (Vol. 158)* (pp. 1580–1585). IEEE.

Abdoos, M., Mozayani, N., & Bazzan, A. (2013). Holonic multi-agent system for traffic signals control. *Engineering Applications of Artificial Intelligence*, *26*, 1575–1587.

Abdoos, M., Mozayani, N., & Bazzan, A. (2014). Hierarchical control of traffic signals using q-learning with tile coding. *Applied Intelligence: The International Journal of Artificial Intelligence, Neural Networks, and Complex Problem-Solving Technologies*, *40*, 201–213.

Abdoos, M., Mozayani, N., & Bazzan, A. (2015). Towards reinforcement learning for holonic multi-agent systems. *Intelligent Data Analysis*, *19*, 211–232.

Abdulhai, B., & Kattan, L. (2003). Reinforcement learning: Introduction to theory and potential for transport applications. *Canadian Journal of Civil Engineering*, *30*, 981–991.

Abdulhai, B., Pringle, R., & Karakoulas, G. J. (2003). Reinforcement learning for true adaptive traffic signal control. *Journal of Transportation Engineering*, *129*, 278–285.

Ajorlou, A., Awasthi, A., & Aghdam, A. G. (2015). Distributed urban traffic control based on locally observable cell occupancies. In *2015 American control conference (Vol. 103)* (pp. 1035–1040). IEEE.

Al Islam, S. B., Aziz, H. A., Wang, H., & Young, S. E. (2018). Minimizing energy consumption from connected signalized intersections by reinforcement learning. In *2018 21st international conference on intelligent transportation systems* (pp. 1870–1875). IEEE.

Anderson, H. J., & Odei, S. A. (2018). The influence of public support on university-industry-government collaboration: the case of the Czech Republic, Slovakia, Hungary and Romania. *Statistika*, *98*(4).

Araghi, S., Khosravi, A., & Creighton, D. (2015). Distributed q-learning controller for a multi-intersection traffic network. In *International conference on neural information processing* (pp. 337–344). Springer.

Arel, I., Liu, C., Urbanik, T., & Kohls, A. G. (2010). Reinforcement learning-based multi-agent system for network traffic signal control. *IET Intelligent Transport Systems*, *4*, 128–135.

Aslani, M., Mesgari, M. S., Seipel, S., & Wiering, M. (2019). Developing adaptive traffic signal control by actor–critic and direct exploration methods. In *Proceedings of the institution of civil engineers-transport (Vol. 172)* (pp. 289–298). Thomas Telford Ltd.

Aslani, M., Mesgari, M. S., & Wiering, M. (2017). Adaptive traffic signal control with actor-critic methods in a real-world traffic network with different traffic disruption events. *Transportation Research Part C (Emerging Technologies)*, *85*, 732–752.

Aslani, M., Seipel, S., Mesgari, M. S., & Wiering, M. (2018). Traffic signal optimization through discrete and continuous reinforcement learning with robustness analysis in downtown tehran. *Advanced Engineering Informatics*, *38*, 639–655.

Aslani, M., Seipel, S., & Wiering, M. (2018). Continuous residual reinforcement learning for traffic signal control optimization. *Canadian Journal of Civil Engineering*, *45*, 690–702.

Aziz, H. A., Zhu, F., & Ukkusuri, S. V. (2018). Learning-based traffic signal control algorithms with neighborhood information sharing: An application for sustainable mobility. *Journal of Intelligent Transportation Systems*, *22*, 40–52.

Bagnell, J. A., Ratliff, N., & Zinkevich, M. (2006). Maximum margin planning. In *Proceedings of the international conference on machine learning*. Citeseer.

Baird, L. C., III (1999). *Reinforcement learning through gradient descent: Technical report*, CARNEGIE-MELLON UNIV PITTSBURGH PA DEPT OF COMPUTER SCIENCE.

Bakker, B., Steingrover, M., Schouten, R., Nijhuis, E., Kester, L., et al. (2005). Cooperative multi-agent reinforcement learning of trafficlights.

Balaji, P., German, X., & Srinivasan, D. (2010). Urban traffic signal control using reinforcement learning agents. *IET Intelligent Transport Systems*, *4*, 177–188.

Baldazo, D., Parras, J., & Zazo, S. (2019). Decentralized multi-agent deep reinforcement learning in swarms of drones for flood monitoring. In *2019 27th European signal processing conference* (pp. 1–5). IEEE.

Bazzan, A. (2009). Opportunities for multiagent systems and multiagent reinforcement learning in traffic control. *Autonomous Agents and Multi-Agent Systems*, *18*, 342.

Bazzan, A., De Oliveira, D., & da Silva, B. C. (2010). Learning in groups of traffic signals. *Engineering Applications of Artificial Intelligence*, *23*, 560–568.

Bazzan, A., & Klügl, F. (2014). A review on agent-based technology for traffic and transportation. *The Knowledge Engineering Review*, *29*, 375–403.

Bazzan, A., Klügl, F., & Nagel, K. (2007). Adaptation in games with many co-evolving agents. In *Portuguese conference on artificial intelligence* (pp. 195–206). Springer.

Bellemare, M. G., Naddaf, Y., Veness, J., & Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, *47*, 253–279.

Blockeel, H., & De Raedt, L. (1998). Top-down induction of first-order logical decision trees. *Artificial Intelligence*, *101*, 285–297.

Bouderba, S. I., & Moussa, N. (2019). Reinforcement learning (q-learning) traffic light controller within intersection traffic system. In *Proceedings of the 4th international conference on big data and internet of things* (pp. 1–6).

Box, S., & Waterson, B. (2013). An automated signalized junction controller that learns strategies by temporal difference reinforcement learning. *Engineering Applications of Artificial Intelligence*, *26*, 652–659.

Brys, T., Nowé, A., Kudenko, D., & Taylor, M. (2014). Combining multiple correlated reward and shaping signals by measuring confidence. In *The 28th conference of the American association on artificial intelligence: AAAI 2014*. York.

Butz, M. V., Goldberg, D. E., & Lanzi, P. L. (2005). Computational complexity of the xcs classifier system. In *Foundations of learning classifier systems* (pp. 91–125). Springer.

Cai, C., Wong, C. K., & Heydecker, B. G. (2009). Adaptive traffic signal control using approximate dynamic programming. *Transportation Research Part C: Emerging Technologies*, *17*, 456–474.

Camponogara, E., & Kraus, W. (2003). Distributed learning agents in urban traffic control. In *Portuguese conference on artificial intelligence* (pp. 324–335). Springer.

Cao, Y., Ireson, N., Bull, L., & Miles, R. (1999). Design of a traffic junction controller using classifier system and fuzzy logic. In *International conference on computational intelligence* (pp. 342–353). Springer.

Cao, Y., Ireson, N., Bull, L., & Miles, R. (2000). Distributed learning control of traffic signals. In *Workshops on real-world applications of evolutionary computation* (pp. 117–126). Springer.

Central Intelligence Agency, C. C. R. (2020). URL: https://www.cia.gov/the-world-factbook/field/roadways/country-comparison.

Chanloha, P., Chinrungrueng, J., Usaha, W., & Aswakul, C. (2014). Cell transmission model-based multiagent q-learning for network-scale signal control with transit priority. *The Computer Journal*, *57*, 451–468.

Chen, Y., Yao, J., He, C., Chen, H., & Jin, H. (2017). Adaptive traffic signal control with network-wide coordination. In *International conference on algorithms and architectures for parallel processing* (pp. 180–194). Springer.

Chen, P., Zhu, Z., & Lu, G. (2019). An adaptive control method for arterial signal coordination based on deep reinforcement learning. In *2019 IEEE intelligent transportation systems conference* (pp. 3553–3558). IEEE.

Chin, Y. K., Tham, H. J., Rao, N. K., Bolong, N., & Teo, K. T. K. (2013). Optimization of urban multi-intersection traffic flow via q-learning.

Chong, L., & Osorio, C. (2017). A simulation-based optimization algorithm for dynamic large-scale urban transportation problems. *Transportation Science*, *52*, 637–656.

Choy, M. C., Srinivasan, D., & Cheu, R. L. (2003). Cooperative, hybrid agent architecture for real-time traffic signal control. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, *33*, 597–607.

Choy, M. C., Srinivasan, D., & Cheu, R. L. (2006). Neural networks for continuous online learning and control. *IEEE Transactions on Neural Networks*, *17*, 1511–1531.

Chu, T., Qu, S., & Wang, J. (2016a). Large-scale multi-agent reinforcement learning using image-based state representation. In *2016 IEEE 55th conference on decision and control* (pp. 7592–7597). IEEE.

Chu, T., Qu, S., & Wang, J. (2016b). Large-scale traffic grid signal control with regional reinforcement learning. In *2016 American control conference* (pp. 815–820). IEEE.

Chu, T., & Wang, J. (2017). Traffic signal control by distributed reinforcement learning with min-sum communication. In *2017 American control conference* (pp. 5095–5100). IEEE.

Chu, T., Wang, J., Codecà, L., & Li, Z. (2019). Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, *21*, 1086–1095.

Claus, C., & Boutilier, C. (1998). *The dynamics of reinforcement learning in cooperative multiagent systems* (p. 2). AAAI/IAAI.

Cools, S.-B., Gershenson, C., & D'Hooghe, B. (2013). Self-organizing traffic lights: A realistic simulation. In *Advances in applied self-organizing systems* (pp. 45–55). Springer.

Da Silva, B. C., Basso, E. W., Bazzan, A., & Engel, P. M. (2006). Dealing with non-stationary environments using context detection. In *Proceedings of the 23rd international conference on machine learning* (pp. 217–224).

da Silva, B. C., Bazzan, A., Andriotti, G. K., Lopes, F., & de Oliveira, D. (2004). Itsumo: an intelligent transportation system for urban mobility. In *International workshop on innovative internet community systems* (pp. 224–235). Springer.

Daeichian, A., & Haghani, A. (2018). Fuzzy q-learning-based multi-agent system for intelligent traffic control by a game theory approach. *Arabian Journal for Science and Engineering*, *43*, 3241–3247.

Daeinabi, A., Rahbar, A. G. P., & Khademzadeh, A. (2011). Vwca: An efficient clustering algorithm in vehicular ad hoc networks. *Journal of Network and Computer Applications*, *34*, 207–222.

Daganzo, C. F. (1995). The cell transmission model, part ii: network traffic. *Transportation Research, Part B (Methodological)*, *29*, 79–93.

Dai, Y., Hu, J., Zhao, D., & Zhu, F. (2011). Neural network based online traffic signal controller design with reinforcement training. In *2011 14th international IEEE conference on intelligent transportation systems* (pp. 1045–1050). IEEE.

Dai, Y., Zhao, D., & Yi, J. (2010). A comparative study of urban traffic signal control with reinforcement learning and adaptive dynamic programming. In *The 2010 international joint conference on neural networks* (pp. 1–7). IEEE.

Darmoul, S., Elkosantini, S., Louati, A., & Said, L. B. (2017). Multi-agent immune networks to control interrupted flow at signalized intersections. *Transportation Research Part C: Emerging Technologies*, *82*, 290–313.

Davarynejad, M., Davarynejad, S., Vrancken, J., & van den Berg, J. (2010). Granular value-function approximation for road network traffic control. In *2010 international conference on networking, sensing and control* (pp. 14–19). IEEE.

de Oliveira, D., Bazzan, A., da Silva, B. C., Basso, E. W., Nunes, L., Rossetti, R., et al. (2006). Reinforcement learning based control of traffic lights in non-stationary environments: A case study in a microscopic simulator. In *EUMAS*.

Diakaki, C., Papageorgiou, M., & Aboudolas, K. (2002). A multivariable regulator approach to traffic-responsive network-wide signal control. *Control Engineering Practice*, *10*, 183–195.

Dietterich, T. G., Ashenfelter, A., & Bulatov, Y. (2004). Training conditional random fields via gradient tree boosting. In *Proceedings of the twenty-first international conference on machine learning* (p. 28).

Dowling, J., Cunningham, R., Harrington, A., Curran, E., & Cahill, V. (2004). Emergent consensus in decentralised systems using collaborative reinforcement learning. In *Self-star workshop* (pp. 63–80). Springer.

Dresner, K., & Stone, P. (2008). A multiagent approach to autonomous intersection management. *Journal of Artificial Intelligence Research*, *31*, 591–656.

Duan, Y., Chen, X., Houthooft, R., Schulman, J., & Abbeel, P. (2016). Benchmarking deep reinforcement learning for continuous control. In *International conference on machine learning* (pp. 1329–1338). PMLR.

Dusparic, I., & Cahill, V. (2009a). Distributed w-learning: Multi-policy optimization in self-organizing systems. In *2009 third IEEE international conference on self-adaptive and self-organizing systems* (pp. 20–29). IEEE.

Dusparic, I., & Cahill, V. (2009b). Using distributed w-learning for multi-policy optimization in decentralized autonomic systems, In *Proceedings of the 6th international conference on Autonomic computing* (pp. 63–64).

Dusparic, I., & Cahill, V. (2009c). Using reinforcement learning for multi-policy optimization in decentralized autonomic systems–an experimental evaluation. In *International Conference on autonomic and trusted computing* (pp. 105–119). Springer.

Dusparic, I., & Cahill, V. (2012). Autonomic multi-policy optimization in pervasive systems: Overview and evaluation. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, *7*, 1–25.

Dusparic, I., Monteil, J., & Cahill, V. (2016). Towards autonomic urban traffic control with collaborative multi-policy reinforcement learning. In *2016 IEEE 19th international conference on intelligent transportation systems* (pp. 2065–2070). IEEE.

El Hatri, C., & Boumhidi, J. (2017). Traffic management model for vehicle re-routing and traffic light control based on multi-objective particle swarm optimization. *Intelligent Decision Technologies*, *11*, 199–208.

El-Tantawy, S., & Abdulhai, B. (2010). Towards multi-agent reinforcement learning for integrated network of optimal traffic controllers (marlin-otc). *Transportation Letters*, *2*, 89–110.

El-Tantawy, S., Abdulhai, B., & Abdelgawad, H. (2013). Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (marlin-atsc): methodology and large-scale application on downtown toronto. *IEEE Transactions on Intelligent Transportation Systems*, *14*, 1140–1150.

El-Tantawy, S., Abdulhai, B., & Abdelgawad, H. (2014). Design of reinforcement learning parameters for seamless application of adaptive traffic signal control. *Journal of Intelligent Transportation Systems*, *18*, 227–245.

El-Tantawy, S., Rezaee, K., & Abdulhai, B. (2015). Closed loop optimal adaptive traffic signal and ramp control: A case study on downtown toronto. In *2015 IEEE 18th international conference on intelligent transportation systems* (pp. 2398–2404). IEEE.

Eom, M., & Kim, B.-I. (2020). The traffic signal control problem for intersections: a review. *European Transport Research Review*, *12*, 1–20.

Fagan, D., & Meier, R. (2014). Dynamic multi-agent reinforcement learning for control optimization. In *2014 5th international conference on intelligent systems, modelling and simulation* (pp. 99–104). IEEE.

Fink, A. (2019). *Conducting research literature reviews: From the internet to paper*. Sage publications.

Gaikwad, V. V., Kadarkar, S. S., & Kasbekar, G. S. (2016). Intelligent traffic signal duration adaptation using q-learning with an evolving state space. In *2016 IEEE 84th vehicular technology conference (VTC-Fall)* (pp. 1–6). IEEE.

Gan, X., Guo, H., & Li, Z. (2019). A new multi-agent reinforcement learning method based on evolving dynamic correlation matrix. *IEEE Access*, *7*, 162127–162138.

Gao, R., Liu, Z., Li, J., & Yuan, Q. (2019). Cooperative traffic signal control based on multi-agent reinforcement learning. In *International conference on blockchain and trustworthy systems* (pp. 787–793). Springer.

Gao, J., Shen, Y., Liu, J., Ito, M., & Shiratori, N. (2017). Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network. ArXiv preprint arXiv:1705.02755.

Ge, H., Song, Y., Wu, C., Ren, J., & Tan, G. (2019). Cooperative deep q-learning with *q*-value transfer for multi-intersection signal control. *IEEE Access*, *7*, 40797–40809.

Genders, W., & Razavi, S. (2020). Policy analysis of adaptive traffic signal control using reinforcement learning. *Journal of Computing in Civil Engineering*, *34*, Article 04019046.

Gershenson, C. (2004). Self-organizing traffic lights. ArXiv preprint arXiv:nlin/0411066.

Gong, Y., Abdel-Aty, M., Cai, Q., & Rahman, M. S. (2019). Decentralized network level adaptive signal control by multi-agent deep reinforcement learning. *Transportation Research Interdisciplinary Perspectives*, *1*, Article 100020.

Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). *Deep learning (Vol. 1)*. MIT press Cambridge.

Greenhalgh, T., & Peacock, R. (2005). Effectiveness and efficiency of search methods in systematic reviews of complex evidence: audit of primary sources. *Bmj*, *331*, 1064–1065.

Gregurić, M., Vujić, M., Alexopoulos, C., & Miletić, M. (2020). Application of deep reinforcement learning in traffic signal control: An overview and impact of open traffic data. *Applied Sciences*, *10*, 4011.

Haydari, A., & Yilmaz, Y. (2020). Deep reinforcement learning for intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation Systems*.

Heinen, M. R., Bazzan, A., & Engel, P. M. (2011). Dealing with continuous-state reinforcement learning for intelligent control of traffic signals. In *2011 14th international IEEE conference on intelligent transportation systems* (pp. 890–895). IEEE.

Higuera, C., Lozano, F., Camacho, E. C., & Higuera, C. H. (2019). Multiagent reinforcement learning applied to traffic light signal control. In *International conference on practical applications of agents and multi-agent systems* (pp. 115–126). Springer.

Horsuwan, T., & Aswakul, C. (2019). Reinforcement learning agent under partial observability for traffic light control in presence of gridlocks. In *SUMO* (pp. 29–47).

Houli, D., Zhiheng, L., & Yi, Z. (2010). Multiobjective reinforcement learning for traffic signal control using vehicular ad hoc network. *EURASIP Journal on Advances in Signal Processing*, *2010*, Article 724035.

Huang, R., Hu, J., Huo, Y., & Pei, X. (2019). Cooperative multi-intersection traffic signal control based on deep reinforcement learning. In *CICTP 2019* (pp. 2959–2970).

Humphrys, M. (1996). Action selection methods using reinforcement learning. *From Animals To Animats*, *4*, 135–144.

Hunt, P., Robertson, D., Bretherton, R., & Winton, R. (1981). *SCOOT-a traffic responsive method of coordinating signals*: *Technical report*.

Hüttenrauch, M., Adrian, S., Neumann, G., et al. (2019). Deep reinforcement learning for swarm systems. *Journal of Machine Learning Research*, *20*, 1–31.

Index, T. (2014). URL: https://www.numbeo.com/traffic/rankings_by_country.jsp?title=2020-mid.

Inrix (2020). Scorecard. URL: https://inrix.com/scorecard/.

Iyer, V., Jadhav, R., Mavchi, U., & Abraham, J. (2016). Intelligent traffic signal synchronization using fuzzy logic and q-learning. In *2016 international conference on computing, analytics and security trends* (pp. 156–161). IEEE.

Jácome, L., Benavides, L., Jara, D., Riofrio, G., Alvarado, F., & Pesantez, M. (2018). A survey on intelligent traffic lights. In *2018 IEEE international conference on automation/XXIII congress of the chilean association of automatic control* (pp. 1–6). IEEE.

Jadhao, N. S., & Jadhao, A. S. (2014). Traffic signal control using reinforcement learning. In *2014 fourth international conference on communication systems and network technologies* (pp. 1130–1135). IEEE.

Jadhao, M. N. S., & Kulkarni, M. P. A. (2012). Reinforcement learning based for traffic signal monitoring and management. *International Journal of Engineering Research and Technology*, 1.

Jin, J., & Ma, X. (2017). A multi-objective multi-agent framework for traffic light control. In *2017 11th Asian control conference* (pp. 1199–1204). IEEE.

Jin, J., & Ma, X. (2018). Hierarchical multi-agent control of traffic lights based on collective learning. *Engineering Applications of Artificial Intelligence*, *68*, 236–248.

Jin, J., & Ma, X. (2019). A multi-objective agent-based control approach with application in intelligent traffic signal system. *IEEE Transactions on Intelligent Transportation Systems*, *20*, 3900–3912.

Kao, Y.-C., & Wu, C.-W. (2018). A self-organizing map-based adaptive traffic light control system with reinforcement learning. In *2018 52nd asilomar conference on signals, systems, and computers* (pp. 2060–2064). IEEE.

Keong, C. K. (1993). The glide system—Singapore's urban traffic control system. *Transport Reviews*, *13*, 295–305.

Khamis, M. A., & Gomaa, W. (2012). Enhanced multiagent multi-objective reinforcement learning for urban traffic light control. In *2012 11th international conference on machine learning and applications (Vol. 1)* (pp. 586–591). IEEE.

Khamis, M. A., & Gomaa, W. (2014). Adaptive multi-objective reinforcement learning with hybrid exploration for traffic signal control based on cooperative multi-agent framework. *Engineering Applications of Artificial Intelligence*, *29*, 134–151.

Khamis, M. A., Gomaa, W., & El-Shishiny, H. (2012). Multi-objective traffic light control system based on bayesian probability interpretation. In *2012 15th international IEEE conference on intelligent transportation systems* (pp. 995–1000). IEEE.

Kim, D., & Jeong, O. (2020). Cooperative traffic signal control with traffic flow prediction in multi-intersection. *Sensors*, *20*(137).

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. ArXiv preprint arXiv:1412.6980.

Kitagawa, S., Moustafa, A., & Ito, T. (2019). Urban traffic control using distributed multi-agent deep reinforcement learning. In *Pacific rim international conference on artificial intelligence* (pp. 337–349). Springer.

Kohonen, T. (2012). *Self-organizing maps (Vol. 30)*. Springer Science & Business Media.

Konda, V. R., & Borkar, V. S. (1999). Actor-critic–type learning algorithms for markov decision processes. *SIAM Journal on Control and Optimization*, *38*, 94–123.

Koonce, P., & Rodegerdts, L. (2008). *Traffic signal timing manual*: *Technical report United States*, Federal Highway Administration.

Kristensen, T., & Ezeora, N. J. (2017). Simulation of intelligent traffic control for autonomous vehicles. In *2017 IEEE international conference on information and automation* (pp. 459–465). IEEE.

Kuyer, L., Whiteson, S., Bakker, B., & Vlassis, N. (2008). Multiagent reinforcement learning for urban traffic control using coordination graphs. In *Joint European conference on machine learning and knowledge discovery in databases* (pp. 656–671). Springer.

Lämmer, S., & Helbing, D. (2008). Self-control of traffic lights and vehicle flows in urban road networks. *Journal of Statistical Mechanics: Theory and Experiment*, *2008*, Article P04019.

Lee, J., Chung, J., & Sohn, K. (2020). Reinforcement learning for joint control of traffic signals in a transportation network. *IEEE Transactions on Vehicular Technology*, *69*, 1375–1387.

Lemos, L. L., Bazzan, A., & Pasin, M. (2018). Co-adaptive reinforcement learning in microscopic traffic systems. In *2018 IEEE congress on evolutionary computation* (pp. 1–8). IEEE.

Li, C.-g., Yan, X.-l., Lin, F.-Y., & Zhang, H.-l. (2011). Multi-intersections traffic signal intelligent control using collaborative q-learning algorithm. In *2011 seventh international conference on natural computation (Vol. 1)* (pp. 185–188). IEEE.

Li, C., Yan, F., Zhou, Y., Wu, J., & Wang, X. (2018). A regional traffic signal control strategy with deep reinforcement learning. In *2018 37th Chinese control conference* (pp. 7690–7695). IEEE.

Li, T., Zhao, D., & Yi, J. (2008a). Adaptive dynamic neuro-fuzzy system for traffic signal control. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)* (pp. 1840–1846). IEEE.

Li, T., Zhao, D., & Yi, J. (2008b). Adaptive dynamic programming for multi-intersections traffic signal intelligent control. In *2008 11th international IEEE conference on intelligent transportation systems* (pp. 286–291). IEEE.

Liang, X., Yan, T., Lee, J., & Wang, G. (2018). A distributed intersection management protocol for safety, efficiency, and driver's comfort. *IEEE Internet of Things Journal*, *5*, 1924–1935.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., et al. (2015). Continuous control with deep reinforcement learning. ArXiv preprint arXiv:1509.02971.

Ling, K., & Shalaby, A. S. (2005). A reinforcement learning approach to streetcar bunching control. *Journal of Intelligent Transportation Systems*, *9*, 59–68.

Little, J. D., Kelson, M. D., & Gartner, N. H. (1981). Maxband: A versatile program for setting signals on arteries and triangular networks.

Liu, Z. (2007). A survey of intelligence methods in urban traffic signal control. *IJCSNS International Journal of Computer Science and Network Security*, *7*, 105–112.

Liu, X.-Y., Ding, Z., Borst, S., & Walid, A. (2018). Deep reinforcement learning for intelligent transportation systems. ArXiv preprint arXiv:1812.00979.

Liu, Y., Liu, L., & Chen, W.-P. (2017). Intelligent traffic light control using distributed multi-agent q learning. In *2017 IEEE 20th international conference on intelligent transportation systems* (pp. 1–8). IEEE.

Liu, W., Liu, J., Peng, J., & Zhu, Z. (2014). Cooperative multi-agent traffic signal control system using fast gradient-descent function approximation for v2i networks. In *2014 IEEE international conference on communications* (pp. 2562–2567). IEEE.

Liu, W., Qin, G., He, Y., & Jiang, F. (2017). Distributed cooperative reinforcement learning-based traffic signal control that integrates v2x networks' dynamic clustering. *IEEE Transactions on Vehicular Technology*, *66*, 8667–8681.

Lu, S., Liu, X., & Dai, S. (2008). Adaptive and coordinated traffic signal control based on q-learning and multiband model. In *2008 IEEE conference on cybernetics and intelligent systems* (pp. 765–770). IEEE.

Lu, C., Wen, F., & Gen, M. (2017). Traffic lights dynamic timing algorithm based on reinforcement learning. In *International conference on management science and engineering management* (pp. 1752–1761). Springer.

Lu, W., Zhang, Y., & Xie, Y. (2011). A multi-agent adaptive traffic signal control system using swarm intelligence and neuro-fuzzy reinforcement learning. In *2011 IEEE forum on integrated and sustainable transportation systems* (pp. 233–238). IEEE.

Mannion, P., Duggan, J., & Howley, E. (2016). An experimental review of reinforcement learning algorithms for adaptive traffic signal control. In *Autonomic road transport support systems* (pp. 47–66). Springer.

Marsetič, R., Šemrov, D., & Žura, M. (2014). Road artery traffic light optimization with use of the reinforcement learning. *Promet-Traffic&Transportation*, *26*, 101–108.

Mashayekhi, M., & List, G. (2015). A multiagent auction-based approach for modeling of signalized intersections. In *IJCAI workshops on synergies between multiagent systems, machine learning and complex systems* (pp. 13–24).

Medina, J. C., & Benekohal, R. F. (2012). Traffic signal control using reinforcement learning and the max-plus algorithm as a coordinating strategy. In *2012 15th international IEEE conference on intelligent transportation systems* (pp. 596–601). IEEE.

Medina, J. C., Hajbabaie, A., & Benekohal, R. F. (2010). Arterial traffic control using reinforcement learning agents and information from adjacent intersections in the state and reward structure. In *13th international IEEE conference on intelligent transportation systems* (pp. 525–530). IEEE.

Mikami, S., & Kakazu, Y. (1994). Genetic reinforcement learning for cooperative traffic signal control. In *Proceedings of the first IEEE conference on evolutionary computation. IEEE world congress on computational intelligence* (pp. 223–228). IEEE.

Miller, A. J. (1963). Settings for fixed-cycle traffic signals. *Journal of the Operational Research Society*, *14*, 373–386.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., et al. (2013). Playing atari with deep reinforcement learning. ArXiv preprint arXiv:1312.5602.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*, 529–533.

Moghadam, M. H., & Mozayani, N. (2013). Urban traffic control using adjusted reinforcement learning in a multi-agent system. *Research Journal of Applied Sciences, Engineering and Technology*, *6*, 2943–2950.

Mohajerpoor, R., Saberi, M., & Ramezani, M. (2019). Analytical derivation of the optimal traffic signal timing: Minimizing delay variability and spillback probability for undersaturated intersections. *Transportation Research Part B: Methodological*, *119*, 45–68.

Natarajan, S., Joshi, S., Tadepalli, P., Kersting, K., & Shavlik, J. (2011). Imitation learning in relational domains: A functional-gradient boosting approach. In *IJCAI proceedings-International joint conference on artificial intelligence (Vol. 22)* (p. 1414). Citeseer.

Natarajan, S., Kunapuli, G., Judah, K., Tadepalli, P., Kersting, K., & Shavlik, J. (2010). Multi-agent inverse reinforcement learning. In *2010 ninth international conference on machine learning and applications* (pp. 395–400). IEEE.

Ng, A. Y., Russell, S. J., et al. (2000). Algorithms for inverse reinforcement learning. In *Icml (Vol. 1)* (p. 2).

Ngai, D. C. K., & Yung, N. H. C. (2011). A multiple-goal reinforcement learning method for complex vehicle overtaking maneuvers. *IEEE Transactions on Intelligent Transportation Systems*, *12*, 509–522.

Nguyen, H., Kieu, L.-M., Wen, T., & Cai, C. (2018). Deep learning methods in transportation domain: a review. *IET Intelligent Transport Systems*, *12*, 998–1004.

Ni, W., & Cassidy, M. J. (2019). Cordon control with spatially-varying metering rates: A reinforcement learning approach. *Transportation Research Part C: Emerging Technologies*, *98*, 358–369.

Nishi, T., Otaki, K., Hayakawa, K., & Yoshimura, T. (2018). Traffic signal control based on reinforcement learning with graph convolutional neural nets. In *2018 21st international conference on intelligent transportation systems* (pp. 877–883). IEEE.

Noaeen, M. (2021). *Managing urban traffic networks using data analysis, traffic theory, and deep reinforcement learning*. Canada: University of Calgary.

Noaeen, M., & Far, B. H. (2019). Social media analysis for traffic management. In *Proceedings of the 14th international conference on global software engineering* (pp. 72–73.

Noaeen, M., & Far, B. H. (2020). The efficacy of using social media data for designing traffic management systems. In *2020 4th international workshop on crowd-based requirements engineering* (pp. 11–17). IEEE.

Noaeen, M., Mohajerpoor, R., Far, B. H., & Ramezani, M. (2021). Real-time decentralized traffic signal control for congested urban networks considering queue spillbacks. *Transportation Research Part C: Emerging Technologies*, *133*, Article 103407.

Noaeen, M., Rassafi, A. A., & Far, B. H. (2016). Traffic signal timing optimization by modelling the lost time effect in the shock wavedelay model. In *International Conference on transportation and development 2016* (pp. 397–408).

Nuli, S., & Mathew, T. V. (2013). Online coordination of signals for heterogeneous traffic using stop line detection. *Procedia-Social and Behavioral Sciences*, *104*, 765–774.

Okoli, C., & Schabram, K. (2010). A guide to conducting a systematic literature review of information systems research.

Oroojlooy Jadid, A., & Hajinezhad, D. (2019). A review of cooperative multi-agent deep reinforcement learning. ArXiv preprint arXiv:1908.03963.

Osorio, C., & Selvam, K. K. (2017). Simulation-based optimization: achieving computational efficiency through the use of multiple simulators. *Transportation Science*, *51*, 395–411.

Ozan, C., Baskan, O., Haldenbilen, S., & Ceylan, H. (2015). A modified reinforcement learning algorithm for solving coordinated signalized networks. *Transportation Research Part C: Emerging Technologies*, *54*, 40–55.

Pham, T. T., Brys, T., Taylor, M. E., Brys, T., Drugan, M. M., Bosman, P., et al. (2013). Learning coordinated traffic light control. In *Proceedings of the adaptive and learning agents workshop (At AAMAS-13) (Vol. 10)* (pp. 1196–1201). IEEE.

Prabuchandran, K., An, H. K., & Bhatnagar, S. (2014). Multi-agent reinforcement learning for traffic signal control. In *17th international IEEE conference on intelligent transportation systems* (pp. 2529–2534). IEEE.

Prabuchandran, K., An, H. K., & Bhatnagar, S. (2015). Decentralized learning for traffic signal control. In *2015 7th international conference on communication systems and networks* (pp. 1–6). IEEE.

Prashanth, L., & Bhatnagar, S. (2010). Reinforcement learning with function approximation for traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, *12*, 412–421.

Prashanth, L., & Bhatnagar, S. (2011). Reinforcement learning with average cost for adaptive control of traffic lights at intersections. *2011 14th international ieee conference on intelligent transportation systems*, 1640–1645.

Prashanth, L., & Ghavamzadeh, M. (2016). Variance-constrained actor-critic algorithms for discounted and average reward mdps. *Machine Learning*, *105*, 367–417.

Prashanth, L., Jie, C., Fu, M., Marcus, S., & Szepesvári, C. (2016). Cumulative prospect theory meets reinforcement learning: Prediction and control. In *International conference on machine learning* (pp. 1406–1415).

Prothmann, H., Branke, J., Schmeck, H., Tomforde, S., Rochner, F., Hahner, J., et al. (2009). Organic traffic light control for urban road networks. *International Journal of Autonomous and Adaptive Communications Systems*, *2*, 203–225.

Qu, Z., Pan, Z., Chen, Y., Wang, X., & Li, H. (2020). A distributed control method for urban networks using multi-agent reinforcement learning based on regional mixed strategy nash-equilibrium. *IEEE Access*, *8*, 19750–19766.

Reda, M., Mountassir, F., & Mohamed, B. (2019). Introduction to coordinated deep agents for traffic signal. In *2019 international conference on wireless technologies, embedded and intelligent systems* (pp. 1–6). IEEE.

Richter, S., Aberdeen, D., & Yu, J. (2007). Natural actor-critic for road traffic optimisation. *Advances in Neural Information Processing Systems*, 1169–1176.

Richter, S., et al. (2006). Learning road traffic control: towards practical traffic control using policy gradients.

Riedmiller, M. (2005). Neural fitted q iteration–first experiences with a data efficient neural reinforcement learning method. In *European conference on machine learning* (pp. 317–328). Springer.

Ritcher, S. (2007). Traffic light scheduling using policy-gradient reinforcement learning. In *The international conference on automated planning and scheduling*.

Rizzo, S. G., Vantini, G., & Chawla, S. (2019a). Reinforcement learning with explainability for traffic signal control. In *2019 IEEE intelligent transportation systems conference* (pp. 3567–3572). IEEE.

Rizzo, S. G., Vantini, G., & Chawla, S. (2019b). Time critic policy gradient methods for traffic signal control in complex and congestedscenarios. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1654–1664).

Robertson, D. I. (1969). Transyt: a traffic network study tool.

Rosyadi, A. R., Wirayuda, T. A. B., & Al-Faraby, S. (2016). Intelligent traffic light control using collaborative q-learning algorithms. In *2016 4th international conference on information and communication technology* (pp. 1–6). IEEE.

Sadigh, D., Kim, E. S., Coogan, S., Sastry, S. S., & Seshia, S. A. (2014). A learning based approach to control synthesis of markov decision processes for linear temporal logic specifications. In *53rd IEEE conference on decision and control* (pp. 1091–1096). IEEE.

Salkham, A. a., & Cahill, V. (2010). Soilse: A decentralized approach to optimization of fluctuating urban traffic using reinforcement learning. *13th international IEEE conference on intelligent transportation systems*, 531–538.

Salkham, A. a., Cunningham, R., Garg, A., & Cahill, V. (2008). A collaborative reinforcement learning approach to urban traffic control optimization. In *2008 IEEE/WIC/ACM international conference on web intelligence and intelligent agent technology (Vol. 2)* (pp. 560–566). IEEE.

Schutera, M., Goby, N., Smolarek, S., & Reischl, M. (2018). Distributed traffic light control at uncoupled intersections with real-world topology by deep reinforcement learning. ArXiv preprint arXiv:1811.11233.

Shabestray, S. M. A., & Abdulhai, B. (2019). Multimodal intelligent deep (mind) traffic signal controller. In *2019 IEEE intelligent transportation systems conference* (pp. 4532–4539). IEEE.

Shen, M. (2016). A fast method to prevent traffic blockage by signal control based on reinforcement learning. In *International conference on communication and electronic information engineering*. Atlantis Press.

Shi, S., & Chen, F. (2018). Deep recurrent q-learning method for area traffic coordination control. *Journal of Advances in Mathematics and Computer Science*, 1–11.

Shu, L., Wu, J., & Li, Z. (2019). Hierarchical regional control for traffic grid signal optimization. In *2019 IEEE intelligent transportation systems conference* (pp. 3547–3552). IEEE.

Sims, A., et al. (1981). Scat the sydney co-ordinated adaptive traffic system. In *Symposium on computer control of transport 1981: Preprints of papers* (p. 22). Australia: Institution of Engineers.

Song, J., Jin, Z., & Zhu, W. (2011). Implementing traffic signal optimal control by multiagent reinforcement learning. In *Proceedings of 2011 international conference on computer science and network technology (Vol. 4)* (pp. 2578–2582). IEEE.

Spall, J. C., & Chin, D. C. (1997). Traffic-responsive signal timing for system-wide traffic control. *Transportation Research Part C: Emerging Technologies*, *5*, 153–163.

Srinivasan, D., & Choy, M. C. (2007). Distributed problem solving using evolutionary learning in multi-agent systems. In *Advances in evolutionary computing for system design* (pp. 211–227). Springer.

Srinivasan, D., Choy, M. C., & Cheu, R. L. (2006). Neural networks for real-time traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, *7*, 261–272.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, *15*, 1929–1958.

Su, S., & Tham, C.-K. (2007). Sensorgrid for real-time traffic management. In *2007 3rd international conference on intelligent sensors, sensor networks and information* (pp. 443–448). IEEE.

Sutton, R. S., & Barto, A. G. (1998). Learning: an introduction.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT Press.

Sutton, R. S., Barto, A. G., et al. (1998). *Introduction to reinforcement learning (Vol. 135)*. MIT press Cambridge.

Tahifa, M., Boumhidi, J., & Yahyaouy, A. (2015). Swarm reinforcement learning for traffic signal control based on cooperative multi-agent framework. In *2015 intelligent systems and computer vision (ISCV)* (pp. 1–6). IEEE.

Tahilyani, S., Darbari, M., & Shukla, P. (2013). Soft computing approaches in traffic control systems: a review. *Aasri Procedia*, *4*, 206–211.

Tan, M. (1993). Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the tenth international conference on machine learning* (pp. 330–337).

Tan, T., Bao, F., Deng, Y., Jin, A., Dai, Q., & Wang, J. (2019). Cooperative deep reinforcement learning for large-scale traffic grid signal control. *IEEE Transactions on Cybernetics*, *50*, 2687–2700.

Teo, K. T. K., Yeo, K. B., Chin, Y. K., Chuo, H. S. E., & Tan, M. K. (2014). Agent-based optimization for multiple signalized intersections using q-learning. *International Journal of Simulation: Systems, Science & Technology*, *15*, 90–96.

Torabi, B., Al-Zinati, M., & Wenkstern, R. Z. (2018). Matisse 3.0: A large-scale multi-agent simulation system for intelligent transportation systems. In *International conference on practical applications of agents and multi-agent systems* (pp. 357–360). Springer.

Torabi, B., Wenkstern, R. Z., & Saylor, R. (2018). A self-adaptive collaborative multi-agent based traffic signal timing system. In *2018 IEEE international smart cities conference* (pp. 1–8). IEEE.

Tricco, A. C., Lillie, E., Zarin, W., O'Brien, K. K., Colquhoun, H., Levac, D., et al. (2018). Prisma extension for scoping reviews (prisma-scr): checklist and explanation. *Annals of Internal Medicine*, *169*, 467–473.

Urbanik, T., Tanaka, A., Lozner, B., Lindstrom, E., Lee, K., Quayle, S., et al. (2015). *Signal timing manual (Vol. 1)*. DC: Transportation Research Board Washington.

US Department of Transportation (2008). Ngsim - next generation simulation. URL: https://ops.fhwa.dot.gov/trafficanalysistools/ngsim.htm.

Van der Pol, E., & Oliehoek, F. A. (2016). Coordinated deep reinforcement learners for traffic light control. In *Proceedings of learning, inference and control of multi-agent systems (at NIPS 2016)*.

Varaiya, P. (2013). Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, *36*, 177–195.

Vidhate, D. A., & Kulkarni, P. (2017). Exploring cooperative multi-agent reinforcement learning algorithm (cmrla) for intelligent traffic signal control. In *International conference on smart trends for information technology and computer communications* (pp. 71–81). Springer.

Vinitsky, E., Kreidieh, A., L. Flem, L., Kheterpal, N., Jang, K., Wu, C., et al. (2018). Benchmarks for reinforcement learning in mixed-autonomy traffic. In *Conference on robot learning* (pp. 399–409).

Vu, H., Aknine, S., & Ramchurn, S. D. (2018). A decentralised approach to intersection traffic management. *IJCAI*, 527–533.

Wang, Y., Geng, S., & Li, Q. (2016). Intelligent transportation control based on proactive complex event processing. In *MATEC web of conferences (Vol. 77)* (p. 09004). EDP Sciences.

Wang, Z., Liu, L., Zhou, M., & Ansari, N. (2008). A position-based clustering technique for ad hoc intervehicle communication. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, *38*, 201–208.

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (pp. 1995–2003). PMLR.

Wang, Y., Yang, X., Liang, H., & Liu, Y. (2018). A review of the self-adaptive traffic signal control system based on future traffic environment. *Journal of Advanced Transportation*.

Wang, Y., Yang, X., Liu, Y., & Liang, H. (2018). Evaluation and application of urban traffic signal optimizing control strategy based on reinforcement learning. *Journal of Advanced Transportation*.

Wang, Y., Zhang, D., Liu, Y., Dai, B., & Lee, L. H. (2019). Enhancing transportation systems via deep learning: A survey. *Transportation Research Part C: Emerging Technologies*, *99*, 144–163.

Waskow, S. J., & Bazzan, A. (2010). Improving space representation in multiagent learning via tile coding. In *Brazilian symposium on artificial intelligence* (pp. 153–162). Springer.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, *8*, 279–292.

Webster, F. V. (1958). *Traffic signal settings: Technical report*.

Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., et al. (2019). Presslight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1290–1298).

Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., et al. (2019). Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 1913–1922).

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019). A survey on traffic signal control methods. ArXiv preprint arXiv:1904.08117.

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2021). Recent advances in reinforcement learning for traffic signal control: A survey of models and evaluation. *ACM SIGKDD Explorations Newsletter*, *22*, 12–18.

Wei, H., Zheng, G., Yao, H., & Li, Z. (2018). Intellilight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2496–2505).

Wiering, M. A. (2000). Multi-agent reinforcement learning for traffic light control. In *Machine Learning: Proceedings of the seventeenth international conference* (pp. 1151–1158).

Wiering, M., Vreeken, J., Van Veenen, J., & Koopman, A. (2004). Simulation and optimization of traffic in a city. In *IEEE intelligent vehicles symposium, 2004* (pp. 453–458). IEEE.

Wu, C., Kreidieh, A., Parvate, K., Vinitsky, E., & Bayen, A. M. (2017). Flow: Architecture and benchmarking for reinforcement learning in traffic control. (p. 10). ArXiv preprint arXiv:1710.05465.

Wu, Q., Shen, J., Yong, B., Wu, J., Li, F., Wang, J., et al. (2019). Smart fog based workflow for traffic control networks. *Future Generation Computer Systems*, *97*, 825–835.

Wu, W., Shufeng, G., & Hongxiu, L. (2009). A coordinated urban traffic signal control approach based on multi-agent. In *2009 international conference on intelligent engineering systems* (pp. 263–267). IEEE.

Wunderlich, R., Elhanany, I., & Urbanik, T. (2007). A stable longest queue first signal scheduling algorithm for an isolated intersection. In *2007 IEEE international conference on vehicular electronics and safety* (pp. 1–6). IEEE.

Xiang, J., & Chen, Z. (2015). Adaptive traffic signal control of bottleneck subzone based on grey qualitative reinforcement learning algorithm. In *ICPRAM (2)* (pp. 295–301).

Xinhai, X., & Lunhui, X. (2009). Traffic signal control agent interaction model based on game theory and reinforcement learning. In *2009 international forum on computer science-technology and applications (Vol. 1)* (pp. 164–168). IEEE.

Xu, M., Wu, J., Huang, L., Zhou, R., Wang, T., & Hu, D. (2020). Network-wide traffic signal control based on the discovery of critical nodes and deep reinforcement learning. *Journal of Intelligent Transportation Systems*, *24*, 1–10.

Xu, L.-H., Xia, X.-H., & Luo, Q. (2013). The study of reinforcement learning for traffic self-adaptive control under multiagent markov game environment. *Mathematical Problems in Engineering*.

Xu, N., Zheng, G., Xu, K., Zhu, Y., & Li, Z. (2019). Targeted knowledge transfer for learning traffic signal plans. In *Pacific-Asia conference on knowledge discovery and data mining* (pp. 175–187). Springer.

Xu, W., Zhong, D., Wu, S., & Ni, H. (2015). A control method of traffic flow based on region coordination. In *2015 international conference on management, education, information and control*. Atlantis Press.

Yang, S., Yang, B., Wong, H.-S., & Kang, Z. (2019). Cooperative traffic signal control using multi-step return and off-policy asynchronous advantage actor-critic graph algorithm. *Knowledge-Based Systems*, *183*, Article 104855.

Yau, K.-L. A., Qadir, J., Khoo, H. L., Ling, M. H., & Komisarczuk, P. (2017). A survey on reinforcement learning models and algorithms for traffic signal control. *ACM Computing Surveys (CSUR)*, *50*, 1–38.

Yen, G., Yang, F., & Hickey, T. (2002). Coordination of exploration and exploitation in a dynamic environment. *International Journal of Smart Engineering System Design*, *4*, 177–182.

Yin, B., Dridi, M., & E. Moudni, A. (2015). Adaptive traffic signal control for multi-intersection based on microscopic model. In *2015 IEEE 27th international conference on tools with artificial intelligence* (pp. 49–55). IEEE.

Yin, B., Dridi, M., & E. Moudni, A. (2016). Traffic network micro-simulation model and control algorithm based on approximate dynamic programming. *IET Intelligent Transport Systems*, *10*, 186–196.

Zhang, X., Aberdeen, D., & Vishwanathan, S. (2007). Conditional random fields for multi-agent reinforcement learning. In *Proceedings of the 24th international conference on machine learning* (pp. 1143–1150).

Zhao, D., Dai, Y., & Zhang, Z. (2011). Computational intelligence in urban traffic signal control: A survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, *42*, 485–494.

Zhao, X., Li, Z., Yu, Q., & Shang, Y. (2009). A study of the cooperation control of two adjacent intersections based on nbs game q-learning algorithm. In *2009 fifth international conference on natural computation (Vol. 6)* (pp. 551–557). IEEE.

Zhao, Y., Ma, J., Shen, L., & Qian, Y. (2020). Optimizing the junction-tree-based reinforcement learning algorithm for network-wide signal coordination. *Journal of Advanced Transportation*.

Zheng, G., Xiong, Y., Zang, X., Feng, J., Wei, H., Zhang, H., et al. (2019). Learning phase competition for traffic signal control. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 1963–1972).

Zheng, G., Zang, X., Xu, N., Wei, H., Yu, Z., Gayah, V., et al. (2019). Diagnosing reinforcement learning for traffic signal control. ArXiv preprint arXiv:1905.04716.

Zhou, P., Braud, T., Alhilal, A., Hui, P., & Kangasharju, J. (2019). Erl: Edge based reinforcement learning for optimized urban traffic light control. In *2019 IEEe international conference on pervasive computing and communications workshops* (pp. 849–854). IEEE.

Zhu, F., Aziz, H. A., Qian, X., & Ukkusuri, S. V. (2015). A junction-tree based learning algorithm to optimize network wide traffic control: A coordinated multi-agent framework. *Transportation Research Part C: Emerging Technologies*, *58*, 487–501.